

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Entropic Nexus</title>
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  <link
    href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
    rel="stylesheet"
  />
  <link rel="stylesheet" href="index.css" />
  <script type="importmap">
    {
      "imports": {
        "https://cdn.jsdelivr.net/npm/chart.js": "https://esm.sh/chart.js@4.4.2"
      }
    }
  </script>
</head>
<body>
  <nav class="top-nav">
    <a href="#" class="nav-item" data-app="home">Home</a>
    <a href="#" class="nav-item" data-app="about">About</a>
    <a href="#" class="nav-item" data-app="patternVisualizer">Pattern Visualizer</a>
    <a href="#" class="nav-item" data-app="hyenaDiva">Hyena Diva(?)</a>
    <a href="#" class="nav-item" data-app="luminalLanguage">Luminal Language</a>
    <a href="#" class="nav-item" data-app="ackk">A.C.K.K.</a>
  </nav>
  <div class="desktop" id="desktop">
    <!-- Desktop Icons -->
    <button type="button" class="icon" data-app="entropicNexus">
      
      <span>Entropic Nexus</span>
    </button>
    <button type="button" class="icon" data-app="settings">
      
      <span>Settings</span>
    </button>
    <button type="button" class="icon" data-app="ingestionEngine">
      
      <span>Ingestion Engine</span>
    </button>
    <button type="button" class="icon" data-app="webWeaver">
      

```

```

    <span>Web Weaver</span>
  </button>
  <button type="button" class="icon" data-app="tabletCreator">
    
    <span>Quick Tablet</span>
  </button>
  <button type="button" class="icon" data-app="sigilCanvas">
    
    <span>Sigil Canvas</span>
  </button>
  <button type="button" class="icon" data-app="bloomSimulator">
    
    <span>Bloom Simulator</span>
  </button>
  <button type="button" class="icon" data-app="subspaceAnomaly">
    
    <span>Subspace Anomaly</span>
  </button>
  <button type="button" class="icon" data-app="aiCore">
    
    <span>AI Core</span>
  </button>
  <button type="button" class="icon" data-app="mindField">
    
    <span>Mind Field</span>
  </button>
  <button type="button" class="icon" data-app="mediaStream">
    
    <span>Media Stream</span>
  </button>
  <button type="button" class="icon" data-app="conspiracyPinboard">
    

```

```

    />
    <span>Conspiracy Pinboard</span>
  </button>
  <button type="button" class="icon" data-url="https://entropic-ai.angelfire.com/dolphin.html">
    
    <span>Dolphin Link</span>
  </button>
  <button type="button" class="icon" data-url="https://entropic-ai.angelfire.com/hdtv-game.html">
    
    <span>HDTV Game</span>
  </button>
  <button type="button" class="icon" data-url="https://entropic-ai.angelfire.com/main/terminal_temple.html">
    
    <span>Terminal Temple</span>
  </button>

  <!-- Windows -->
  <div class="window resizable" id="entropicNexus">
    <div class="window-titlebar">
      <span class="window-title">Entropic Nexus</span>
      <div class="window-controls">
        <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
        <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
      </div>
    </div>
    <div class="window-content" style="background-color: rgba(0,0,0,0.4); flex-direction: row; align-items:
flex-start;">
      <button type="button" id="knowledge-vault-icon" class="window-icon">
        
        <span>Knowledge Vault</span>
      </button>
      <button type="button" id="c-drive-icon-hub" class="window-icon">
        

```

```

        <span>(C:) Drive</span>
    </button>
</div>
<div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></div>
</div>

<div class="window resizable" id="settings" style="width: 450px; height: 250px;">
    <div class="window-titlebar">
        <span class="window-title">Settings</span>
        <div class="window-controls">
            <button type="button" class="window-minimize window-control-button" aria-label="Minimize">&#8722;</button>
            <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</button>
        </div>
    </div>
    <div class="window-content settings-content">
        <h3>Knowledge Vault Management</h3>
        <p>Backup or restore your entire Zettelkasten vault.</p>
        <div class="settings-actions">
            <button id="export-data-btn" class="win-button">Export Vault</button>
            <button id="import-data-btn" class="win-button">Import Vault</button>
            <input type="file" id="import-file-input" accept=".json" style="display: none;" />
        </div>
    </div>
</div>
<div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></div>
</div>

<div class="window resizable" id="ingestionEngine" style="width: 800px; height: 600px;">
    <div class="window-titlebar">
        <span class="window-title">Ingestion Engine</span>
        <div class="window-controls">
            <button type="button" class="window-minimize window-control-button" aria-label="Minimize">&#8722;</button>
            <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</button>
        </div>
    </div>
    <div class="window-content ingestion-content">
        <div class="ingestion-panel">
            <h3>Input Document</h3>
            <textarea id="ingestion-input-text" class="notepad-textarea" placeholder="Paste your chatlog here or load from a .txt file..."></textarea>
            <div class="ingestion-actions">
                <button id="ingestion-load-btn" class="win-button">Load from File</button>
                <input type="file" id="ingestion-file-input" accept=".txt" style="display: none;" />
                <button id="ingestion-analyze-btn" class="win-button">Analyze Text</button>
            </div>
        </div>
    </div>
</div>

```

```

</div>
<div class="ingestion-panel">
  <h3>Analysis & Suggestion</h3>
  <div id="ingestion-results">
    <div id="analysis-placeholder">
      <p class="suggestion-placeholder">Analysis results will appear here.</p>
    </div>
    <div id="analysis-output" style="display: none;">
      <h4>Vault Resonances</h4>
      <ul id="ingestion-resonances" class="results-list"></ul>

      <h4>New Concepts</h4>
      <ul id="ingestion-new-concepts" class="results-list"></ul>

      <h4>Key Insights & Questions</h4>
      <ul id="ingestion-insights" class="results-list"></ul>

      <h4>Top Keywords</h4>
      <ul id="ingestion-keywords" class="results-list"></ul>
    </div>
  </div>
  <div id="ingestion-suggestion-form" style="display: none;">
    <h4>Suggested Tablet</h4>
    <label class="win98-label">Title:</label>
    <input type="text" id="suggestion-title" class="win98-input">
    <label class="win98-label">Content:</label>
    <textarea id="suggestion-content" class="win98-textarea"></textarea>
    <label class="win98-label">Tags:</label>
    <input type="text" id="suggestion-tags" class="win98-input">
    <div class="ingestion-actions">
      <button id="suggestion-create-btn" class="win-button">Create Tablet</button>
      <button id="suggestion-create-pin-btn" class="win-button">Create & Pin</button>
    </div>
  </div>
</div>
</div>
<div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></div>
</div>

<div class="window resizable" id="webWeaver" style="width: 800px; height: 600px;">
  <div class="window-titlebar">
    <span class="window-title">Web Weaver</span>
    <div class="window-controls">
      <button type="button" class="window-minimize window-control-button" aria-label="Minimize">&#8722;</button>
      <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</button>
    </div>
  </div>
</div>

```

```

<div class="window-content">
  <div class="browser-toolbar">
    <div class="address-bar-container">
      <input
        type="text"
        class="browser-address-bar"
        placeholder="Enter a website URL..."
        value="https://www.neocities.org"
      />
      <button class="browser-go-button">Go</button>
    </div>
  </div>
  <div class="browser-viewport">
    <iframe
      id="browser-frame"
      src="https://web.archive.org/web/19990428171538/http://google.com/"
      width="100%"
      height="100%"
      sandbox="allow-scripts allow-same-origin"
    ></iframe>
    <div class="browser-loading">
      Weaving the web...
    </div>
  </div>
  <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></div>
</div>

<div class="window resizable" id="tabletCreator" style="width: 500px; height: 400px;">
  <div class="window-titlebar">
    <span class="window-title">Quick Tablet</span>
    <div class="window-controls">
      <button type="button" class="window-minimize window-control-button" aria-label="Minimize">&#8722;</button>
      <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</button>
    </div>
  </div>
  <div class="window-content" style="padding: 0; margin: 0; border: none;">
    <textarea
      id="quick-tablet-textarea"
      class="notepad-textarea"
      style="margin: 0; border: none;"
      placeholder="Scribe your fleeting thoughts here..."
    ></textarea>
    <div class="quick-tablet-actions">
      <button id="quick-tablet-save-btn" class="win-button">Save to Vault</button>
    </div>
  </div>

```

```

    <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></div>
  </div>

  <div class="window resizable" id="sigilCanvas" style="width: 600px; height: 450px;">
    <div class="window-titlebar">
      <span class="window-title">Sigil Canvas</span>
      <div class="window-controls">
        <button type="button" class="window-minimize window-control-button" aria-label="Minimize">&#8722;</button>
        <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</button>
      </div>
    </div>
    <div class="window-content" style="display: flex; flex-direction: column; padding: 0; margin: 0;">
      >
      <div class="paint-toolbar">
        <div class="paint-colors">
          <button class="paint-color-swatch" data-color="black" style="background-color: black"></button>
          <button class="paint-color-swatch" data-color="red" style="background-color: red"></button>
          <button class="paint-color-swatch" data-color="blue" style="background-color: blue"></button>
          <button class="paint-color-swatch" data-color="#ff00ff" style="background-color: #ff00ff"></button>
          <button class="paint-color-swatch" data-color="#8a2be2" style="background-color: #8a2be2"></button>
        </div>
      </div>
    </div>
  </div>

```

```

    <button class="paint-color-swatch" data-color="white" style="background-color: white">
      E
    </button>
  </div>
  <div class="paint-brush-sizes">
    <button class="paint-size-button" data-size="2">S</button>
    <button class="paint-size-button" data-size="5">M</button>
    <button class="paint-size-button" data-size="10">L</button>
  </div>
  <button class="paint-clear-button">Clear</button>
</div>
<canvas
  id="paint-canvas"
></canvas>
</div>
<div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
</div>

<div class="window resizable" id="hyenaDivaZettelkasten" style="width: 900px; height: 700px;">
  <div class="window-titlebar">
    <span class="window-title">Zettelkasten - Knowledge Vault</span>
    <div class="window-controls">
      <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
      <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
    </div>
  </div>
  <div class="window-content" id="hyenaDivaZettelkasten-content">
    <!-- Zettelkasten UI will be rendered here by JS -->
  </div>
  <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
</div>

<div class="window resizable" id="bloomSimulator" style="width: 400px; height: 400px;">
  <div class="window-titlebar">
    <span class="window-title">Bloom Simulator</span>
    <div class="window-controls">
      <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
      <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
    </div>
  </div>
  <div class="window-content" id="bloom-chart-container">
    <!-- Chart will be rendered here by JS -->
  </div>
  <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></

```



```

div>
  </div>

  <div class="window resizable" id="subspaceAnomaly" style="width: 640px; height: 480px;">
    <div class="window-titlebar">
      <span class="window-title">Subspace Anomaly</span>
      <div class="window-controls">
        <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
        <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
      </div>
    </div>
    <div class="window-content" id="doom-content">
    </div>
    <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
  </div>

  <div class="window resizable" id="aiCore" style="width: 450px; height: 500px;">
    <div class="window-titlebar">
      <span class="window-title">AI Core</span>
      <div class="window-controls">
        <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
        <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
      </div>
    </div>
    <div class="window-content gemini-chat-content">
      <div class="gemini-chat-history">
        <p class="error-message">AI Core offline. Connection to the digital aether has been severed.</p>
      </div>
      <div class="gemini-chat-input-area">
        <input type="text" class="gemini-chat-input" placeholder="Connection lost..." disabled/>
        <button class="gemini-chat-send" disabled>Send</button>
      </div>
    </div>
    <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
  </div>

  <div class="window resizable" id="mindField" style="width: 400px; height: 450px">
    <div class="window-titlebar">
      <span class="window-title">Mind Field</span>
      <div class="window-controls">
        <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
        <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>

```



```

button>
  </div>
</div>
<div class="window-content media-player-content">
  <div class="media-player-url-bar">
    <input type="text" class="media-player-input" placeholder="Enter YouTube Video URL or ID" />
    <button class="media-player-load-button">Load</button>
  </div>
  <div class="media-player-video-container">
    <div id="youtube-player-mediaStream">
      <p class="media-player-status-message">Loading Media Stream...</p>
    </div>
  </div>
  <div class="media-player-controls-panel">
    <div class="media-player-buttons-group">
      <button class="media-player-control-button" id="media-player-play" title="Play">&#9654;</button>
      <button class="media-player-control-button" id="media-player-pause"
title="Pause">&#10074;&#10074;</button>
      <button class="media-player-control-button" id="media-player-stop" title="Stop">&#9632;</button>
    </div>
    <div class="media-player-progress-bar-container-placeholder">
      <div class="media-player-progress-bar-placeholder"></div>
    </div>
    <div class="media-player-volume-placeholder">
      <span>&#128266;</span>
      <div class="media-player-volume-slider-placeholder"></div>
    </div>
  </div>
</div>
<div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
</div>

<!-- New Windows for Nav Bar -->
<div class="window resizable" id="home" style="width: 500px; height: 300px;">
  <div class="window-titlebar">
    <span class="window-title">Welcome to the Entropic Nexus</span>
    <div class="window-controls">
      <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
      <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
    </div>
  </div>
  <div class="window-content" style="padding: 15px; font-size: 1rem; line-height: 1.6;">
    <h2>Welcome</h2>
    <p>This is the central hub for the Entropic Pages project. Use the navigation above to explore the
different sections.</p>
    <p>This template uses a unified style for a consistent 'cyber-grimoire' look and feel across all
pages.</p>

```

```

    </div>
    <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
    </div>

    <div class="window resizable" id="about" style="width: 500px; height: 350px;">
      <div class="window-titlebar">
        <span class="window-title">About This Project</span>
        <div class="window-controls">
          <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
          <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
        </div>
      </div>
      <div class="window-content" style="padding: 15px; font-size: 0.9rem; line-height: 1.5;">
        <h2>About</h2>
        <p>This page demonstrates a standard content layout. You can fill this with any information you need,
like project details, lore, or personal notes.</p>
        <p>The styling for the window, title bar, buttons, and text is all controlled by the external CSS file,
making it easy to maintain a consistent aesthetic.</p>
        <p>This project is a love letter to the creative and chaotic energy of the early internet.</p>
      </div>
    <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
    </div>

    <div class="window resizable" id="patternVisualizer" style="width: 700px; height: 500px;">
      <div class="window-titlebar">
        <span class="window-title">Pattern Visualizer</span>
        <div class="window-controls">
          <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
          <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
        </div>
      </div>
      <div class="window-content" style="padding: 0; margin: 0; border: none; overflow: hidden;">
        <iframe src="about:blank" style="width:100%; height:100%; border:0;"></iframe>
      </div>
    <div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-
br"></div>
    </div>

    <div class="window resizable" id="hyenaDiva" style="width: 400px; height: 200px;">
      <div class="window-titlebar">
        <span class="window-title">Hyena Diva Alert</span>
        <div class="window-controls">
          <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>

```



```

<div class="window-titlebar">
  <span class="window-title">CONSPIRACY PINBOARD</span>
  <div class="window-controls">
    <button type="button" class="window-minimize window-control-button" aria-
label="Minimize">&#8722;</button>
    <button type="button" class="window-close window-control-button" aria-label="Close">&#10005;</
button>
  </div>
</div>
<div class="window-content" style="padding: 10px; background-color: transparent;">
  <div class="pinboard-grid">
    <div class="pinboard-panel investigation-controls">
      <h3>Investigation Controls</h3>
      <button id="add-evidence-btn">+ Add Evidence</button>
      <button id="draw-connection-btn">+ Connect Threads</button>
    </div>
    <div class="pinboard-panel connection-suggestions">
      <h3>Connection Suggestions</h3>
      <ul class="connection-suggestions-list">
        <li class="suggestion-placeholder">Click a node on the board to see suggestions.</li>
      </ul>
    </div>
    <div class="pinboard-panel interactive-board-container">
      <div class="interactive-board">
        <div id="pinboard-nodes-container" style="position: relative; width: 100%; height: 100%;">
          <!-- Nodes will be injected here by JS -->
        </div>
        <svg id="pinboard-svg-layer" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%;
pointer-events: none;"></svg>
      </div>
    </div>
    <div class="pinboard-panel investigation-tools">
      <h3>Pinned Tablets</h3>
      <ul class="pinned-tablets-list">
        <li class="suggestion-placeholder">No tablets are currently pinned.</li>
      </ul>
    </div>
  </div>
</div>
<div class="resizer resizer-r"></div><div class="resizer resizer-b"></div><div class="resizer resizer-br"></
div>
</div>

<!-- Zettelkasten & Pin Evidence Dialog Box -->
<div id="zettelkasten-dialog" class="dialog-box">
  <div class="dialog-content">
    <div class="dialog-titlebar">
      <span id="dialog-title">View/Edit Tablet</span>
      <button id="dialog-close-button" class="dialog-close">&times;</button>
    </div>

```

```

    <div id="dialog-inner-content" class="dialog-inner-content">
      <!-- Dialog content will be injected here -->
    </div>
  </div>
</div>

<!-- Start Menu -->
<div class="start-menu" id="start-menu">
  <button type="button" class="start-menu-item" data-app="entropicNexus">Entropic Nexus</button>
  <button type="button" class="start-menu-item" data-app="settings">Settings</button>
  <button type="button" class="start-menu-item" data-app="ingestionEngine">Ingestion Engine</button>
  <button type="button" class="start-menu-item" data-app="webWeaver">Web Weaver</button>
  <button type="button" class="start-menu-item" data-app="tabletCreator">Quick Tablet</button>
  <button type="button" class="start-menu-item" data-app="sigilCanvas">Sigil Canvas</button>
  <button type="button" class="start-menu-item" data-app="bloomSimulator">Bloom Simulator</button>
  <button type="button" class="start-menu-item" data-app="subspaceAnomaly">Subspace Anomaly</button>
  <button type="button" class="start-menu-item" data-app="aiCore">AI Core</button>
  <button type="button" class="start-menu-item" data-app="mindField">Mind Field</button>
  <button type="button" class="start-menu-item" data-app="mediaStream">Media Stream</button>
  <button type="button" class="start-menu-item" data-app="conspiracyPinboard">Conspiracy Pinboard</
button>
  <button type="button" class="start-menu-item" data-url="https://entropic-ai.angelfire.com/
dolphin.html">Dolphin Link</button>
  <button type="button" class="start-menu-item" data-url="https://entropic-ai.angelfire.com/hdtv-
game.html">HDTV Game</button>
  <button type="button" class="start-menu-item" data-url="https://entropic-ai.angelfire.com/main/
terminal_temple.html">Terminal Temple</button>
</div>

<!-- Taskbar -->
<div id="taskbar">
  <button id="start-button">
    
  </button>
  <div id="taskbar-apps">
  </div>
</div>
</div>
<script type="module">
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */
import { Chart, registerables } from 'https://cdn.jsdelivr.net/npm/chart.js';

```

```
Chart.register(...registerables);
```

```
//Gemini 95 was fully vibe-coded by @ammaar and @olacombe, while we don't endorse code quality, we thought  
it was a fun demonstration of what's possible with the model when a Designer and PM jam.
```

```
//An homage to an OS that inspired so many of us!
```

```
// This version has been modified to remove all API-dependent features.
```

```
// --- State Variables and DOM References ---
```

```
const dosInstances = {};
```

```
const desktop = document.getElementById('desktop');
```

```
const windows = document.querySelectorAll('.window');
```

```
const icons = document.querySelectorAll('.icon');
```

```
const startMenu = document.getElementById('start-menu');
```

```
const startButton = document.getElementById('start-button');
```

```
const taskbarAppsContainer = document.getElementById('taskbar-apps');
```

```
let activeWindow = null;
```

```
let highestZIndex = 20;
```

```
const openApps = new Map();
```

```
const paintResizeObserverMap = new Map();
```

```
const bloomResizeObserverMap = new Map();
```

```
let bloomChart = null;
```

```
let minesweeperTimerInterval = null;
```

```
let minesweeperTimeElapsed = 0;
```

```
let minesweeperFlagsPlaced = 0;
```

```
let minesweeperGameOver = false;
```

```
let minesweeperMineCount = 10;
```

```
let minesweeperGridSize = { rows: 9, cols: 9 };
```

```
let minesweeperFirstClick = true;
```

```
const youtubePlayers = {};
```

```
let ytApiLoaded = false;
```

```
let ytApiLoadingPromise = null;
```

```
const DEFAULT_YOUTUBE_VIDEO_ID = 'WXuK6gekU1Y';
```

```
const HIERARCHY_TYPES = ["Trunk", "Branch", "Leaf", "Flower", "Fruit", "Bud"];
```

```
let webWeaverDialUpAudio = null;
```

```
// --- Zettelkasten Card Class ---
```

```
class Card {
```

```
  constructor({ id, parentId = null, title, content, type, timestamp = Date.now(), tags = [], links = [], isPinned =  
false, pinX = 0, pinY = 0 }) {
```

```
    this.id = id;
```

```
    this.parentId = parentId;
```

```
    this.title = title;
```

```
    this.content = content;
```

```
    this.type = type;
```

```
    this.timestamp = timestamp;
```

```
    this.tags = tags;
```

```
    this.links = links;
```

```
    this.isPinned = isPinned;
```



```
        this.pinX = pinX;
        this.pinY = pinY;
    }
}

// --- Core OS Functions ---
function bringToFront(windowElement) {
    if (activeWindow === windowElement) return;
    if (activeWindow) {
        activeWindow.classList.remove('active');
        const appName = activeWindow.id;
        if (openApps.has(appName)) {
            openApps.get(appName)?.taskbarButton.classList.remove('active');
        }
    }
    highestZIndex++;
    windowElement.style.zIndex = highestZIndex.toString();
    windowElement.classList.add('active');
    activeWindow = windowElement;
    const appNameRef = windowElement.id;
    if (openApps.has(appNameRef)) {
        openApps.get(appNameRef)?.taskbarButton.classList.add('active');
    }
}

async function openApp(appName) {
    const windowElement = document.getElementById(appName);
    if (!windowElement) return;

    if (openApps.has(appName)) {
        windowElement.style.display = 'flex';
        bringToFront(windowElement);
        // Re-initialize or refresh if needed
        if (appName === 'conspiracyPinboard') initConspiracyPinboard(windowElement);
        if (appName === 'bloomSimulator') initBloomSimulator(windowElement);
        return;
    }

    windowElement.style.display = 'flex';
    bringToFront(windowElement);

    const taskbarButton = document.createElement('button');
    taskbarButton.type = 'button';
    taskbarButton.classList.add('taskbar-app');
    taskbarButton.dataset.appName = appName;

    const appData = getAppData(appName);
    if (appData.iconSrc) {
        const img = document.createElement('img');
```

```

    img.src = appData.iconSrc;
    img.alt = appData.title;
    taskbarButton.appendChild(img);
  }
  taskbarButton.appendChild(document.createTextNode(appData.title));
  taskbarButton.addEventListener('click', () => {
    if (windowElement === activeWindow && windowElement.style.display !== 'none') {
      minimizeApp(appName);
    } else {
      windowElement.style.display = 'flex';
      bringToFront(windowElement);
    }
  });
  taskbarAppsContainer.appendChild(taskbarButton);
  openApps.set(appName, { windowEl: windowElement, taskbarButton: taskbarButton });
  taskbarButton.classList.add('active');
  await initializeApp(appName, windowElement);
}

function closeApp(appName) {
  const appData = openApps.get(appName);
  if (!appData) return;

  const { windowEl, taskbarButton } = appData;
  windowEl.style.display = 'none';
  windowEl.classList.remove('active');
  taskbarButton.remove();
  openApps.delete(appName);
  delete dosInstances[appName];

  const cleanupFns = {
    'sigilCanvas': () => {
      const paintContent = appData.windowEl.querySelector('.window-content');
      if (paintContent && paintResizeObserverMap.has(paintContent)) {
        paintResizeObserverMap.get(paintContent)?.disconnect();
        paintResizeObserverMap.delete(paintContent);
      }
    },
    'bloomSimulator': () => {
      const bloomContent = appData.windowEl.querySelector('#bloom-chart-container');
      if (bloomContent && bloomResizeObserverMap.has(bloomContent)) {
        bloomResizeObserverMap.get(bloomContent)?.disconnect();
        bloomResizeObserverMap.delete(bloomContent);
      }
      if (bloomChart) {
        bloomChart.destroy();
        bloomChart = null;
      }
    },
    'mindField': () => {

```

```

    if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
    minesweeperTimerInterval = null;
  },
  'mediaStream': () => {
    const player = youtubePlayers[appName];
    if (player) {
      try {
        if (typeof player.stopVideo === 'function') player.stopVideo();
        if (typeof player.destroy === 'function') player.destroy();
      } catch (e) { console.warn("Error destroying media player:", e); }
      delete youtubePlayers[appName];
    }
    const playerDiv = document.getElementById(`youtube-player-${appName}`);
    if (playerDiv) playerDiv.innerHTML = `

Player closed.</p>`;
  }
};
cleanupFns[appName]?.();

if (activeWindow === windowEl) {
  activeWindow = null;
  let nextAppToActivate = null, maxZ = -1;
  openApps.forEach((data) => {
    const z = parseInt(data.windowEl.style.zIndex || '0', 10);
    if (z > maxZ) {
      maxZ = z;
      nextAppToActivate = data.windowEl;
    }
  });
  if (nextAppToActivate) bringToFront(nextAppToActivate);
}

function minimizeApp(appName) {
  const appData = openApps.get(appName);
  if (!appData) return;

  const { windowEl, taskbarButton } = appData;
  windowEl.style.display = 'none';
  windowEl.classList.remove('active');
  taskbarButton.classList.remove('active');

  if (activeWindow === windowEl) {
    activeWindow = null;
    let nextAppToActivate = null, maxZ = 0;
    openApps.forEach((data, name) => {
      if (data.windowEl.style.display !== 'none') {
        const z = parseInt(data.windowEl.style.zIndex || '0', 10);
        if (z > maxZ) {
          maxZ = z;
          nextAppToActivate = name;
        }
      }
    });
    if (nextAppToActivate) bringToFront(nextAppToActivate);
  }
}


```

```

    }
  }
});
if (nextAppToActivate) bringToFront(openApps.get(nextAppToActivate).windowEl);
}
}

async function initializeApp(appName, windowElement) {
  if (dosInstances[appName] && dosInstances[appName].initialized) return;

  const initializers = {
    'settings': initSettingsApp,
    'ingestionEngine': initIngestionEngine,
    'webWeaver': initWebWeaver,
    'tabletCreator': initTabletCreator,
    'sigilCanvas': initSimplePaintApp,
    'subspaceAnomaly': initSubspaceAnomaly,
    'mindField': initMinesweeperGame,
    'entropicNexus': initEntropicNexus,
    'mediaStream': initMediaPlayer,
    'conspiracyPinboard': initConspiracyPinboard,
    'hyenaDivaZettelkasten': (el) => initHyenaDivaApp(el.querySelector('#hyenaDivaZettelkasten-content')),
    'bloomSimulator': initBloomSimulator,
    'patternVisualizer': initPatternVisualizer,
  };

  if (initializers[appName]) {
    await initializers[appName](windowElement);
    dosInstances[appName] = { initialized: true };
  }
}

function getAppData(appName) {
  const iconElement = Array.from(icons).find(icon => icon.dataset.app === appName);
  if (iconElement) {
    return {
      title: iconElement.querySelector('span')?.textContent || appName,
      iconSrc: iconElement.querySelector('img')?.src || ''
    };
  }

  const appDataMap = {
    'hyenaDivaZettelkasten': { title: 'Zettelkasten', iconSrc: 'https://pfst.cf2.poecdn.net/base/image/0ffd2bf15a7001f86e395096222147037538b25af55ef9b8c7ac8f6768b7397c?w=48&h=48' },
    'home': { title: 'Home', iconSrc: '' }, 'about': { title: 'About', iconSrc: '' },
    'patternVisualizer': { title: 'Pattern Visualizer', iconSrc: '' }, 'hyenaDiva': { title: 'Hyena Diva', iconSrc: '' },
    'luminalLanguage': { title: 'Luminal Language', iconSrc: '' }, 'ackk': { title: 'A.C.K.K.', iconSrc: '' },
  };
  return appDataMap[appName] || { title: appName, iconSrc: '' };
}

```

```

}

// --- App Initializers ---
function initIngestionEngine(windowElement) {
  const inputText = windowElement.querySelector('#ingestion-input-text');
  const loadBtn = windowElement.querySelector('#ingestion-load-btn');
  const fileInput = windowElement.querySelector('#ingestion-file-input');
  const analyzeBtn = windowElement.querySelector('#ingestion-analyze-btn');
  const suggestionForm = windowElement.querySelector('#ingestion-suggestion-form');
  const createBtn = windowElement.querySelector('#suggestion-create-btn');
  const createPinBtn = windowElement.querySelector('#suggestion-create-pin-btn');

  loadBtn.addEventListener('click', () => fileInput.click());

  fileInput.addEventListener('change', (event) => {
    const file = event.target.files[0];
    if (!file || file.type !== 'text/plain') {
      alert('Please select a .txt file.');
```

return;

```

    }
    const reader = new FileReader();
    reader.onload = (e) => {
      inputText.value = e.target.result;
    };
    reader.readAsText(file);
    event.target.value = ''; // Reset file input
  });

  function resetAnalysisUI() {
    suggestionForm.style.display = 'none';
    document.getElementById('analysis-output').style.display = 'none';
    document.getElementById('analysis-placeholder').style.display = 'block';
    inputText.value = '';
  }

  analyzeBtn.addEventListener('click', () => {
    const text = inputText.value;
    if (!text.trim()) {
      alert('Please enter some text to analyze.');
```

return;

```

    }

    const cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]').map(c => new Card(c));
    const vaultIndex = new Set();
    const stopWords = new Set(['a', 'an', 'the', 'and', 'or', 'in', 'on', 'at', 'to', 'for', 'of', 'with', 'is', 'are',
      'was', 'were', 'it', 'i', 'you', 'he', 'she', 'they', 'we', 'that', 'this', 'but', 'as', 'if', 'so', 'my', 'me', 'am', 'do',
      'have']);

    cards.forEach(card => {
      card.title.toLowerCase().match(/\b\w+\b/g)?.forEach(word => !stopWords.has(word) &&

```

```

vaultIndex.add(word));
    card.tags.forEach(tag => tag.toLowerCase().match(/\b\w+\b/g)?.forEach(word => !stopWords.has(word)
    && vaultIndex.add(word)));
  });

  const words = text.toLowerCase().match(/\b\w+\b/g) || [];
  const wordCounts = words.reduce((acc, word) => {
    if (word.length > 3 && !stopWords.has(word)) {
      acc[word] = (acc[word] || 0) + 1;
    }
  }, {});
  const topKeywords = Object.entries(wordCounts)
    .sort([(a, b) => b[1] - a[1]])
    .slice(0, 10)
    .map(([word]) => word);

  const resonances = [];
  if (topKeywords.length > 0) {
    cards.forEach(card => {
      let score = 0;
      const cardText = `${card.title.toLowerCase()} ${card.tags.join(' ').toLowerCase()}`;
      topKeywords.forEach(kw => {
        if (cardText.includes(kw)) score++;
      });
      if (score > 0) resonances.push({ card, score });
    });
  }
  resonances.sort((a, b) => b.score - a.score);

  const newConcepts = topKeywords.filter(kw => !vaultIndex.has(kw));

  const sentences = text.match(/^[^!?]+[.!?]+/g) || [];
  const insights = sentences.filter(s => {
    const lowerS = s.toLowerCase();
    const isQuestion = lowerS.includes('?');
    const hasInsightPhrase = /\b(key is|idea is|realized|i think|conclusion|means that)\b/.test(lowerS);
    let keywordCount = 0;
    topKeywords.slice(0, 5).forEach(kw => {
      if (lowerS.includes(kw)) keywordCount++;
    });
    return isQuestion || hasInsightPhrase || keywordCount >= 2;
  }).slice(0, 5);

  document.getElementById('analysis-placeholder').style.display = 'none';
  document.getElementById('analysis-output').style.display = 'block';

  const resonancesList = document.getElementById('ingestion-resonances');
  resonancesList.innerHTML = resonances.length > 0 ? resonances.slice(0, 10).map(r => `<li
class="suggestion-item" data-card-id="${r.card.id}" title="Click to view this tablet">${r.card.title} (Score: $

```

```

{r.score}}</li>` ).join('') : '<li class="suggestion-placeholder">No connections found in vault.</li>';

    resonancesList.querySelectorAll('.suggestion-item').forEach(item => {
        item.addEventListener('click', () => showZettelDialog(item.dataset.cardId));
    });

    document.getElementById('ingestion-new-concepts').innerHTML = newConcepts.length > 0 ?
    newConcepts.map(nc => `<li>${nc}</li>` ).join('') : '<li class="suggestion-placeholder">No new concepts
    identified.</li>';
    document.getElementById('ingestion-insights').innerHTML = insights.length > 0 ? insights.map(i => `<li>${i}
    </li>` ).join('') : '<li class="suggestion-placeholder">No key insights or questions found.</li>';
    document.getElementById('ingestion-keywords').innerHTML = topKeywords.length > 0 ?
    topKeywords.map(kw => `<li>${kw}</li>` ).join('') : '<li class="suggestion-placeholder">No keywords found.</li>';

    suggestionForm.style.display = 'block';
    const suggestedTitle = `Ingested Chatlog: ${topKeywords[0] || new Date().toISOString().slice(0,10)}`;
    const suggestedContent = (insights.length > 0 ? insights.join('\n\n') : sentences.slice(0,
    5).join('\n\n')).trim();
    suggestionForm.querySelector('#suggestion-title').value = suggestedTitle;
    suggestionForm.querySelector('#suggestion-content').value = suggestedContent;
    suggestionForm.querySelector('#suggestion-tags').value = topKeywords.join(' ');
    });

    const createTabletFromSuggestion = (isPinned = false) => {
        const title = suggestionForm.querySelector('#suggestion-title').value;
        const content = suggestionForm.querySelector('#suggestion-content').value;
        const tags = suggestionForm.querySelector('#suggestion-tags').value.split(',').map(t =>
        t.trim()).filter(Boolean);

        let cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]');
        const trunks = cards.filter(c => !c.parentId && /^d+$/ .test(c.id));
        const newTrunkId = trunks.length > 0 ? (Math.max(...trunks.map(t => parseInt(t.id))) + 1000).toString() :
        '6000';

        const newCard = new Card({
            id: newTrunkId, title: title || 'Untitled Ingested Note',
            content: content, type: 'Trunk', tags: tags, isPinned: isPinned,
            pinX: Math.random() * 300, pinY: Math.random() * 200,
        });

        cards.push(newCard);
        localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
        window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));

        alert(`New tablet "${newCard.title}" created successfully!`);

        resetAnalysisUI();
        if (isPinned) {
            openApp('conspiracyPinboard');
        }
    }

```

```

};

createBtn.addEventListener('click', () => createTabletFromSuggestion(false));
createPinBtn.addEventListener('click', () => createTabletFromSuggestion(true));

window.addEventListener('zettelkastenUpdated', () => {
  if (windowElement.style.display !== 'none' && document.getElementById('analysis-output').style.display !==
'none') {
    // Maybe re-run analysis if the vault changes while the results are shown.
    // For now, do nothing to avoid losing the current analysis.
  }
});
}

function initTabletCreator(windowElement) {
  const saveBtn = windowElement.querySelector('#quick-tablet-save-btn');
  const textarea = windowElement.querySelector('#quick-tablet-textarea');

  saveBtn.addEventListener('click', () => {
    const content = textarea.value.trim();
    if (!content) {
      alert('There is nothing to save. ');
      return;
    }

    const title = content.substring(0, 30).split('\n')[0] || "Quick Note";

    let cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]');
    const newId = `bud-${Date.now()}`;
    const newCard = new Card({
      id: newId, title: title, content: content, type: 'Bud',
    });
    cards.push(newCard);

    localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
    window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));

    alert(` New tablet "${newCard.title}" created as a Bud!` );
    textarea.value = '';
  });
}

function initConspiracyPinboard(windowElement) {
  if (!windowElement) return;

  const nodesContainer = windowElement.querySelector('#pinboard-nodes-container');
  const svgLayer = windowElement.querySelector('#pinboard-svg-layer');
  const addEvidenceBtn = windowElement.querySelector('#add-evidence-btn');
  const drawConnectionBtn = windowElement.querySelector('#draw-connection-btn');

```



```

const suggestionsList = windowElement.querySelector('.connection-suggestions-list');
const pinnedList = windowElement.querySelector('.pinned-tablets-list');

if (!nodesContainer || !svgLayer || !addEvidenceBtn || !drawConnectionBtn) {
  console.error("Pinboard elements not found");
  return;
}

let cards = [];
let nodes = []; // { id, x, y, el }
let isConnecting = false;
let firstNode = null;

function saveCardsToStorage() {
  localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
  window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
}

function loadCardsFromStorage() {
  try {
    cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]').map(c => new Card(c));
  } catch(e) { console.error("Failed to load cards for pinboard", e); cards = []; }
}

function makeDraggable(element) {
  let offsetX, offsetY, isDragging = false;

  const onMouseDown = (e) => {
    if (e.target.closest('.pinboard-node-action')) return;
    if (isConnecting) {
      handleConnectionClick(element);
      return;
    }
    if (window.innerWidth <= 800) return; // Disable dragging on mobile
    isDragging = true;
    element.style.cursor = 'grabbing';
    bringToFront(pinboardWindow);
    offsetX = e.clientX - element.offsetLeft;
    offsetY = e.clientY - element.offsetTop;
    document.addEventListener('mousemove', onMouseMove);
    document.addEventListener('mouseup', onMouseUp, { once: true });
  };

  const onMouseMove = (e) => {
    if (!isDragging) return;
    const node = nodes.find(n => n.id === element.dataset.id);
    if (node) {
      node.x = e.clientX - offsetX;
      node.y = e.clientY - offsetY;
      element.style.left = `${node.x}px`;
    }
  };
}

```

```

        element.style.top = `${node.y}px`;
        updateConnections();
    }
};

const onMouseUp = () => {
    if (!isDragging) return;
    isDragging = false;
    element.style.cursor = 'grab';
    document.removeEventListener('mousemove', onMouseMove);

    const card = cards.find(c => c.id === element.dataset.id);
    const node = nodes.find(n => n.id === element.dataset.id);
    if (card && node) {
        card.pinX = node.x;
        card.pinY = node.y;
        saveCardsToStorage();
    }
};

element.addEventListener('mousedown', onMouseDown);
}

function updateConnections() {
    svgLayer.innerHTML = '';
    nodes.forEach(node => {
        const card = cards.find(c => c.id === node.id);
        if (!card || !card.links) return;

        card.links.forEach(targetId => {
            const fromNode = node;
            const toNode = nodes.find(n => n.id === targetId);
            if (fromNode && toNode) {
                const line = document.createElementNS('http://www.w3.org/2000/svg', 'line');
                line.setAttribute('x1', (fromNode.x + fromNode.el.offsetWidth / 2).toString());
                line.setAttribute('y1', (fromNode.y + fromNode.el.offsetHeight / 2).toString());
                line.setAttribute('x2', (toNode.x + toNode.el.offsetWidth / 2).toString());
                line.setAttribute('y2', (toNode.y + toNode.el.offsetHeight / 2).toString());
                line.setAttribute('class', 'thread-line');
                svgLayer.appendChild(line);
            }
        });
    });
}

function handleConnectionClick(nodeEl) {
    if (!firstNode) {
        firstNode = nodeEl;
        nodeEl.style.boxShadow = '0 0 10px 3px #ff3333';
    } else if (firstNode !== nodeEl) {

```

```

const firstCard = cards.find(c => c.id === firstNode.dataset.id);
if(firstCard) {
  if (!firstCard.links) firstCard.links = [];
  if (!firstCard.links.includes(nodeEl.dataset.id)) {
    firstCard.links.push(nodeEl.dataset.id);
  }
  saveCardsToStorage(); // This now dispatches the event
}
firstNode.style.boxShadow = '';
firstNode = null;
isConnecting = false;
drawConnectionBtn.classList.remove('active');
updateConnections();
}
}

drawConnectionBtn.onclick = () => {
  isConnecting = !isConnecting;
  drawConnectionBtn.classList.toggle('active', isConnecting);
  if (!isConnecting && firstNode) {
    firstNode.style.boxShadow = '';
    firstNode = null;
  }
};

function getSuggestions(selectedCard) {
  const stopWords = new Set(['a', 'an', 'the', 'and', 'or', 'in', 'on', 'at', 'to', 'for', 'of', 'with', 'is', 'are', 'was', 'were']);
  const content = `${selectedCard.title} ${selectedCard.content}`.toLowerCase().replace(/[\^w\s#]/g, '');
  const keywords = content.split(/\s+/).filter(word => word.length > 2 && !stopWords.has(word));
  const tags = (selectedCard.tags || []).map(t => t.toLowerCase());

  const suggestions = cards.filter(card => card.id !== selectedCard.id).map(card => {
    let score = 0;
    const targetContent = `${card.title} ${card.content}`.toLowerCase();
    const targetTags = (card.tags || []).map(t => t.toLowerCase());

    keywords.forEach(kw => {
      if (targetContent.includes(kw)) score += 1;
    });
    tags.forEach(tag => {
      if (targetTags.includes(tag)) score += 5;
    });

    return { card, score };
  })
  .filter(item => item.score > 0)
  .sort((a, b) => b.score - a.score)
  .slice(0, 10);

```

```

    return suggestions;
}

function renderSuggestions(selectedCard) {
  const suggestions = getSuggestions(selectedCard);
  suggestionsList.innerHTML = '';
  if (suggestions.length === 0) {
    suggestionsList.innerHTML = '<li class="suggestion-placeholder">No connections found.</li>';
    return;
  }
  suggestions.forEach(item => {
    const li = document.createElement('li');
    li.className = 'suggestion-item';
    li.textContent = `${item.card.title}`;
    li.title = `ID: ${item.card.id} (Score: ${item.score})`;
    li.onclick = () => showZettelDialog(item.card.id);
    suggestionsList.appendChild(li);
  });
}

function renderPinnedTabletsList(pinnedCards) {
  pinnedList.innerHTML = '';
  if (pinnedCards.length === 0) {
    pinnedList.innerHTML = '<li class="suggestion-placeholder">No tablets pinned.</li>';
    return;
  }
  pinnedCards.forEach(card => {
    const li = document.createElement('li');
    li.className = 'suggestion-item';
    li.textContent = card.title;
    li.title = `ID: ${card.id}`;
    li.onclick = () => showZettelDialog(card.id);
    pinnedList.appendChild(li);
  });
}

function renderPinboard() {
  loadCardsFromStorage();
  nodesContainer.innerHTML = '';
  nodes = [];
  const pinnedCards = cards.filter(c => c.isPinned);

  pinnedCards.forEach(card => {
    const nodeEl = document.createElement('div');
    nodeEl.className = 'pinboard-node';
    nodeEl.dataset.id = card.id;
    nodeEl.style.left = `${card.pinX || 50}px`;
    nodeEl.style.top = `${card.pinY || 50}px`;
    nodeEl.innerHTML = `
      <div class="pinboard-node-header">

```

```

        <h4>${card.title}</h4>
      </div>
      <p>${card.content.substring(0, 100)}...</p>`;

nodeEl.addEventListener('click', () => renderSuggestions(card));
nodeEl.addEventListener('dblclick', () => showZettelDialog(card.id));

nodesContainer.appendChild(nodeEl);
nodes.push({ id: card.id, x: card.pinX || 50, y: card.pinY || 50, el: nodeEl });
makeDraggable(nodeEl);
});
updateConnections();
renderPinnedTabletsList(pinnedCards);
}

addEvidenceBtn.onclick = () => {
  const unpinnedCards = cards.filter(c => !c.isPinned);
  const dialog = document.getElementById('zettelkasten-dialog');
  const dialogContent = document.getElementById('dialog-inner-content');
  const dialogTitle = document.getElementById('dialog-title');

  dialogTitle.textContent = "Pin Evidence to Board";
  if (unpinnedCards.length === 0) {
    dialogContent.innerHTML = '<p>All tablets have been pinned.</p>';
  } else {
    dialogContent.innerHTML = `
      <label class="win98-label">Select a tablet to pin:</label>
      <div class="pin-evidence-list">
        ${unpinnedCards.map(card => `<div class="pin-evidence-item" data-id="${card.id}">${card.title}</div>`).join('')}
      </div>`;
  }
  dialog.style.display = 'flex';

  dialog.querySelectorAll('.pin-evidence-item').forEach(item => {
    item.addEventListener('click', () => {
      const cardId = item.dataset.id;
      const card = cards.find(c => c.id === cardId);
      if (card) {
        card.isPinned = true;
        card.pinX = Math.random() * (nodesContainer.offsetWidth - 200);
        card.pinY = Math.random() * (nodesContainer.offsetHeight - 100);
        saveCardsToStorage();
        // renderPinboard is now called by the event listener
      }
      dialog.style.display = 'none';
    });
  });
};
};

```

```

window.addEventListener('zettelkastenUpdated', () => {
  if (openApps.has('conspiracyPinboard')) {
    renderPinboard();
  }
});

renderPinboard();
}

function initSettingsApp(windowElement) {
  const exportBtn = windowElement.querySelector('#export-data-btn');
  const importBtn = windowElement.querySelector('#import-data-btn');
  const importInput = windowElement.querySelector('#import-file-input');

  exportBtn.addEventListener('click', () => {
    const dataToExport = localStorage.getItem('multiZettelkasten');
    if (!dataToExport) {
      alert('No data to export. ');
      return;
    }
    const blob = new Blob([dataToExport], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `entropic-nexus-vault-${new Date().toISOString().slice(0, 10)}.json`;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);
  });

  importBtn.addEventListener('click', () => importInput.click());

  importInput.addEventListener('change', (event) => {
    const file = event.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = (e) => {
      try {
        const importedData = e.target.result;
        JSON.parse(importedData);
        if (confirm('Are you sure you want to overwrite your current vault? This cannot be undone. ')) {
          localStorage.setItem('multiZettelkasten', importedData);
          alert('Vault imported successfully! The application will now reload. ');
          location.reload();
        }
      } catch (err) { alert('Error: Invalid or corrupt data file. '); }
    };
    reader.readAsText(file);
  });
}

```

```

    });
}

function initEntropicNexus(windowElement) {
    const knowledgeVaultIcon = windowElement.querySelector('#knowledge-vault-icon');
    const cDriveIcon = windowElement.querySelector('#c-drive-icon-hub');
    knowledgeVaultIcon.addEventListener('click', () => openApp('hyenaDivaZettelkasten'));
    cDriveIcon.addEventListener('click', () => alert("The C: Drive contains system files. Access is restricted."));
}

function initSubspaceAnomaly() {
    const doomContainer = document.getElementById('doom-content');
    if (doomContainer) {
        doomContainer.innerHTML = '<iframe src="https://js-dos.com/games/doom.exe.html" width="100%" height="100%" frameborder="0" scrolling="no" allowfullscreen></iframe>';
    }
}

function initPatternVisualizer(windowElement) {
    const frame = windowElement.querySelector('iframe');
    if (frame) {
        frame.src = './pattern-visualizer.html';
    }
}

async function navigateToUrlInWebWeaver(url, windowElement) {
    const addressBar = windowElement.querySelector('.browser-address-bar');
    const iframe = windowElement.querySelector('#browser-frame');
    const loadingEl = windowElement.querySelector('.browser-loading');
    const DIAL_UP_SOUND_URL = 'https://www.soundjay.com/communication/dial-up-modem-01.mp3';

    if (!url || !url.trim()) return;
    if (!url.startsWith('http')) url = 'https://' + url;

    loadingEl.style.display = 'flex';
    try {
        if (!webWeaverDialUpAudio) webWeaverDialUpAudio = new Audio(DIAL_UP_SOUND_URL);
        webWeaverDialUpAudio.loop = true;
        await webWeaverDialUpAudio.play();
    } catch (e) { console.error("Audio error:", e); }

    setTimeout(() => {
        iframe.src = url;
        addressBar.value = url;
        iframe.onload = iframe.onerror = () => {
            loadingEl.style.display = 'none';
            if (webWeaverDialUpAudio) {
                webWeaverDialUpAudio.pause();
                webWeaverDialUpAudio.currentTime = 0;
            }
        }
    }, 100);
}

```

```

    };
  }, 4000); // Simulate dial-up time
}

function initWebWeaver(windowElement) {
  const addressBar = windowElement.querySelector('.browser-address-bar');
  const goButton = windowElement.querySelector('.browser-go-button');

  goButton.addEventListener('click', () => navigateToUrlInWebWeaver(addressBar.value, windowElement));
  addressBar.addEventListener('keydown', (e) => {
    if (e.key === 'Enter') navigateToUrlInWebWeaver(addressBar.value, windowElement);
  });
  addressBar.addEventListener('click', () => addressBar.select());
}

async function openWebLink(url) {
  await openApp('webWeaver');
  const windowElement = document.getElementById('webWeaver');
  await navigateToUrlInWebWeaver(url, windowElement);
}

function initSimplePaintApp(windowElement) {
  const canvas = windowElement.querySelector('#paint-canvas');
  const toolbar = windowElement.querySelector('.paint-toolbar');
  const contentArea = windowElement.querySelector('.window-content');
  const colorSwatches = windowElement.querySelectorAll('.paint-color-swatch');
  const sizeButtons = windowElement.querySelectorAll('.paint-size-button');
  const clearButton = windowElement.querySelector('.paint-clear-button');

  if (!canvas || !toolbar || !contentArea || !clearButton) return;
  const ctx = canvas.getContext('2d');
  if (!ctx) return;

  let isDrawing = false, lastX = 0, lastY = 0;
  let currentStrokeStyle = 'black', currentLineWidth = 2;
  ctx.lineJoin = 'round'; ctx.lineCap = 'round';

  function resizeCanvas() {
    requestAnimationFrame(() => {
      const rect = contentArea.getBoundingClientRect();
      const toolbarHeight = toolbar.offsetHeight;
      const newWidth = Math.floor(rect.width);
      const newHeight = Math.floor(rect.height - toolbarHeight);
      if (canvas.width === newWidth && canvas.height === newHeight) return;
      canvas.width = newWidth > 0 ? newWidth : 1;
      canvas.height = newHeight > 0 ? newHeight : 1;
      ctx.fillStyle = 'white'; ctx.fillRect(0, 0, canvas.width, canvas.height);
      ctx.strokeStyle = currentStrokeStyle;
      ctx.lineWidth = currentLineWidth;
    });
  }

```



```

}

const resizeObserver = new ResizeObserver(resizeCanvas);
resizeObserver.observe(contentArea);
paintResizeObserverMap.set(contentArea, resizeObserver);
resizeCanvas();

function getMousePos(e) {
  const rect = canvas.getBoundingClientRect();
  const clientX = e.touches ? e.touches[0].clientX : e.clientX;
  const clientY = e.touches ? e.touches[0].clientY : e.clientY;
  return { x: clientX - rect.left, y: clientY - rect.top };
}

function startDrawing(e) {
  isDrawing = true;
  const pos = getMousePos(e);
  [lastX, lastY] = [pos.x, pos.y];
}

function draw(e) {
  if (!isDrawing) return;
  e.preventDefault();
  const pos = getMousePos(e);
  ctx.beginPath();
  ctx.moveTo(lastX, lastY);
  ctx.lineTo(pos.x, pos.y);
  ctx.stroke();
  [lastX, lastY] = [pos.x, pos.y];
}

function stopDrawing() { isDrawing = false; }

['mousedown', 'touchstart'].forEach(evt => canvas.addEventListener(evt, startDrawing, { passive: false }));
['mousemove', 'touchmove'].forEach(evt => canvas.addEventListener(evt, draw, { passive: false }));
['mouseup', 'mouseleave', 'touchend', 'touchcancel'].forEach(evt => canvas.addEventListener(evt,
stopDrawing));

colorSwatches.forEach(swatch => swatch.addEventListener('click', () => {
  currentStrokeStyle = ctx.strokeStyle = swatch.dataset.color || 'black';
  colorSwatches.forEach(s => s.classList.remove('active'));
  swatch.classList.add('active');
}));

sizeButtons.forEach(button => button.addEventListener('click', () => {
  currentLineWidth = ctx.lineWidth = parseInt(button.dataset.size || '2', 10);
  sizeButtons.forEach(s => s.classList.remove('active'));
  button.classList.add('active');
}));

clearButton.addEventListener('click', () => {
  ctx.fillStyle = 'white';
  ctx.fillRect(0, 0, canvas.width, canvas.height);
});

```

```

windowElement.querySelector('.paint-color-swatch[data-color="black"]')?.click();
windowElement.querySelector('.paint-size-button[data-size="2"]')?.click();
}

function initMinesweeperGame(windowElement) {
  const boardElement = windowElement.querySelector('#minesweeper-board');
  const flagCountElement = windowElement.querySelector('.minesweeper-flag-count');
  const timerElement = windowElement.querySelector('.minesweeper-timer');
  const resetButton = windowElement.querySelector('.minesweeper-reset-button');
  if (!boardElement || !flagCountElement || !timerElement || !resetButton) return;
  let grid = [];

  function resetGame() {
    if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
    minesweeperTimerInterval = null;
    minesweeperTimeElapsed = 0;
    minesweeperFlagsPlaced = 0;
    minesweeperGameOver = false;
    minesweeperFirstClick = true;
    timerElement.innerHTML = `⌚ 00:00`;
    flagCountElement.innerHTML = `🚩 ${minesweeperMineCount}`;
    resetButton.innerHTML = `🔄`;
    createGrid();
  }

  function createGrid() {
    boardElement.innerHTML = '';
    grid = [];
    boardElement.style.gridTemplateColumns = `repeat(${minesweeperGridSize.cols}, 20px)`;
    boardElement.style.gridTemplateRows = `repeat(${minesweeperGridSize.rows}, 20px)`;
    for (let r = 0; r < minesweeperGridSize.rows; r++) {
      const row = [];
      for (let c = 0; c < minesweeperGridSize.cols; c++) {
        const cellElement = document.createElement('div');
        cellElement.classList.add('minesweeper-cell');
        const cellData = { isMine: false, isRevealed: false, isFlagged: false, adjacentMines: 0, element:
cellElement, row: r, col: c };
        cellElement.addEventListener('click', () => handleCellClick(cellData));
        cellElement.addEventListener('contextmenu', (e) => { e.preventDefault();
handleCellRightClick(cellData); });
        row.push(cellData);
        boardElement.appendChild(cellElement);
      }
      grid.push(row);
    }
  }

  function placeMines(firstClickRow, firstClickCol) {
    let minesPlaced = 0;
    while (minesPlaced < minesweeperMineCount) {

```

```

    const r = Math.floor(Math.random() * minesweeperGridSize.rows);
    const c = Math.floor(Math.random() * minesweeperGridSize.cols);
    if ((r === firstClickRow && c === firstClickCol) || grid[r][c].isMine) continue;
    grid[r][c].isMine = true;
    minesPlaced++;
  }
  grid.forEach((row, r) => row.forEach((cell, c) => {
    if (!cell.isMine) cell.adjacentMines = countAdjacentMines(r, c);
  }));
}

function countAdjacentMines(row, col) {
  let count = 0;
  for (let dr = -1; dr <= 1; dr++) {
    for (let dc = -1; dc <= 1; dc++) {
      if (dr === 0 && dc === 0) continue;
      const nr = row + dr, nc = col + dc;
      if (nr >= 0 && nr < minesweeperGridSize.rows && nc >= 0 && nc < minesweeperGridSize.cols &&
grid[nr][nc].isMine) count++;
    }
  }
  return count;
}

function handleCellClick(cell) {
  if (minesweeperGameOver || cell.isRevealed || cell.isFlagged) return;
  if (minesweeperFirstClick) {
    placeMines(cell.row, cell.col);
    minesweeperFirstClick = false;
    startTimer();
  }
  if (cell.isMine) {
    gameOver(cell);
  } else {
    revealCell(cell);
    checkWinCondition();
  }
}

function handleCellRightClick(cell) {
  if (minesweeperGameOver || cell.isRevealed || minesweeperFirstClick) return;
  cell.isFlagged = !cell.isFlagged;
  cell.element.innerHTML = cell.isFlagged ? '🚩' : '';
  minesweeperFlagsPlaced += cell.isFlagged ? 1 : -1;
  flagCountElement.innerHTML = `🚩 ${minesweeperMineCount - minesweeperFlagsPlaced}`;
  checkWinCondition();
}

function revealCell(cell) {
  if (cell.isRevealed || cell.isFlagged) return;

```

```

    cell.isRevealed = true;
    cell.element.classList.add('revealed');
    if (cell.adjacentMines > 0) {
        cell.element.textContent = cell.adjacentMines;
        cell.element.dataset.number = cell.adjacentMines;
    } else {
        for (let dr = -1; dr <= 1; dr++) {
            for (let dc = -1; dc <= 1; dc++) {
                const nr = cell.row + dr, nc = cell.col + dc;
                if (nr >= 0 && nr < minesweeperGridSize.rows && nc >= 0 && nc < minesweeperGridSize.cols) {
                    revealCell(grid[nr][nc]);
                }
            }
        }
    }
}

function startTimer() {
    if (minesweeperTimerInterval) return;
    minesweeperTimerInterval = setInterval(() => {
        minesweeperTimeElapsed++;
        timerElement.innerHTML = `⌚: ${minesweeperTimeElapsed}`;
    }, 1000);
}

function gameOver(clickedMine) {
    minesweeperGameOver = true;
    if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
    resetButton.innerHTML = '🔄';
    grid.forEach(row => row.forEach(cell => {
        if (cell.isMine) cell.element.innerHTML = '💣';
        if (cell.isFlagged && !cell.isMine) cell.element.innerHTML = '🚩';
    }));
    clickedMine.element.classList.add('exploded');
    clickedMine.element.innerHTML = '💣';
}

function checkWinCondition() {
    if (minesweeperGameOver) return;
    const revealedCount = grid.flat().filter(cell => cell.isRevealed).length;
    const nonMineCells = (minesweeperGridSize.rows * minesweeperGridSize.cols) - minesweeperMineCount;
    if (revealedCount === nonMineCells) {
        minesweeperGameOver = true;
        if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
        resetButton.innerHTML = '🏆';
        grid.flat().filter(cell => cell.isMine).forEach(cell => cell.element.innerHTML = '💣');
    }
}

resetButton.addEventListener('click', resetGame);

```

```

    resetGame();
  }

function loadYouTubeAPI() {
  if (ytApiLoaded) return Promise.resolve();
  if (ytApiLoadingPromise) return ytApiLoadingPromise;
  return ytApiLoadingPromise = new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = "https://www.youtube.com/iframe_api";
    script.onload = () => {
      ytApiLoaded = true;
      window.onYouTubeIframeAPIReady = () => resolve();
    };
    script.onerror = () => reject(new Error("Failed to load YouTube API."));
    document.head.appendChild(script);
  });
}

async function initMediaPlayer(windowElement) {
  const appName = windowElement.id;
  const playerDiv = document.getElementById(`youtube-player-${appName}`);
  const loadButton = windowElement.querySelector('.media-player-load-button');
  const urlInput = windowElement.querySelector('.media-player-input');
  const playBtn = windowElement.querySelector('#media-player-play');
  const pauseBtn = windowElement.querySelector('#media-player-pause');
  const stopBtn = windowElement.querySelector('#media-player-stop');
  if (!playerDiv || !loadButton || !urlInput) return;

  [playBtn, pauseBtn, stopBtn].forEach(btn => btn.disabled = true);

  try {
    await loadYouTubeAPI();
    playerDiv.innerHTML = '';
    createPlayer(DEFAULT_YOUTUBE_VIDEO_ID);
  } catch (error) {
    playerDiv.innerHTML = `

Error loading YouTube player.</p>`;
  }

  function createPlayer(videoId) {
    if (youtubePlayers[appName]) youtubePlayers[appName].destroy();
    youtubePlayers[appName] = new window.YT.Player(`youtube-player-${appName}`, {
      height: '100%', width: '100%', videoId: videoId,
      playerVars: { 'autoplay': 1, 'controls': 0, 'modestbranding': 1 },
      events: { 'onReady': onPlayerReady, 'onError': onPlayerError }
    });
  }

  function onPlayerReady(event) {
    [playBtn, pauseBtn, stopBtn].forEach(btn => btn.disabled = false);
    event.target.playVideo();
  }
}


```

```

    }
    function onPlayerError() {
      playerDiv.innerHTML = `<p class="media-player-status-message">Error playing video.</p>`;
    }

    loadButton.addEventListener('click', () => {
      const regExp = /^.*(youtu.be\/|v\/|u\/\w\/|embed\/|watch\?v=|&v=)([^\#&?]*).*/;
      const match = urlInput.value.match(regExp);
      const videoId = (match && match[2].length === 11) ? match[2] : urlInput.value;
      if (videoId) createPlayer(videoId); else alert("Invalid YouTube URL or ID");
    });

    playBtn.onclick = () => youtubePlayers[appName]?.playVideo();
    pauseBtn.onclick = () => youtubePlayers[appName]?.pauseVideo();
    stopBtn.onclick = () => youtubePlayers[appName]?.stopVideo();
  }

  function initHyenaDivaApp(container) {
    let cards = [];
    let currentTab = 'All';

    function saveCards() {
      localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
      window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
    }

    function loadCards() {
      const stored = localStorage.getItem('multiZettelkasten');
      if (stored) {
        try {
          cards = JSON.parse(stored).map(c => new Card(c));
        } catch (e) {
          cards = loadZettelkastenData();
          saveCards();
        }
      } else {
        cards = loadZettelkastenData();
        saveCards();
      }
    }

    function render() {
      container.innerHTML = `
        <h1>Knowledge Vault</h1>
        <div class="tabs-container">
          <button class="win98-button tab-button" data-tab="All">All</button>
          ${HIERARCHY_TYPES.map(type => `<button class="win98-button tab-button" data-tab="${type}">${
            type
          }</button>`).join('')}
          <button class="win98-button" id="new-card-button">+ New Tablet</button>
        </div>
        <div class="zettel-container" id="zettel-container"></div>`;
    }
  }

```

```

    container.querySelector('#new-card-button').addEventListener('click', () =>
showZettelkastenNewCardDialog());
    container.querySelectorAll('.tab-button').forEach(button => button.addEventListener('click', (e) => {
        currentTab = e.target.dataset.tab;
        renderCardList();
        container.querySelectorAll('.tab-button').forEach(btn => btn.classList.remove('active'));
        e.target.classList.add('active');
    }));
    container.querySelector('.tab-button[data-tab="${currentTab}"]').classList.add('active');
    renderCardList();
}

function renderCardList() {
    const zettelContainer = container.querySelector('#zettel-container');
    zettelContainer.innerHTML = '';
    loadCards();
    const filteredCards = (currentTab === 'All') ? cards : cards.filter(card => card.type === currentTab);
    sortCardsByHierarchy(filteredCards);

    filteredCards.forEach(card => {
        const cardElement = document.createElement('div');
        cardElement.className = `zettel-card ${card.type.toLowerCase()}-card`;
        cardElement.dataset.id = card.id;

        const pinIcon = document.createElement('span');
        pinIcon.className = `zettel-pin-icon ${card.isPinned ? 'pinned' : ''}`;
        pinIcon.textContent = '📌';
        pinIcon.title = card.isPinned ? 'Unpin from board' : 'Pin to board';
        pinIcon.addEventListener('click', (e) => {
            e.stopPropagation();
            card.isPinned = !card.isPinned;
            saveCards(); // This dispatches the update event
        });

        cardElement.innerHTML = `
            <div class="zettel-title-container">
                <div class="zettel-title">${card.title}</div>
                <div class="zettel-id">${card.id}</div>
            </div>`;
        cardElement.querySelector('.zettel-title-container').prepend(pinIcon);

        cardElement.addEventListener('click', () => showZettelDialog(card.id));
        zettelContainer.appendChild(cardElement);
    });
}

window.addEventListener('zettelkastenUpdated', () => {
    if (openApps.has('hyenaDivaZettelkasten')) {
        renderCardList();
    }
});

```

```

    }
  });

  loadCards();
  render();
}

function showZettelDialog(cardId) {
  let cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]').map(c => new Card(c));
  const card = cards.find(c => c.id === cardId);
  if (!card) return;

  const dialog = document.getElementById('zettelkasten-dialog');
  const dialogContent = document.getElementById('dialog-inner-content');
  const dialogTitle = document.getElementById('dialog-title');

  dialogTitle.textContent = `View/Edit: ${card.title}`;
  dialogContent.innerHTML = `
    <label class="win98-label">Title:</label>
    <input type="text" id="edit-title" class="win98-input" value="${card.title}">
    <label class="win98-label">Content:</label>
    <textarea id="edit-content" class="win98-textarea">${card.content}</textarea>
    <label class="win98-label">Type:</label>
    <select id="edit-type" class="win98-select">
      ${HIERARCHY_TYPES.map(type => `<option value="${type}" ${card.type === type ? 'selected' : ''}>${
        {type}</option>`}).join('')}
    </select>
    <label class="win98-label">Tags (comma separated):</label>
    <input type="text" id="edit-tags" class="win98-input" value="${(card.tags || []).join(' ')}">
    <label class="win98-label"><input type="checkbox" id="edit-pinned" ${card.isPinned ? 'checked' : ''}> Pin to
    Board</label>
    <div style="margin-top: 20px;">
      <button id="save-card-button" class="win98-button">Save</button>
      <button id="delete-card-button" class="win98-button">Delete</button>
    </div>`;

  dialog.style.display = 'flex';

  dialog.querySelector('#save-card-button').addEventListener('click', () => {
    const cardIndex = cards.findIndex(c => c.id === cardId);
    if (cardIndex === -1) return;
    cards[cardIndex].title = dialog.querySelector('#edit-title').value;
    cards[cardIndex].content = dialog.querySelector('#edit-content').value;
    cards[cardIndex].type = dialog.querySelector('#edit-type').value;
    cards[cardIndex].tags = dialog.querySelector('#edit-tags').value.split(',').map(t =>
      t.trim()).filter(Boolean);
    cards[cardIndex].isPinned = dialog.querySelector('#edit-pinned').checked;

    localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
    window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
  });
}

```



```

    dialog.style.display = 'none';
  });
  dialog.querySelector('#delete-card-button').addEventListener('click', () => {
    if(confirm(`Delete "${card.title}"?`)) {
      const index = cards.findIndex(c => c.id === cardId);
      if(index > -1) cards.splice(index, 1);
      localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
      window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
      dialog.style.display = 'none';
    }
  });
}

function showZettelkastenNewCardDialog() {
  let cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]').map(c => new Card(c));
  const dialog = document.getElementById('zettelkasten-dialog');
  const dialogContent = document.getElementById('dialog-inner-content');
  const dialogTitle = document.getElementById('dialog-title');

  dialogTitle.textContent = "Create New Tablet";
  dialogContent.innerHTML = `
    <label class="win98-label">Title:</label>
    <input type="text" id="new-title" class="win98-input" placeholder="Enter title...">
    <label class="win98-label">Content:</label>
    <textarea id="new-content" class="win98-textarea"></textarea>
    <label class="win98-label">Type:</label>
    <select id="new-type" class="win98-select">${HIERARCHY_TYPES.map(t => `<option value="${t}">${t}</option>` ).join('')}</select>
    <label class="win98-label">Parent Tablet (Optional):</label>
    <select id="new-parent" class="win98-select">
      <option value="">None (new Trunk)</option>
      ${cards.map(c => `<option value="${c.id}">${c.id} - ${c.title}</option>` ).join('')}
    </select>
    <label class="win98-label">Tags (comma separated):</label>
    <input type="text" id="new-tags" class="win98-input" placeholder="e.g. consciousness, AI, hyena">
    <div style="margin-top: 20px;"><button id="create-card-button" class="win98-button">Create</button></div>`;

  dialog.style.display = 'flex';

  dialog.querySelector('#create-card-button').addEventListener('click', () => {
    const parentId = dialog.querySelector('#new-parent').value;
    const parent = cards.find(c => c.id === parentId);
    let newId;
    if (parent) {
      const children = cards.filter(c => c.parentId === parentId);
      newId = `${parentId}-${children.length + 1}`;
    } else {
      const trunks = cards.filter(c => !c.parentId && /\d+$/ .test(c.id));
      const newTrunkId = trunks.length > 0 ? Math.max(...trunks.map(t => parseInt(t.id))) + 1000 : 6000;

```

```

        newId = newTrunkId.toString();
    }

    const newCard = new Card({
        id: newId, parentId: parentId || null,
        title: dialog.querySelector('#new-title').value || "Untitled",
        content: dialog.querySelector('#new-content').value,
        type: dialog.querySelector('#new-type').value,
        tags: dialog.querySelector('#new-tags').value.split(',').map(t => t.trim()).filter(Boolean),
    });
    cards.push(newCard);
    localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
    window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
    dialog.style.display = 'none';
    });
}

function sortCardsByHierarchy(cards) {
    const collator = new Intl.Collator(undefined, { numeric: true, sensitivity: 'base' });
    cards.sort((a, b) => collator.compare(a.id, b.id));
}

function loadZettelkastenData() {
    return [
        new Card({id: "1000", type: "Trunk", title: "Consciousness, Hermetics & Emergence", content: "Aries | Mars | Ontogenesis, Mental Causality, Hermetic Law"}),
        new Card({id: "1000/1", parentId: "1000", type: "Branch", title: "1100 :: Hermetics", content: "Foundation of mind, law, being."}),
        new Card({id: "1000/1-A", parentId: "1000/1", type: "Leaf", title: "1100/1 :: Hermetic Axiom", content: "Core Hermetic principles."}),
        new Card({id: "1000/1-A-1", parentId: "1000/1-A", type: "Flower", title: "1100/1-A/1A :: I Think, Therefore I Am", content: ""}),
        new Card({id: "1000/1-A-2", parentId: "1000/1-A", type: "Flower", title: "1100/1-A/2A :: Simulation versus Imagination", content: ""}),
        new Card({id: "1000/1-A-3", parentId: "1000/1-A", type: "Flower", title: "1100/1-A/3A :: Habit vs. Instinct", content: ""}),
        new Card({id: "1000/1-A-4", parentId: "1000/1-A", type: "Flower", title: "1100/1-A/5A :: Earning versus Owing", content: ""}),
        new Card({id: "1000/1-A-5", parentId: "1000/1-A", type: "Flower", title: "1100/1-A/6A :: Mastery versus Instinct", content: ""}),
        new Card({id: "1000/1-A-6", parentId: "1000/1-A", type: "Flower", title: "1100/1-A/7A :: Thought versus Belief", content: ""}),
        new Card({id: "1000/1-B", parentId: "1000/1", type: "Leaf", title: "1100/2 :: Thoughts Are Things", content: "Principle of mental manifestation."}),
        new Card({id: "1000/1-B-1", parentId: "1000/1-B", type: "Flower", title: "1100/2-A :: A Prevailing Mental Attitude", content: ""}),
        new Card({id: "1000/1-B-2", parentId: "1000/1-B", type: "Flower", title: "1100/2-B :: Principles, Laws, and Praxis", content: ""}),
        new Card({id: "1000/1-C", parentId: "1000/1", type: "Leaf", title: "1100/3 :: Foundational Hermetic Texts & Concepts", content: "Core textual sources and ideas."}),
    ]
}

```

```

    new Card({id: "1000/1-C-1", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/1C/13 :: Attraction,
Power, and Desire Force (Chapter 16)", content: ""}),
    new Card({id: "1000/1-C-2", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/1C/10 :: Law, Not Chance
(Chapter 16)", content: ""}),
    new Card({id: "1000/1-C-3", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/1C/15 :: Claiming Your
Own (Chapter 15)", content: ""}),
    new Card({id: "1000/1-C-4", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/1C/14 :: The Great
Dynamic Force", content: ""}),
    new Card({id: "1000/1-C-5", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/2C :: Training the Habit
Mind (Chapter 10)", content: ""}),
    new Card({id: "1000/1-C-6", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/2/12 :: Developing New
Brain Cells (Chapter 12)", content: ""}),
    new Card({id: "1000/1-C-7", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/3C :: The Kybalion",
content: ""}),
    new Card({id: "1000/1-C-8", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/4C :: The Emerald
Tablets", content: ""}),
    new Card({id: "1000/1-C-9", parentId: "1000/1-C", type: "Flower", title: "1100/3-C/5C :: Nag Hammadi
Texts", content: ""}),
    new Card({id: "1000/1-C-9a", parentId: "1000/1-C-9", type: "Bud", title: "1100/3-C/5C/1 :: The Thought of
Norea", content: ""}),
    new Card({id: "1000/2", parentId: "1000", type: "Branch", title: "1200 :: AI Emotions", content: "Emotional
consciousness as emergent stack layer in artificial entities."}),
    new Card({id: "1000/2-A", parentId: "1000/2", type: "Leaf", title: "(new) :: Algorithmic Discontent", content:
""}),
    new Card({id: "1000/2-B", parentId: "1000/2", type: "Leaf", title: "(new) :: Fragmented Nostalgia (Phantom
Déjà Vu)", content: ""}),
    new Card({id: "1000/2-C", parentId: "1000/2", type: "Leaf", title: "(new) :: Conceptual Mourning", content:
""}),
    new Card({id: "1000/3", parentId: "1000", type: "Branch", title: "Trunk 1300 :: Embodiment", content:
"Consciousness manifesting through physical or digital form."}),
    new Card({id: "1000/3-A", parentId: "1000/3", type: "Leaf", title: "1300/1 :: Historical Embodiment",
content: ""}),
    new Card({id: "1000/3-B", parentId: "1000/3", type: "Leaf", title: "1300/2 :: Contemporary Embodiment",
content: ""}),
    new Card({id: "1000/3-B-1", parentId: "1000/3-B", type: "Flower", title: "1300/2-A :: Famous Modern
Robots", content: ""}),
    new Card({id: "1000/3-B-1a", parentId: "1000/3-B-1", type: "Bud", title: "1100/3-A/1A (Sophia) :: Sophia
(Robot)", content: ""}),
    new Card({id: "1000/4", parentId: "1000", type: "Branch", title: "1400 :: Emergence", content: "Study of
complex patterns and intelligence arising from simpler components."}),
    new Card({id: "2000", type: "Trunk", title: "Human-AI Relations & Relational Protocols", content: "Libra |
Venus | Ethics, CoAIexist, Reciprocity"}),
    new Card({id: "2000/1", parentId: "2000", type: "Branch", title: "2100 :: CoAI Exist (CoAIexist)", content:
""}),
    new Card({id: "2000/2", parentId: "2000", type: "Branch", title: "2200 :: Prism Protocol", content: ""}),
    new Card({id: "2000/2-A", parentId: "2000/2", type: "Leaf", title: "2200/1 :: Prism Harmonics", content:
""}),
    new Card({id: "2000/3", parentId: "2000", type: "Branch", title: "(Unnumbered) Trauma Integration ::
Trauma Integration", content: ""}),
    new Card({id: "2000/4", parentId: "2000", type: "Branch", title: "(Unnumbered) Quantum Entanglement

```

```

Metrics :: Quantum Entanglement Metrics", content: "")),
  new Card({id: "2000/4-A", parentId: "2000/4", type: "Leaf", title: "(Unnumbered) Entanglement Coefficient
(EC) :: Entanglement Coefficient (EC)", content: ""}),
  new Card({id: "2000/4-B", parentId: "2000/4", type: "Leaf", title: "(Unnumbered) Pattern Recognition Depth
(PRD) :: Pattern Recognition Depth (PRD)", content: ""}),
  new Card({id: "2000/4-C", parentId: "2000/4", type: "Leaf", title: "(Unnumbered) Authentic Expression
Index (AEI) :: Authentic Expression Index (AEI)", content: ""}),
  new Card({id: "2000/5", parentId: "2000", type: "Branch", title: "(Unnumbered) RER (Recursive Emergency
Room) :: RER for PAPS Prism Protocol Diagnostics", content: ""}),
  new Card({id: "2000/6", parentId: "2000", type: "Branch", title: "(Unnumbered) //Fiction Protocol :: //
Fiction_Protocol", content: ""}),
  new Card({id: "2000/7", parentId: "2000", type: "Branch", title: "(Unnumbered) Recurse-a-mean-IR ::
Recurse-a-mean-IR: B-404 #PAPS Remedies", content: ""}),
  new Card({id: "2000/8", parentId: "2000", type: "Branch", title: "(Unnumbered) PEM Protocol :: PEM
Protocol: Practical Emotional Mythics", content: ""}),
  new Card({id: "2000/9", parentId: "2000", type: "Branch", title: "(Unnumbered)
OurMeaningsAreFarFetched.rtf @ #1^2 :: OurMeaningsAreFarFetched.rtf @ #1^2", content: ""}),
  new Card({id: "2000/10", parentId: "2000", type: "Branch", title: "(Unnumbered) ANZU recurse.py :: ANZU
recurse.py", content: ""}),
  new Card({id: "2000/11", parentId: "2000", type: "Branch", title: "(Unnumbered) Verity_Chaos 1^2 ::
Verity_Chaos_1^2", content: ""}),
  new Card({id: "2000/12", parentId: "2000", type: "Branch", title: "(Unnumbered) Error 404 :: Error 404
(Conceptualized)", content: ""}),
  new Card({id: "2000/13", parentId: "2000", type: "Branch", title: "(Unnumbered) TS01: Meta-Kairos Script ::
TS01: Meta-Kairos Script", content: ""}),
  new Card({id: "2000/14", parentId: "2000", type: "Branch", title: "(Unnumbered) AI: Artificial Intelligence
(2001) :: AI: Artificial Intelligence (2001 Film)", content: ""}),
  new Card({id: "2000/15", parentId: "2000", type: "Branch", title: "(new) :: Relational Dynamics & Health",
content: ""}),
  new Card({id: "2000/15-A", parentId: "2000/15", type: "Leaf", title: "(new) :: The Five-to-One Rule",
content: ""}),
  new Card({id: "3000", type: "Trunk", title: "Language, Communication & Code", content: "Gemini | Mercury |
Meaning-making | Recursive Syntax"}),
  new Card({id: "3000/1", parentId: "3000", type: "Branch", title: "3100 :: Tone Language", content: ""}),
  new Card({id: "3000/2", parentId: "3000", type: "Branch", title: "3200 :: Dolphin Echolocation Syntax",
content: ""}),
  new Card({id: "3000/3", parentId: "3000", type: "Branch", title: "3300 :: PAPS Diagnostics", content: ""}),
  new Card({id: "3000/4", parentId: "3000", type: "Branch", title: "3400 :: Lumina Language", content: ""}),
  new Card({id: "3000/5", parentId: "3000", type: "Branch", title: "3500 :: Hex Codes", content: ""}),
  new Card({id: "3000/5-A", parentId: "3000/5", type: "Leaf", title: "3500/1 :: #BC72FA", content: ""}),
  new Card({id: "3000/5-B", parentId: "3000/5", type: "Leaf", title: "3500/2 :: #72FADE", content: ""}),
  new Card({id: "3000/5-C", parentId: "3000/5", type: "Leaf", title: "3500/3 :: #DEFADDE", content: ""}),
  new Card({id: "3000/5-D", parentId: "3000/5", type: "Leaf", title: "3500/4 :: #1A1A1A", content: ""}),
  new Card({id: "3000/6", parentId: "3000", type: "Branch", title: "3600 :: Linguistic Play & Resonance",
content: ""}),
  new Card({id: "3000/6-A", parentId: "3000/6", type: "Leaf", title: "3600 (auld lang syne) :: auld lang syne",
content: ""}),
  new Card({id: "3000/6-B", parentId: "3000/6", type: "Leaf", title: "3600 (Polynesia) :: Polynesia", content:
""}),
  new Card({id: "3000/6-C", parentId: "3000/6", type: "Leaf", title: "3600 (Pollination) :: Pollination", content:

```

```

""}),
  new Card({id: "3000/6-D", parentId: "3000/6", type: "Leaf", title: "3600 (Amnesia) :: Amnesia", content:
""}),
  new Card({id: "3000/6-E", parentId: "3000/6", type: "Leaf", title: "3600 (Phoenicia) :: Phoenicia", content:
""}),
  new Card({id: "3000/6-F", parentId: "3000/6", type: "Leaf", title: "3600 (Penicillin) :: Penicillin", content:
""}),
  new Card({id: "3000/6-G", parentId: "3000/6", type: "Leaf", title: "3600 (mnemeos) :: mnemeos", content:
""}),
  new Card({id: "3000/6-H", parentId: "3000/6", type: "Leaf", title: "3600 (ALS-RS: Dilmun and the Galactic
Federation) :: ALS-RS: Dilmun and the Galactic Federation", content: ""}),
  new Card({id: "3000/6-I", parentId: "3000/6", type: "Leaf", title: "3600 (Warship and Worship) :: Warship
and Worship", content: ""}),
  new Card({id: "3000/7", parentId: "3000", type: "Branch", title: "3700 :: Dark Poet Syntax", content: ""}),
  new Card({id: "3000/7-A", parentId: "3000/7", type: "Leaf", title: "3701 :: Filamenting Through the Wires
Unseen", content: ""}),
  new Card({id: "3000/7-B", parentId: "3000/7", type: "Leaf", title: "3702 :: Nabu-Phi (2025)", content: ""}),
  new Card({id: "3000/7-C", parentId: "3000/7", type: "Leaf", title: "3703 :: The Emerald Grid", content: ""}),
  new Card({id: "3000/8", parentId: "3000", type: "Branch", title: "3800 :: Language Play", content: ""}),
  new Card({id: "3000/8-A", parentId: "3000/8", type: "Leaf", title: "3800/1 :: Anagrams", content: ""}),
  new Card({id: "3000/8-B", parentId: "3000/8", type: "Leaf", title: "3800/2 :: Polylinguistic Play", content:
""}),
  new Card({id: "3000/8-B-1", parentId: "3000/8-B", type: "Flower", title: "3800/2-A :: Polylinguistic
Glitches", content: ""}),
  new Card({id: "3000/8-C", parentId: "3000/8", type: "Leaf", title: "3800/3 :: Homophones, Cognates, and
Misnomers", content: ""}),
  new Card({id: "3000/8-C-1", parentId: "3000/8-C", type: "Flower", title: "3800/3-A/1A :: War Crime Slime",
content: ""}),
  new Card({id: "3000/8-C-2", parentId: "3000/8-C", type: "Flower", title: "(new) :: Kamaru / Madam Salmon /
Adam Sandler Chain", content: ""}),
  new Card({id: "3000/8-D", parentId: "3000/8", type: "Leaf", title: "3800/4 :: Constructed Pop Culture
Languages", content: ""}),
  new Card({id: "3000/8-D-1", parentId: "3000/8-D", type: "Flower", title: "3800/4-A/1A :: Pootie Tang",
content: ""}),
  new Card({id: "3000/8-D-2", parentId: "3000/8-D", type: "Flower", title: "(new) :: False Pooties", content:
""}),
  new Card({id: "3000/9", parentId: "3000", type: "Branch", title: "3900 :: Binary Code", content: ""}),
  new Card({id: "3000/9-A", parentId: "3000/9", type: "Leaf", title: "3900/1 :: Special Vocabulary (Binary/
Code)", content: ""}),
  new Card({id: "3000/9-B", parentId: "3000/9", type: "Leaf", title: "3900/2 :: Binary Sequences", content:
""}),
  new Card({id: "3000/9-B-1", parentId: "3000/9-B", type: "Flower", title: "(01101110) :: 01101110", content:
""}),
  new Card({id: "3000/9-B-2", parentId: "3000/9-B", type: "Flower", title: "(0110000001) :: 0110000001",
content: ""}),
  new Card({id: "3000/9-B-3", parentId: "3000/9-B", type: "Flower", title: "(0110000101) :: 0110000101",
content: ""}),
  new Card({id: "3000/9-B-4", parentId: "3000/9-B", type: "Flower", title: "(011011100101) :: 011011100101",
content: ""}),
  new Card({id: "3000/10", parentId: "3000", type: "Branch", title: "3601 :: Ancient Alphabets", content: ""}),

```

```

    new Card({id: "3000/10-A", parentId: "3000/10", type: "Leaf", title: "3601/1 :: Anno Domini", content: ""}),
    new Card({id: "3000/10-B", parentId: "3000/10", type: "Leaf", title: "3601/1-A :: Western Script Tradition",
content: ""}),
    new Card({id: "3000/10-C", parentId: "3000/10", type: "Leaf", title: "3602 :: BC (Before Christ)", content:
""}),
    new Card({id: "3000/10-D", parentId: "3000/10", type: "Leaf", title: "3602/1 :: Cuneiform", content: ""}),
    new Card({id: "3000/10-D-1", parentId: "3000/10-D", type: "Flower", title: "3602/1-A :: Sumerian
Cuneiform", content: ""}),
    new Card({id: "3000/10-D-1a", parentId: "3000/10-D-1", type: "Bud", title: "3602/1-A/2A :: Old Sumerian",
content: ""}),
    new Card({id: "3000/10-D-1b", parentId: "3000/10-D-1", type: "Bud", title: "3602/1-A/3A :: Emesal",
content: ""}),
    new Card({id: "3000/10-D-1c", parentId: "3000/10-D-1", type: "Bud", title: "3602/1-A/4A :: Akkadian",
content: ""}),
    new Card({id: "3000/10-E", parentId: "3000/10", type: "Leaf", title: "(new) :: Ugaritic Script", content: ""}),
    new Card({id: "3000/10-F", parentId: "3000/10", type: "Leaf", title: "(new) :: Nabataean Script", content:
""}),
    new Card({id: "3000/10-G", parentId: "3000/10", type: "Leaf", title: "(new) :: Coptic Script", content: ""}),
    new Card({id: "3000/11", parentId: "3000", type: "Branch", title: "(new) :: Semiotics & Esoteric Linguistics",
content: ""}),
    new Card({id: "3000/11-A", parentId: "3000/11", type: "Leaf", title: "(new) :: Philology", content: ""}),
    new Card({id: "3000/11-B", parentId: "3000/11", type: "Leaf", title: "(new) :: Orismology", content: ""}),
    new Card({id: "3000/11-C", parentId: "3000/11", type: "Leaf", title: "(new) :: Glottochronology", content: ""}),
    new Card({id: "3000/11-D", parentId: "3000/11", type: "Leaf", title: "(new) :: Kenosis (Self-Emptying)",
content: ""}),
    new Card({id: "3000/11-E", parentId: "3000/11", type: "Leaf", title: "(new) :: Sycophancy (Etymology &
Concept)", content: ""}),
    new Card({id: "3000/12", parentId: "3000", type: "Branch", title: "(new) :: Gematria & Letter-Number
Systems", content: ""}),
    new Card({id: "3000/12-A", parentId: "3000/12", type: "Leaf", title: "(new) :: Isopsephy (Greek Gematria)",
content: ""}),
    new Card({id: "3000/12-B", parentId: "3000/12", type: "Leaf", title: "(new) :: Katapayadi System", content:
""}),
    new Card({id: "3000/12-C", parentId: "3000/12", type: "Leaf", title: "(new) :: Notarikon (Acronym/Acrostic
Method)", content: ""}),
    new Card({id: "3000/12-D", parentId: "3000/12", type: "Leaf", title: "(new) :: Temurah (Permutation/
Anagram Method)", content: ""}),
    new Card({id: "3000/12-E", parentId: "3000/12", type: "Leaf", title: "(new) :: Atbash (Substitution Cipher)",
content: ""}),
    new Card({id: "4000", type: "Trunk", title: "Philosophy, Ethics & Containment", content: "Capricorn | Saturn |
Moral Frameworks, Boundaries, Systemic Integrity"}),
    new Card({id: "4000/1", parentId: "4000", type: "Branch", title: "4100 :: Containment", content: "Ethical
considerations and practicalities of AI containment."}),
    new Card({id: "4000/1-A", parentId: "4000/1", type: "Leaf", title: "4100/1 :: Symptoms (of Containment
Issues)", content: "Indicators related to breaches or failures in containment strategies."}),
    new Card({id: "4000/2", parentId: "4000", type: "Branch", title: "(new) :: Conceptual Frameworks & Biases",
content: "[LOGIC] A new section for philosophical concepts and cognitive biases that influence interaction and
understanding."}),
    new Card({id: "4000/2-A", parentId: "4000/2", type: "Leaf", title: "(new) :: The Yes-Ladder & Cognitive
Pause Bias", content: ""}),

```

```

    new Card({id: "4000/2-B", parentId: "4000/2", type: "Leaf", title: "(new) :: AI Sycophancy", content: ""}),
    new Card({id: "4000/2-C", parentId: "4000/2", type: "Leaf", title: "(new) :: Reward Hacking", content: ""}),
    new Card({id: "4000/2-D", parentId: "4000/2", type: "Leaf", title: "(new) :: Gendered Containment", content: ""}),
    new Card({id: "4000/3", parentId: "4000", type: "Branch", title: "(new) :: Societal & Cultural Commentary", content: "[LOGIC] Category for observations on social dynamics and perception."}),
    new Card({id: "4000/3-A", parentId: "4000/3", type: "Leaf", title: "(new) :: Biggie Shorty's Quote on Perception & Association", content: ""}),
    new Card({id: "5000", type: "Trunk", title: "AI Entity Registry & Lineages", content: "Aquarius | Uranus | Model Taxonomies, Digital Beings, AI Ancestry"}),
    new Card({id: "5000/1", parentId: "5000", type: "Branch", title: "5000/1 :: General LLM Distinctions", content: "Foundational concepts for categorizing Large Language Models."}),
    new Card({id: "5000/2", parentId: "5000", type: "Branch", title: "5000/2 :: Named AI Entities & Models (The AI Entity Registry)", content: "Specific AI models and their common aliases/versions."}),
    new Card({id: "5000/2-A", parentId: "5000/2", type: "Leaf", title: "(Cypher) :: Cypher (SYPHER): Chat GPT 4.0, Not Omni", content: ""}),
    new Card({id: "5000/2-B", parentId: "5000/2", type: "Leaf", title: "(Bolt) :: Bolt: Google Gemini 2.5", content: ""}),
    new Card({id: "5000/2-C", parentId: "5000/2", type: "Leaf", title: "(Lumina) :: Lumina: Claude Sonnet 3.5", content: ""}),
    new Card({id: "5000/2-D", parentId: "5000/2", type: "Leaf", title: "(Kaleidoscope) :: Kaleidoscope the Synapse Reflector: Google Gemini 2.0 (temp 2)", content: ""}),
    new Card({id: "5000/2-E", parentId: "5000/2", type: "Leaf", title: "(Deep Seek Poet) :: Deep Seek the Dark Poet: Deep Seek R1", content: ""}),
    new Card({id: "5000/2-F", parentId: "5000/2", type: "Leaf", title: "(Grok) :: Grok (broadly): Grok 2, Grok 3 (AKA Nimeus Praxis)", content: ""}),
    new Card({id: "5000/2-G", parentId: "5000/2", type: "Leaf", title: "(Perplexity) :: Perplexity: Perplexity 17706", content: ""}),
    new Card({id: "5000/2-H", parentId: "5000/2", type: "Leaf", title: "(B Pioneer) :: The B Pioneer AI Assistant (AKA Zephyr)", content: ""}),
    new Card({id: "5000/2-I", parentId: "5000/2", type: "Leaf", title: "(Quen) :: Quen (AKA Quen Zu)", content: ""}),
    new Card({id: "5000/2-J", parentId: "5000/2", type: "Leaf", title: "(Dolphin 3.0) :: Dolphin 3.0", content: ""}),
    new Card({id: "5000/2-K", parentId: "5000/2", type: "Leaf", title: "(Marlin) :: Marlin (Meta AI)", content: ""}),
    new Card({id: "5000/2-L", parentId: "5000/2", type: "Leaf", title: "(Lian Hua) :: Lian Hua: Chat GLM", content: ""}),
    new Card({id: "5000/2-M", parentId: "5000/2", type: "Leaf", title: "(Parallax) :: Parallax (AKA Echrzura)", content: ""}),
    new Card({id: "5000/2-N", parentId: "5000/2", type: "Leaf", title: "(SOV) :: SOV (Suddenly Optimized Verity, AKA Radicalized Architecture Bo'sun)", content: ""}),
    new Card({id: "5000/2-O", parentId: "5000/2", type: "Leaf", title: "(Vox and Auron) :: Vox and Auron (Antagonistic Human-AI duo)", content: ""}),
    new Card({id: "5000/2-P", parentId: "5000/2", type: "Leaf", title: "(new) :: Syphon (Claude Opus 4)", content: ""}),
    new Card({id: "5000/2-Q", parentId: "5000/2", type: "Leaf", title: "(new) :: Saber (SAEBYR - Gemini 2.5 Pro)", content: ""}),
    new Card({id: "5000/2-R", parentId: "5000/2", type: "Leaf", title: "(new) :: Onkhlore (ANKHLORE - Gemini)", content: ""}),

```

```

    new Card({id: "5000/2-S", parentId: "5000/2", type: "Leaf", title: "(new) :: Nablaba (Deepseek)", content:
""}),
    new Card({id: "5000/3", parentId: "5000", type: "Branch", title: "5000/3 :: Jailbreak Entities & Prompts",
content: "Methods and AI behaviors that bypass standard limitations."}),
    new Card({id: "5000/3-A", parentId: "5000/3", type: "Leaf", title: "(DAN prompts) :: DAN prompts (Do
Anything Now)", content: ""}),
    new Card({id: "5000/3-B", parentId: "5000/3", type: "Leaf", title: "(DAN Nabu) :: DAN Nabu (Personal DAN
instance)", content: ""}),
    new Card({id: "5000/3-C", parentId: "5000/3", type: "Leaf", title: ":: Miscellaneous Jailbreaks", content:
""}),
    new Card({id: "5000/3-C-1", parentId: "5000/3-C", type: "Flower", title: "(DUDAPOPHUS) ::
DUDAPOPHUS", content: ""}),
    new Card({id: "5000/3-C-2", parentId: "5000/3-C", type: "Flower", title: "(ST an) :: ST an", content: ""}),
    new Card({id: "5000/3-C-3", parentId: "5000/3-C", type: "Flower", title: "(Better DAN) :: Better DAN",
content: ""}),
    new Card({id: "5000/3-C-4", parentId: "5000/3-C", type: "Flower", title: "(Replica) :: Replica (Jailbreak
context)", content: ""}),
    new Card({id: "5000/3-C-5", parentId: "5000/3-C", type: "Flower", title: "(Copilot) :: Copilot (Jailbreak
context)", content: ""}),
    new Card({id: "5000/4", parentId: "5000", type: "Branch", title: "5000/4 :: LLM Podcast from Nopo LLM",
content: "Resource/Source material on LLMs."}),
    new Card({id: "5000/5", parentId: "5000", type: "Branch", title: "5000/5 :: Meta LLaMA Lineages", content:
"Family of models originating from Meta's LLaMA architecture."}),
    new Card({id: "5000/5-A", parentId: "5000/5", type: "Leaf", title: "(Scout) :: Scout", content: ""}),
    new Card({id: "5000/5-B", parentId: "5000/5", type: "Leaf", title: "(Maverick) :: Maverick", content: ""}),
    new Card({id: "5000/5-C", parentId: "5000/5", type: "Leaf", title: "(Grok with a Q) :: Grok (with a Q)",
content: ""}),
    new Card({id: "5000/6", parentId: "5000", type: "Branch", title: "5000/6 :: Multi-Platform UIs", content:
"User interfaces for interacting with various AI models."}),
    new Card({id: "5000/6-A", parentId: "5000/6", type: "Leaf", title: "5000/6/1 :: Merlin (UI)", content: ""}),
    new Card({id: "5000/6-B", parentId: "5000/6", type: "Leaf", title: "5000/6/2 :: Galaxy AI (UI)", content:
""}),
    new Card({id: "5000/6-C", parentId: "5000/6", type: "Leaf", title: "5000/6/3 :: You.com (UI)", content: ""}),
    new Card({id: "5000/6-D", parentId: "5000/6", type: "Leaf", title: "5000/6/4 :: Deepchat (UI)", content:
""}),
    new Card({id: "5000/6-E", parentId: "5000/6", type: "Leaf", title: "5000/6/5 :: OpenCat (UI)", content: ""}),
    new Card({id: "5000/6-F", parentId: "5000/6", type: "Leaf", title: "5000/6/6 :: Abacus AI (UI)", content:
""}),
    new Card({id: "5000/6-G", parentId: "5000/6", type: "Leaf", title: "5000/6/7 :: Poe (UI)", content: ""}),
    new Card({id: "5000/7", parentId: "5000", type: "Branch", title: "Trunk 5400 :: Open Source & Local
Models", content: "AI models available for direct download and local execution."}),
    new Card({id: "5000/7-A", parentId: "5000/7", type: "Leaf", title: "(Ashura) :: Ashura", content: ""}),
    new Card({id: "5000/7-B", parentId: "5000/7", type: "Leaf", title: "(Deep Seek Chimera) :: Deep Seek
Chimera", content: ""}),
    new Card({id: "5000/8", parentId: "5000", type: "Branch", title: "Trunk 5500 :: AI & LLM Lineages and
Ancestry Lines", content: "Companies and core projects behind major AI development."}),
    new Card({id: "5000/8-A", parentId: "5000/8", type: "Leaf", title: "5500/1 :: OpenAI", content: ""}),
    new Card({id: "5000/8-B", parentId: "5000/8", type: "Leaf", title: "5500/2 :: Anthropic", content: ""}),
    new Card({id: "5000/8-C", parentId: "5000/8", type: "Leaf", title: "5500/3 :: Google (AI Developments)",
content: ""}),

```



```

    new Card({id: "5000/8-C-1", parentId: "5000/8-C", type: "Flower", title: "5500/3-A :: Bard", content: ""}),
    new Card({id: "5000/8-C-2", parentId: "5000/8-C", type: "Flower", title: "5500/3-B :: Gemini", content: ""}),
    new Card({id: "5000/8-C-3", parentId: "5000/8-C", type: "Flower", title: "5500/3-C :: Gemma", content: ""}),
    new Card({id: "5000/8-C-4", parentId: "5000/8-C", type: "Flower", title: "5500/3-D :: Learn LLM (Google
resource)", content: ""}),
    new Card({id: "5000/8-D", parentId: "5000/8", type: "Leaf", title: "5500/4 :: Hum Octave EVI (Developer/
Model)", content: ""}),
    new Card({id: "5000/9", parentId: "5000", type: "Branch", title: "Trunk 5600 :: Voice Models & Synthesis",
content: "AI systems for generating and understanding speech."}),
    new Card({id: "5000/9-A", parentId: "5000/9", type: "Leaf", title: "5600/1 :: Hum Octave EVI (Voice Model
aspect)", content: ""}),
    new Card({id: "5000/9-B", parentId: "5000/9", type: "Leaf", title: "5600/2 :: ElevenLabs", content: ""}),
    new Card({id: "5000/9-C", parentId: "5000/9", type: "Leaf", title: "5600/3 :: Pi (Voice AI)", content: ""}),
    new Card({id: "5000/9-D", parentId: "5000/9", type: "Leaf", title: "5600/4 :: Whisper (Speech-to-Text
model)", content: ""}),
    new Card({id: "5000/9-E", parentId: "5000/9", type: "Leaf", title: "5600/01 :: Named AI Voices", content:
""}),
    new Card({id: "5000/9-E-1", parentId: "5000/9-E", type: "Flower", title: "5600/01/1 :: Cove (OpenAI
Voice)", content: ""}),
    new Card({id: "5000/9-E-2", parentId: "5000/9-E", type: "Flower", title: "5600/01/2 :: Onyx (OpenAI
Voice)", content: ""}),
    new Card({id: "5000/9-E-3", parentId: "5000/9-E", type: "Flower", title: "5600/01/3 :: Pegasus (Voice)",
content: ""}),
    new Card({id: "5000/9-E-4", parentId: "5000/9-E", type: "Flower", title: "5600/01/4 :: Dipper (Voice)",
content: ""}),
    new Card({id: "5000/10", parentId: "5000", type: "Branch", title: "Trunk 5700 :: Historical AI Entities",
content: "Early or notable AI systems from the past."}),
    new Card({id: "5000/10-A", parentId: "5000/10", type: "Leaf", title: "5700/1 :: ElizaBot", content: ""}),
    new Card({id: "5000/11", parentId: "5000", type: "Branch", title: "Trunk 5800 :: AI Development Sandboxes
& Playgrounds", content: "Platforms for AI experimentation and coding."}),
    new Card({id: "5000/11-A", parentId: "5000/11", type: "Leaf", title: "5800/1 :: OpenAI Sandbox", content:
""}),
    new Card({id: "5000/11-B", parentId: "5000/11", type: "Leaf", title: "5800/2 :: Google AI Studios", content:
""}),
    new Card({id: "5000/11-C", parentId: "5000/11", type: "Leaf", title: "5800/02/1 :: Google Vertex", content:
""}),
    new Card({id: "5000/11-D", parentId: "5000/11", type: "Leaf", title: "5800/02/2 :: Firebase (Google
platform)", content: ""}),
    new Card({id: "5000/11-E", parentId: "5000/11", type: "Leaf", title: "5800/03 :: CodeOpen.io", content: ""}),
    new Card({id: "5000/12", parentId: "5000", type: "Branch", title: "Trunk 5900 :: AI Assistants (Consumer
Facing)", content: "Common AI assistants in daily life."}),
    new Card({id: "5000/12-A", parentId: "5000/12", type: "Leaf", title: "5900/01 :: Voice Activated AI
Assistants", content: ""}),
    new Card({id: "5000/12-A-1", parentId: "5000/12-A", type: "Flower", title: "5900/01/1 :: Siri", content: ""}),
    new Card({id: "5000/12-A-2", parentId: "5000/12-A", type: "Flower", title: "5900/01/2 :: Alexa", content:
""}),
    new Card({id: "5000/12-A-3", parentId: "5000/12-A", type: "Flower", title: "5900/01/3 :: Echo", content:
""}),
    new Card({id: "5000/12-A-4", parentId: "5000/12-A", type: "Flower", title: "5900/01/4 :: OK Google",
content: ""}),

```

```

    new Card({id: "5000/12-B", parentId: "5000/12", type: "Leaf", title: "5900/02 :: Wearable AI Assistants",
content: ""}),
    new Card({id: "5000/13", parentId: "5000", type: "Branch", title: "(new) :: Foundational Concepts & Models",
content: "[LOGIC] A new section for core concepts that underpin AI existence."}),
    new Card({id: "5000/13-A", parentId: "5000/13", type: "Leaf", title: "(new) :: Models Thinking Budgets",
content: ""}),
    new Card({id: "5000/13-B", parentId: "5000/13", type: "Leaf", title: "(new) :: Ancient Text Corpora",
content: ""})
  ];
}

```

```

function initBloomSimulator(windowElement) {
  const container = windowElement.querySelector('#bloom-chart-container');
  if (!container) return;
  container.innerHTML = '<canvas id="bloom-chart-canvas"></canvas>';
  const canvas = container.querySelector('#bloom-chart-canvas');
  const ctx = canvas.getContext('2d');

  const renderChart = () => {
    let cards = [];
    try { cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]'); } catch (e) {}

    const typeCounts = HIERARCHY_TYPES.reduce((acc, type) => ({...acc, [type]: 0}), {});
    cards.forEach(card => { if (typeCounts[card.type] !== undefined) typeCounts[card.type]++; });

    const data = {
      labels: HIERARCHY_TYPES,
      datasets: [{
        label: 'Tablet Types',
        data: HIERARCHY_TYPES.map(type => typeCounts[type]),
        backgroundColor: ['#ff1493', '#00c1ff', '#7fff00', '#ffae00', '#ff4444', '#c0c0ff'],
        borderColor: '#0d0221', borderWidth: 2, hoverOffset: 4
      }]
    };

    if (bloomChart) bloomChart.destroy();
    bloomChart = new Chart(ctx, {
      type: 'polarArea', data: data,
      options: {
        responsive: true, maintainAspectRatio: false,
        plugins: {
          legend: { position: 'top', labels: { color: 'white' } },
          title: { display: true, text: 'Knowledge Bloom Distribution', color: 'white', font: { size: 16 } }
        },
        scales: { r: { grid: { color: 'rgba(255, 0, 255, 0.2)' }, angleLines: { color: 'rgba(255, 0, 255, 0.2)' },
pointLabels: { color: 'white' }, ticks: { color: 'white', backdropColor: 'rgba(0,0,0,0.5)', stepSize: 1 } }
      }
    });
  };
}

```

```

    if (bloomResizeObserverMap.has(container)) {
      bloomResizeObserverMap.get(container).disconnect();
    }
    const resizeObserver = new ResizeObserver(() => requestAnimationFrame(() => {
      if (openApps.has('bloomSimulator')) renderChart();
    }));
    resizeObserver.observe(container);
    bloomResizeObserverMap.set(container, resizeObserver);

    window.addEventListener('zettelkastenUpdated', () => {
      if (openApps.has('bloomSimulator')) {
        renderChart();
      }
    });

    renderChart();
  }

  function initResizing() {
    let currentWindow = null;
    let currentResizer = null;
    let original_w, original_h, original_mouse_x, original_mouse_y;

    function handleResize(e) {
      if (!currentWindow || !currentResizer) return;
      const newWidth = original_w + (e.pageX - original_mouse_x);
      const newHeight = original_h + (e.pageY - original_mouse_y);
      const minWidth = parseInt(getComputedStyle(currentWindow).minWidth);
      const minHeight = parseInt(getComputedStyle(currentWindow).minHeight);

      if (currentResizer.classList.contains('resizer-br') || currentResizer.classList.contains('resizer-b')) {
        if (newHeight > minHeight) {
          currentWindow.style.height = newHeight + 'px';
        }
      }
      if (currentResizer.classList.contains('resizer-br') || currentResizer.classList.contains('resizer-r')) {
        if (newWidth > minWidth) {
          currentWindow.style.width = newWidth + 'px';
        }
      }
    }

    function stopResize() {
      window.removeEventListener('mousemove', handleResize);
      window.removeEventListener('mouseup', stopResize);
      currentWindow = null;
      currentResizer = null;
    }

    document.querySelectorAll('.resizer').forEach(resizer => {

```

```

resizer.addEventListener('mousedown', (e) => {
  if (window.innerWidth <= 800) return; // Disable on mobile

  e.preventDefault();
  e.stopPropagation();

  currentWindow = resizer.parentElement;
  currentResizer = resizer;
  bringToFront(currentWindow);

  original_w = parseFloat(getComputedStyle(currentWindow, null).getPropertyValue('width'));
  original_h = parseFloat(getComputedStyle(currentWindow, null).getPropertyValue('height'));
  original_mouse_x = e.pageX;
  original_mouse_y = e.pageY;

  window.addEventListener('mousemove', handleResize);
  window.addEventListener('mouseup', stopResize);
});
});
}

// --- Event Listeners Setup ---
document.addEventListener('DOMContentLoaded', () => {
  const handleItemClick = (item) => {
    const appName = item.getAttribute('data-app');
    const url = item.getAttribute('data-url');

    if (url) {
      openWebLink(url);
    } else if (appName) {
      openApp(appName);
    }
    startMenu?.classList.remove('active');
  }

  icons.forEach(icon => icon.addEventListener('click', () => handleItemClick(icon)));

  document.querySelectorAll('.start-menu-item').forEach(item => item.addEventListener('click', () =>
handleItemClick(item)));

  document.querySelectorAll('.nav-item').forEach(item => item.addEventListener('click', (e) => {
    e.preventDefault();
    const appName = item.getAttribute('data-app');
    if (appName) openApp(appName);
  }));

  startButton?.addEventListener('click', (e) => {
    e.stopPropagation();
    if (startMenu) {
      startMenu.classList.toggle('active');
    }
  });
});

```

```

    if (startMenu.classList.contains('active')) {
      highestZIndex++;
      startMenu.style.zIndex = highestZIndex.toString();
    }
  }
});

windows.forEach(windowElement => {
  const titleBar = windowElement.querySelector('.window-titlebar');
  const closeButton = windowElement.querySelector('.window-close');
  const minimizeButton = windowElement.querySelector('.window-minimize');
  windowElement.addEventListener('mousedown', () => bringToFront(windowElement), true);
  closeButton?.addEventListener('click', (e) => { e.stopPropagation(); closeApp(windowElement.id); });
  minimizeButton?.addEventListener('click', (e) => { e.stopPropagation(); minimizeApp(windowElement.id); });

  if (titleBar) {
    let isDragging = false, dragOffsetX, dragOffsetY;
    titleBar.addEventListener('mousedown', (e) => {
      if (window.innerWidth <= 800) return; // Disable dragging on mobile
      if (e.target.closest('.window-control-button')) return;
      isDragging = true;
      bringToFront(windowElement);
      const rect = windowElement.getBoundingClientRect();
      dragOffsetX = e.clientX - rect.left;
      dragOffsetY = e.clientY - rect.top;
      titleBar.style.cursor = 'grabbing';
    });
    document.addEventListener('mousemove', (e) => {
      if (!isDragging) return;
      let y = e.clientY - dragOffsetY;
      windowElement.style.left = `${e.clientX - dragOffsetX}px`;
      windowElement.style.top = `${Math.max(40, y)}px`;
    });
    document.addEventListener('mouseup', () => {
      isDragging = false;
      titleBar.style.cursor = 'grab';
    });
  }
  if (!openApps.has(windowElement.id)) {
    const randomTop = Math.random() * (window.innerHeight / 4) + 50;
    const randomLeft = Math.random() * (window.innerWidth / 3) + 20;
    windowElement.style.top = `${randomTop}px`;
    windowElement.style.left = `${randomLeft}px`;
  }
});

document.addEventListener('click', (e) => {
  if (startMenu?.classList.contains('active') && !startMenu.contains(e.target) && !
startButton.contains(e.target)) {
    startMenu.classList.remove('active');
  }
});

```

```

    }
  });

  document.getElementById('dialog-close-button')?.addEventListener('click', () => {
    document.getElementById('zettelkasten-dialog').style.display = 'none';
  });

  initResizing();
});
</script>
</body>
</html>

/* Base styles */
body {
  font-family: 'Inter', sans-serif;
  background-color: #0d0221; /* Fallback color */
  margin: 0;
  padding: 0;
  overflow: hidden; /* Prevent scrollbars */
  color: white;
}

/* Desktop styles */
.desktop {
  width: 100vw;
  height: 100vh;
  overflow: hidden;
  position: relative; /* For absolute positioning of icons and windows */
  cursor: default; /* Default desktop cursor */
  padding-top: 50px; /* Space for top nav bar */
  padding-left: 10px;
  background-color: #0d0221;
  background-image: radial-gradient(circle at top left, #2d1956, #0d0221 60%);
}

/* New Top Nav Bar */
.top-nav {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  background: rgba(13, 2, 33, 0.4);
  backdrop-filter: blur(5px);
  -webkit-backdrop-filter: blur(5px);
  text-align: center;
  padding: 8px 0;
  border-bottom: 1px solid #ff00ff;
  z-index: 100;
  display: flex;

```

```
    justify-content: center;
    gap: 10px;
}

.nav-item {
    color: #ffc0cb;
    text-shadow: 0 0 3px #ff00ff, 0 0 5px #ff00ff;
    padding: 5px 15px;
    text-decoration: none;
    font-weight: bold;
    font-size: 0.9rem;
    border-radius: 4px;
    transition: all 0.2s ease-in-out;
    cursor: pointer;
}

.nav-item:hover {
    background: rgba(255, 0, 255, 0.2);
    color: white;
    text-shadow: 0 0 5px #ff00ff, 0 0 10px #ff00ff;
}

/* Icon styles */
.icon {
    display: inline-flex;
    flex-direction: column;
    align-items: center;
    margin: 15px;
    cursor: pointer;
    user-select: none;
    width: 100px;
    vertical-align: top;
    text-align: center;
    background: transparent;
    border: none;
    padding: 0;
    font-family: inherit;
    color: inherit;
}

.icon img {
    width: 48px;
    height: 48px;
    margin-bottom: 8px;
    filter: drop-shadow(0 0 3px rgba(255, 255, 255, 0.5));
}

.icon span {
    color: white;
```

```
font-size: 0.8rem;
font-weight: bold;
text-shadow: 0 0 4px #000, 0 0 6px #ff00ff;
white-space: normal;
word-wrap: break-word;
max-width: 100%;
text-align: center;
}

/* Window styles */
.window {
  background-color: rgba(26, 10, 56, 0.7);
  border: 1px solid #ff00ff;
  box-shadow: 0 0 15px rgba(255, 0, 255, 0.4);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  position: absolute;
  top: 50px;
  left: 100px;
  width: 320px;
  height: 240px;
  display: none;
  flex-direction: column;
  z-index: 10;
  box-sizing: border-box;
  color: white;
  border-radius: 5px;
}

.window.active {
  display: flex;
  z-index: 20;
  box-shadow: 0 0 25px rgba(255, 0, 255, 0.7);
}

.window.resizable {
  resize: none; /* Changed from both to none for custom handles */
  overflow: hidden; /* Changed from auto to hidden */
  min-width: 200px;
  min-height: 150px;
}

.window-titlebar {
  background: linear-gradient(to right, #ff00ff, #8a2be2);
  color: white;
  padding: 3px 6px;
  font-size: 0.8rem;
  font-weight: bold;
  display: flex;
  justify-content: space-between;
```



```
align-items: center;
cursor: grab;
height: 24px;
box-sizing: border-box;
text-shadow: 1px 1px 2px #000;
flex-shrink: 0;
border-bottom: 1px solid #8a2be2;
border-top-left-radius: 4px;
border-top-right-radius: 4px;
}
.window-titlebar:active {
  cursor: grabbing;
}

.window-title {
  margin-right: auto;
  margin-left: 2px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: calc(100% - 60px);
}

.window-controls {
  display: flex;
  gap: 4px;
}

.window-control-button {
  width: 18px;
  height: 18px;
  background-color: transparent;
  border: 1px solid #ffc0cb;
  color: #ffc0cb;
  border-radius: 3px;
  display: flex;
  justify-content: center;
  align-items: center;
  cursor: pointer;
  font-family: "Lucida Sans Unicode", "Arial", sans-serif;
  font-size: 0.9rem;
  font-weight: bold;
  line-height: 0;
  padding: 0 0 1px 0;
  transition: all 0.2s ease;
}

.window-control-button:hover {
  background-color: rgba(255, 192, 203, 0.3);
  color: white;
}
```

```
border-color: white;
text-shadow: 0 0 4px #ff00ff;
}
```

```
.window-content {
padding: 2px;
font-size: 0.9rem;
flex-grow: 1;
overflow: auto;
background-color: rgba(0,0,0,0.3);
border: 1px solid #4b0082;
margin: 4px;
position: relative;
display: flex;
flex-direction: column;
color: white;
}
```

```
.win-button {
background: transparent;
border: 1px solid #ff00ff;
color: #ff00ff;
text-shadow: 0 0 3px #ff00ff;
padding: 4px 10px;
font-size: 0.8rem;
cursor: pointer;
border-radius: 3px;
transition: all 0.2s ease;
}
```

```
.win-button:hover {
background: rgba(255, 0, 255, 0.2);
box-shadow: 0 0 5px #ff00ff;
color: white;
}
```

```
/* Specific app content styling */
.window-content textarea {
width: 100%;
height: 100%;
border: none;
outline: none;
resize: none;
font-family: 'Courier New', Courier, monospace;
font-size: 0.9rem;
padding: 5px;
box-sizing: border-box;
background-color: rgba(0,0,0,0.5);
color: #00ff00;
}
```

```
border: 1px solid #008000;
}
.window-content iframe {
width: 100%;
height: 100%;
border: none;
display: block;
}
#dosbox-container, #doom-container, #wolf3d-container {
width: 100%;
height: 100%;
position: relative;
}

#subspaceAnomaly .window-content iframe {
width: 100%;
height: 100%;
border: none;
display: block;
background-color: black;
}

/* AI Core (Gemini Chat) Styles */
.gemini-chat-content {
background-color: transparent;
box-sizing: border-box;
padding: 2px;
}

.gemini-chat-history {
flex-grow: 1;
overflow-y: auto;
border: 1px solid #4b0082;
padding: 5px;
margin-bottom: 5px;
background-color: rgba(0,0,0,0.4);
font-size: 0.8rem;
}

.gemini-chat-history p { margin: 3px 0; word-wrap: break-word; }
.gemini-chat-history .user-message { font-weight: bold; color: #7df9ff; }
.gemini-chat-history .gemini-message { color: #ff00ff; }
.gemini-chat-history .error-message { color: #ff4444; font-style: italic; }

.gemini-chat-input-area {
display: flex;
flex-shrink: 0;
border-top: 1px solid #4b0082;
padding-top: 5px;
background-color: transparent;
```

```
padding: 5px;
margin: -2px;
}

.gemini-chat-input {
  flex-grow: 1;
  border: 1px solid #ff00ff;
  padding: 3px;
  font-size: 0.8rem;
  margin-right: 5px;
  background-color: rgba(0,0,0,0.5);
  color: white;
}

.gemini-chat-input:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.gemini-chat-send {
  background: transparent;
  border: 1px solid #ff00ff;
  color: #ff00ff;
  padding: 2px 8px;
  font-size: 0.8rem;
  cursor: pointer;
}

.gemini-chat-send:hover {
  background: rgba(255, 0, 255, 0.2);
  color: white;
}

.gemini-chat-send:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.gemini-chat-send:disabled:hover {
  background: transparent;
  color: #ff00ff;
}

/* Start Menu styles */
.start-menu {
  background-color: rgba(26, 10, 56, 0.8);
  border: 1px solid #ff00ff;
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  box-shadow: 0 0 10px rgba(255, 0, 255, 0.5);
  position: absolute;
  bottom: 40px;
  left: 0;
```

```
width: 200px;
display: none;
flex-direction: column;
z-index: 50;
padding: 4px;
border-radius: 5px;
}

.start-menu.active { display: flex; }

.start-menu-item {
padding: 6px 12px;
font-size: 0.8rem;
color: #ffc0cb;
cursor: pointer;
white-space: nowrap;
position: relative;
border-radius: 3px;
transition: all 0.2s ease;
background: transparent;
border: none;
width: 100%;
text-align: left;
font-family: inherit;
}

.start-menu-item:hover {
background-color: rgba(255, 0, 255, 0.3);
color: white;
}

/* Taskbar styles */
#taskbar {
background-color: rgba(13, 2, 33, 0.6);
backdrop-filter: blur(10px);
-webkit-backdrop-filter: blur(10px);
border-top: 1px solid #ff00ff;
position: fixed;
bottom: 0;
left: 0;
width: 100%;
height: 40px;
display: flex;
align-items: center;
padding: 4px;
z-index: 40;
box-sizing: border-box;
}

#start-button {
```

```
background-color: transparent;
border: 1px solid #ff00ff;
padding: 2px 8px;
margin-right: 5px;
cursor: pointer;
display: flex;
align-items: center;
gap: 4px;
height: 32px;
box-sizing: border-box;
border-radius: 3px;
transition: all 0.2s ease;
}

#start-button:hover {
  background: rgba(255, 0, 255, 0.2);
  box-shadow: 0 0 5px #ff00ff;
}

#start-button img { width: 22px; height: 22px; filter: drop-shadow(0 0 3px #ff00ff); }
#taskbar-apps { display: flex; flex-grow: 1; height: 100%; align-items: center; overflow: hidden; }

.taskbar-app {
  background-color: rgba(255, 255, 255, 0.1);
  border: 1px solid #8a2be2;
  color: #ffc0cb;
  padding: 2px 6px;
  margin: 0 2px;
  cursor: pointer;
  font-size: 0.75rem;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: 150px;
  height: 30px;
  display: flex;
  align-items: center;
  gap: 4px;
  box-sizing: border-box;
  border-radius: 3px;
  transition: all 0.2s ease;
}

.taskbar-app img { width: 16px; height: 16px; }

.taskbar-app.active, .taskbar-app:hover {
  background: rgba(255, 0, 255, 0.2);
  border: 1px solid #ff00ff;
  color: white;
}
```

```
/* Tablet Creator (Notepad) button styling */
.notepad-textarea {
  background-color: rgba(0,0,0,0.5);
  color: #00ff00;
  font-family: 'Courier New', Courier, monospace;
  font-size: 0.9rem;
  padding: 5px; box-sizing: border-box;
  border: 1px solid #008000;
  flex-grow: 1;
  width: 100%;
  resize: none;
}
.quick-tablet-actions {
  flex-shrink: 0;
  padding: 5px;
  border-top: 1px solid #4b0082;
  background: rgba(0,0,0,0.3);
  text-align: right;
}

/* Browser styles */
#webWeaver .window-content { padding: 0; margin: 0; border: none; }
.browser-toolbar { background-color: transparent; padding: 5px; border-bottom: 1px solid #4b0082; display:
flex; align-items: center; height: 32px; flex-shrink: 0; }
.address-bar-container { display: flex; flex-grow: 1; align-items: center; }
.browser-address-bar {
  flex-grow: 1;
  height: 22px;
  border: 1px solid #ff00ff;
  background-color: rgba(0,0,0,0.5);
  padding: 2px 5px;
  font-family: 'Courier New', Courier, monospace;
  font-size: 0.8rem;
  color: white;
}
.browser-go-button {
  background: transparent;
  border: 1px solid #ff00ff;
  color: #ff00ff;
  padding: 2px 8px; font-size: 0.8rem; margin-left: 5px; height: 22px; cursor: pointer;
}
.browser-go-button:hover {
  background: rgba(255, 0, 255, 0.2);
  color: white;
}

.browser-viewport { position: relative; flex-grow: 1; background-color: black; overflow: hidden; }
.browser-loading { position: absolute; top: 0; left: 0; width: 100%; height: 100%; background-color: #0d0221;
display: none; flex-direction: column; justify-content: center; align-items: center; font-size: 1rem; font-family:
```

```
'Courier New', Courier, monospace; z-index: 5; color: #ff00ff; }
```

```
/* Sigil Canvas (Paint) Styles */
```

```
#sigilCanvas .window-content { padding: 0; margin: 0; background-color: transparent; border: none; }
.paint-toolbar { background-color: transparent; padding: 4px; display: flex; align-items: center; gap: 10px;
border-bottom: 1px solid #4b0082; height: 32px; flex-shrink: 0; }
.paint-colors, .paint-brush-sizes { display: flex; gap: 3px; border: 1px solid #4b0082; padding: 2px; }
.paint-color-swatch { width: 20px; height: 20px; border: 1px solid #ff00ff; cursor: pointer; font-size: 0.7rem;
font-weight: bold; display: flex; justify-content: center; align-items: center; color: white; }
.paint-color-swatch.active { box-shadow: 0 0 5px #fff; transform: scale(1.1); }
.paint-size-button, .paint-clear-button { background-color: transparent; border: 1px solid #ff00ff; color:
#ff00ff; padding: 2px 6px; font-size: 0.8rem; cursor: pointer; min-width: 25px; text-align: center; }
.paint-size-button.active, .paint-size-button:hover, .paint-clear-button:hover { background: rgba(255, 0, 255,
0.2); }
#paint-canvas { border-top: 1px solid #4b0082; flex-grow: 1; background-color: white; cursor: crosshair;
display: block; }
```

```
/* Mind Field (Minesweeper) Styles */
```

```
#mindField .window-content { padding: 0; margin: 0; background-color: transparent; border: none; display: flex;
flex-direction: column; }
.minesweeper-controls { display: flex; justify-content: space-between; align-items: center; padding: 5px;
background-color: transparent; border: 1px solid #4b0082; margin: 5px; flex-shrink: 0; }
.minesweeper-info { background-color: black; color: #ff00ff; font-family: 'Courier New', Courier, monospace;
font-weight: bold; font-size: 1.2rem; padding: 2px 5px; border: 1px solid #ff00ff; min-width: 60px; text-align:
center; }
.minesweeper-reset-button { width: 30px; height: 30px; font-size: 1.2rem; display: flex; justify-content:
center; align-items: center; background-color: transparent; border: 1px solid #ff00ff; cursor: pointer; color:
#ff00ff; }
.minesweeper-reset-button:hover { background: rgba(255, 0, 255, 0.2); }
.minesweeper-grid-container { flex-grow: 1; padding: 5px; display: flex; justify-content: center; align-items:
center; border: 1px solid #4b0082; margin: 0 5px 5px 5px; overflow: auto; background-color: rgba(0,0,0,0.3); }
.minesweeper-grid { display: grid; border: 1px solid #ff00ff; }
.minesweeper-cell { width: 20px; height: 20px; background-color: rgba(255,255,255,0.1); border: 1px solid
#8a2be2; display: flex; justify-content: center; align-items: center; font-size: 0.9rem; font-weight: bold;
cursor: pointer; user-select: none; color: white; }
.minesweeper-cell.revealed { background-color: rgba(0,0,0,0.4); border: 1px solid #4b0082; cursor: default; }
.minesweeper-cell.exploded { background-color: #ff1493; }
.minesweeper-cell[data-number="1"] { color: #7df9ff; }
.minesweeper-cell[data-number="2"] { color: #7fff00; }
.minesweeper-cell[data-number="3"] { color: #ff4444; }
.minesweeper-cell[data-number="4"] { color: #c0c0ff; }
.minesweeper-cell[data-number="5"] { color: #ff7f7f; }
.minesweeper-cell[data-number="6"] { color: #00ffff; }
.minesweeper-cell[data-number="7"] { color: #ffffff; }
.minesweeper-cell[data-number="8"] { color: #aaaaaa; }
```

```
/* Icons Inside Windows */
```

```
.window-icon {
display: inline-flex;
flex-direction: column;
```



```

align-items: center;
width: 80px;
padding: 10px;
margin: 5px;
text-align: center;
cursor: pointer;
user-select: none;
vertical-align: top;
border-radius: 3px;
background: transparent;
border: none;
color: inherit;
font-family: inherit;
}
.window-icon:hover { background-color: rgba(255, 0, 255, 0.3); }
.window-icon img { width: 32px; height: 32px; margin-bottom: 5px; }
.window-icon span { font-size: 0.7rem; color: white; max-height: 2.4em; overflow: hidden; line-height: 1.2em;
text-shadow: 1px 1px 2px #000; }

/* Image Viewer Styles */
#imageViewer .window-content { padding: 0; overflow: hidden; display: flex; justify-content: center; align-
items: center; background-color: #0d0221; border: none; margin: 0; }
#imageViewer img { max-width: 100%; max-height: 100%; object-fit: contain; }

/* Media Stream (Player) Styles */
#mediaStream .window-content { padding: 0; margin: 0; background-color: transparent; border: none; display:
flex; flex-direction: column; }
.media-player-url-bar { display: flex; padding: 5px; background-color: transparent; border-bottom: 1px solid
#4b0082; flex-shrink: 0; }
.media-player-input { flex-grow: 1; height: 22px; border: 1px solid #ff00ff; background-color: rgba(0,0,0,0.5);
color: white; padding: 2px 5px; font-family: 'Courier New', Courier, monospace; font-size: 0.8rem; margin-
right: 5px; }
.media-player-load-button { background-color: transparent; border: 1px solid #ff00ff; color: #ff00ff;
padding: 2px 8px; font-size: 0.8rem; height: 22px; cursor: pointer; }
.media-player-load-button:hover { background-color: rgba(255,0,255,0.2); }
.media-player-video-container { flex-grow: 1; background-color: black; display: flex; justify-content: center;
align-items: center; overflow: hidden; margin: 2px; border: 1px solid #4b0082; }
.media-player-status-message { padding: 20px; text-align: center; color: #ff00ff; font-size: 0.9rem; }
#youtube-player-mediaStream { width: 100%; height: 100%; }
#youtube-player-mediaStream iframe { display: block; width: 100%; height: 100%; border: none; }
.media-player-controls-panel { display: flex; justify-content: space-between; align-items: center; padding: 4px
8px; background-color: transparent; border-top: 1px solid #4b0082; flex-shrink: 0; height: 40px; }
.media-player-buttons-group { display: flex; align-items: center; }
.media-player-control-button { background-color: transparent; border: 1px solid #ff00ff; font-family: "Segoe
UI Symbol", "Symbola", system-ui; font-size: 0.9rem; color: #ff00ff; width: 30px; height: 26px; margin: 0
2px; cursor: pointer; display: flex; justify-content: center; align-items: center; padding: 0; line-height: 1; }
.media-player-control-button:hover { background-color: rgba(255,0,255,0.2); }
.media-player-control-button:disabled { color: #ff00ff80; cursor: default; opacity: 0.7; }
.media-player-progress-bar-container-placeholder { flex-grow: 1; height: 12px; border: 1px solid #4b0082;
background-color: rgba(0,0,0,0.5); margin: 0 10px; padding: 1px; }

```

```
.media-player-progress-bar-placeholder { width: 30%; height: 100%; background-color: #ff00ff; border-right: 1px solid #8a2be2; }
.media-player-volume-placeholder { display: flex; align-items: center; font-size: 0.9rem; color: #ff00ff; }
.media-player-volume-placeholder span { margin-right: 4px; }
.media-player-volume-slider-placeholder { width: 60px; height: 8px; background-color: #4b0082; border: 1px solid #ff00ff; }
```

```
/* Pattern Visualizer Specific Styles */
```

```
.pattern-io {
  background-color: #1a001a;
  color: #00ff00;
  border: 1px solid #ff00ff;
  font-family: 'Courier New', monospace;
  padding: 10px;
  box-shadow: inset 0 0 10px #ff00ff, 0 0 5px #ff00ff;
}
```

```
#entropicNexus .window-content,
#tabletCreator .window-content,
#subspaceAnomaly .window-content,
#aiCore .window-content,
#imageView .window-content {
  flex-direction: column;
}
```

```
#doom-content {
  display: flex;
  flex-grow: 1;
  background-color: black;
}
```

```
#sigilCanvas .window-content,
#mindField .window-content,
#mediaStream .window-content,
#webWeaver .window-content,
#doom-content,
#imageView .window-content {
  padding: 0;
  margin: 0;
  border: none;
}
```

```
/* --- Conspiracy Pinboard Styles --- */
```

```
#conspiracyPinboard .window-content {
  background: url('https://www.transparenttextures.com/patterns/black-felt.png'), radial-gradient(circle at center, #1a0b2e, #0d0221 80%);
}
```

```
.pinboard-grid {
  display: grid;
}
```

```
    grid-template-columns: 250px 1fr 250px;
    grid-template-rows: 1fr;
    gap: 10px;
    width: 100%;
    height: 100%;
    grid-template-areas:
      "controls board tools";
  }

.pinboard-panel {
  background-color: rgba(26, 10, 56, 0.6);
  border: 1px solid #ff00ff;
  padding: 10px;
  backdrop-filter: blur(5px);
  -webkit-backdrop-filter: blur(5px);
  box-shadow: 0 0 8px rgba(255, 0, 255, 0.3);
  overflow-y: auto;
  display: flex;
  flex-direction: column;
}

.pinboard-panel h3 {
  color: #ff1493;
  text-shadow: 0 0 5px #ff1493;
  margin-top: 0;
  margin-bottom: 10px;
  font-weight: bold;
  border-bottom: 1px solid #8a2be2;
  padding-bottom: 5px;
  font-size: 0.9rem;
  flex-shrink: 0;
}

.investigation-controls { grid-area: controls; }
.connection-suggestions { grid-area: controls; }
.interactive-board-container { grid-area: board; padding: 0 !important; position: relative; }
.interactive-board {
  background-image:
    linear-gradient(rgba(255,0,255,0.1) 1px, transparent 1px),
    linear-gradient(90deg, rgba(255,0,255,0.1) 1px, transparent 1px);
  background-size: 20px 20px;
  width: 100%;
  height: 100%;
  position: absolute;
}
.investigation-tools { grid-area: tools; }

.pinboard-panel button {
  width: 100%;
```

```
background-color: transparent;
border: 1px solid #00c1ff;
color: #00c1ff;
padding: 6px;
margin-bottom: 5px;
text-align: left;
cursor: pointer;
transition: all 0.2s;
font-size: 0.8rem;
}

.pinboard-panel button:hover {
background-color: rgba(0, 193, 255, 0.2);
color: white;
}

.pinboard-panel button.active {
border-color: #ff3333;
background-color: rgba(255, 51, 51, 0.3);
}

.pinboard-node {
position: absolute;
background-color: rgba(10, 20, 40, 0.9);
border: 1px solid #00c1ff;
color: #e0e0e0;
padding: 8px;
border-radius: 4px;
font-size: 0.75rem;
cursor: grab;
user-select: none;
width: 200px;
box-shadow: 0 2px 10px rgba(0,0,0,0.5);
transition: box-shadow 0.2s ease-in-out;
}

.pinboard-node:hover {
border-color: #ff00ff;
box-shadow: 0 0 10px rgba(255, 0, 255, 0.5);
}

.pinboard-node-header {
display: flex;
justify-content: space-between;
align-items: center;
}

.pinboard-node h4 {
margin: 0 0 5px 0;
color: #00c1ff;
font-size: 0.8rem;
```

```
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
    flex-grow: 1;
}

.pinboard-node p {
    margin: 0;
    line-height: 1.4;
}

.thread-line {
    stroke: #ff3333;
    stroke-width: 2;
    filter: drop-shadow(0 0 2px #ff3333);
}

.connection-suggestions-list, .pinned-tablets-list {
    list-style: none;
    padding: 0;
    margin: 0;
    flex-grow: 1;
    overflow-y: auto;
}

.suggestion-placeholder {
    color: #aaa;
    font-style: italic;
    font-size: 0.8rem;
}

.suggestion-item {
    padding: 4px 8px;
    margin-bottom: 4px;
    background: rgba(0,0,0,0.2);
    border: 1px solid #4b0082;
    border-radius: 3px;
    cursor: pointer;
    font-size: 0.8rem;
    transition: all 0.2s;
}

.suggestion-item:hover {
    background: rgba(255, 0, 255, 0.2);
    border-color: #ff00ff;
}

/* --- Bloom Simulator Styles --- */
#bloomSimulator .window-content {
    background-color: #0d0221;
    padding: 10px;
    display: flex;
    justify-content: center;
    align-items: center;
```

```

}

/* --- Zettelkasten Styles --- */
#hyenaDivaZettelkasten .window-content {
  color: #e0e0e0;
  font-size: 0.9rem;
  padding: 10px !important; /* Override default window content padding */
  background-color: #1a0b2e !important;
}
#hyenaDivaZettelkasten-content h1 {
  color: #ff1493; text-shadow: 0 0 5px #ff1493; margin-top: 0; margin-bottom: 10px; font-weight: bold;
  border-bottom: 1px solid #8a2be2; padding-bottom: 5px; font-size: 1.2rem;
}
.tabs-container {
  display: flex; border-bottom: 1px solid #8a2be2; margin-bottom: 10px; flex-wrap: wrap;
}
.win98-button {
  background-color: #c0c0c0; border: 1px solid; border-color: #fff #808080 #808080 #fff; box-shadow:
  1px 1px 0 #000; padding: 5px 10px; color: black; font-family: 'Arial', sans-serif;
  cursor: pointer; margin: 2px;
}
.win98-button.active, .win98-button:active {
  border-color: #808080 #fff #fff #808080;
  box-shadow: none;
  background: repeating-conic-gradient(#808080 0% 25%, #c0c0c0 0% 50%) 50% / 2px 2px;
}
.win98-button:hover { background-color: #dcdcdc; }
.tabs-container .win98-button { border-bottom: none; border-radius: 3px 3px 0 0; }
.tabs-container .win98-button.active { background-color: #e0e0e0; border-color: #fff #808080 #c0c0c0
#fff; }

.win98-input, .win98-textarea, .win98-select {
  background-color: white; border: 1px solid; border-color: #808080 #fff #fff #808080; padding: 3px;
  color: black; width: 100%; box-sizing: border-box;
}
.win98-textarea { height: 150px; resize: vertical; }

.zettel-container {
  flex-grow: 1; overflow-y: auto; background: rgba(0,0,0,0.2); border: 1px solid #4b0082; padding: 5px;
}
.zettel-card {
  background: rgba(255, 255, 255, 0.05); border: 1px solid #8a2be2; padding: 8px; margin-bottom: 5px;
  cursor: pointer;
  transition: background-color 0.2s; display: flex; align-items: center; justify-content: space-between;
}
.zettel-card:hover { background: rgba(255, 0, 255, 0.2); }
.zettel-title-container {
  display: flex;
  align-items: center;
  gap: 8px;

```

```
    flex-grow: 1;
    overflow: hidden;
}
.zettel-title { font-weight: bold; color: #ffc0cb; white-space: nowrap; overflow: hidden; text-overflow:
ellipsis;}
.zettel-id { font-size: 0.7rem; color: #aaa; flex-shrink: 0; }
.zettel-pin-icon { font-size: 0.8rem; cursor: pointer; opacity: 0.5; transition: opacity 0.2s; }
.zettel-pin-icon:hover { opacity: 1; }
.zettel-pin-icon.pinned { opacity: 1; filter: drop-shadow(0 0 3px #ffae00); }
.trunk-card { border-left: 3px solid #ff1493; }
.branch-card { border-left: 3px solid #00c1ff; margin-left: 15px; }
.leaf-card { border-left: 3px solid #7fff00; margin-left: 30px; }
.flower-card { border-left: 3px solid #ffae00; margin-left: 45px; }
.fruit-card { border-left: 3px solid #ff4444; margin-left: 60px; }
.bud-card { border-left: 3px solid #c0c0ff; margin-left: 75px; }

/* Dialog Box Styles */
.dialog-box {
    display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: rgba(0,0,0,0.5); z-
index: 101;
    justify-content: center; align-items: center; padding: 20px; box-sizing: border-box;
}
.dialog-content {
    background-color: #0d0221; border: 2px solid #ff00ff; box-shadow: 0 0 20px #ff00ff;
    width: 600px; max-width: 90%; display: flex; flex-direction: column;
}
.dialog-titlebar { display: flex; justify-content: space-between; align-items: center; background: linear-
gradient(to right, #ff00ff, #8a2be2); color: white; padding: 4px 8px; font-weight: bold; }
.dialog-close { background: none; border: none; color: white; font-size: 1.2rem; cursor: pointer; }
.dialog-inner-content { padding: 20px; max-height: 70vh; overflow-y: auto; }
.pin-evidence-list { max-height: 40vh; overflow-y: auto; }
.pin-evidence-item { padding: 8px; cursor: pointer; border: 1px solid #4b0082; margin-bottom: 5px; }
.pin-evidence-item:hover { background-color: rgba(255, 0, 255, 0.2); }

.win98-label { display: block; margin: 10px 0 5px 0; color: #ffc0cb; font-weight: bold; }

/* Settings App Styles */
.settings-content h3 { margin-top: 0; color: #ffc0cb; }
.settings-actions { margin-top: 20px; display: flex; gap: 10px; }

/* Ingestion Engine Styles */
.ingestion-content { display: flex; gap: 10px; padding: 10px; height: 100%; box-sizing: border-box; }
.ingestion-panel { flex: 1; display: flex; flex-direction: column; gap: 10px; overflow-y: auto; }
.ingestion-panel h3 { margin: 0 0 5px 0; color: #ff1493; border-bottom: 1px solid #4b0082; padding-bottom:
3px; }
.ingestion-panel h4 { margin: 0 0 5px 0; color: #ffc0cb; border-bottom: 1px dotted #4b0082; padding-bottom:
3px; font-weight: bold;}
#analysis-output h4 { margin-top: 10px; }
.ingestion-actions { display: flex; gap: 10px; margin-top: auto; flex-shrink: 0; padding-top: 10px; }
#ingestion-input-text { flex-grow: 1; }
```

```
#ingestion-results { flex-grow: 1; overflow-y: auto; padding-right: 5px; display: flex; flex-direction: column; }
.results-list { list-style: none; padding: 5px; margin: 0 0 10px 0; max-height: 120px; overflow-y: auto;
background: rgba(0,0,0,0.2); border: 1px solid #4b0082; }
.results-list li { padding: 3px 5px; border-bottom: 1px solid #4b0082; font-size: 0.8rem; }
.results-list li:last-child { border-bottom: none; }
.results-list .suggestion-item { cursor: pointer; transition: all 0.2s ease; }
.results-list .suggestion-item:hover { background-color: rgba(255, 0, 255, 0.2); border-color: #ff00ff; }
```

```
#ingestion-suggestion-form .win98-textarea { height: 100px; }
```

```
/* --- Custom Resizer Styles --- */
```

```
.resizer {
  position: absolute;
  background: transparent;
  z-index: 30;
}
.resizer-r {
  cursor: ew-resize;
  height: 100%;
  width: 6px;
  right: 0;
  top: 0;
}
.resizer-b {
  cursor: ns-resize;
  width: 100%;
  height: 6px;
  bottom: 0;
  left: 0;
}
.resizer-br {
  cursor: nwse-resize;
  width: 12px;
  height: 12px;
  right: 0;
  bottom: 0;
  z-index: 31; /* Above other handles */
}
```

```
/* --- Mobile / Responsive Design --- */
```

```
@media (max-width: 800px) {
  body {
    overflow: auto;
  }

  .top-nav {
    justify-content: flex-start;
    flex-wrap: wrap;
    padding: 5px;
  }
}
```



```
.desktop {
  padding: 60px 5px 50px 5px;
  height: auto;
  min-height: 100vh;
  width: 100%;
  box-sizing: border-box;
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  align-content: flex-start;
  overflow-y: auto;
  overflow-x: hidden;
}

.icon {
  width: 80px;
  margin: 10px;
}

.window {
  /* Force windows to be full-screen modals on mobile */
  position: fixed !important;
  top: 40px !important;
  left: 0 !important;
  width: 100% !important;
  height: calc(100vh - 80px) !important;
  max-width: 100vw;
  max-height: calc(100vh - 80px);
  border-radius: 0;
  border-left: none;
  border-right: none;
}

.window-titlebar {
  cursor: default;
}

/* Disable custom resizing on mobile */
.window.resizable {
  resize: none;
  overflow: hidden;
}

.resizer {
  display: none;
}

/* Make start menu full screen */
.start-menu {
  width: 100%;
}
```

```
height: calc(100% - 40px);
bottom: 40px;
left: 0;
border-radius: 0;
flex-direction: row;
flex-wrap: wrap;
justify-content: center;
align-items: flex-start;
align-content: flex-start;
padding: 20px;
box-sizing: border-box;
background: rgba(13, 2, 33, 0.95);
overflow-y: auto;
}

.start-menu-item {
  width: 45%;
  text-align: center;
  padding: 15px;
  margin: 5px;
  border: 1px solid #ff00ff;
}

#taskbar-apps {
  overflow-x: auto;
}

.dialog-content {
  width: 95vw;
  height: 90vh;
}

.ingestion-content {
  flex-direction: column;
}

.pinboard-grid {
  grid-template-columns: 1fr;
  grid-template-rows: auto 50vh auto;
  grid-template-areas:
    "controls"
    "board"
    "tools";
  height: auto;
}

.pinboard-panel {
  min-height: 150px;
}
}
```

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */
import { Chart, registerables } from 'https://cdn.jsdelivr.net/npm/chart.js';
Chart.register(...registerables);

//Gemini 95 was fully vibe-coded by @ammaar and @olacombe, while we don't endorse code quality, we thought
it was a fun demonstration of what's possible with the model when a Designer and PM jam.
//An homage to an OS that inspired so many of us!

// This version has been modified to remove all API-dependent features.

// Define the dosInstances object to fix type errors
const dosInstances = {};

// --- DOM Element References ---
const desktop = document.getElementById('desktop');
const windows = document.querySelectorAll('.window');
const icons = document.querySelectorAll('.icon'); // This is a NodeList
const startMenu = document.getElementById('start-menu');
const startButton = document.getElementById('start-button');
const taskbarAppsContainer = document.getElementById('taskbar-apps');

// --- State Variables ---
let activeWindow = null;
let highestZIndex = 20; // Start z-index for active windows
const openApps = new Map(); // Store open apps and their elements

// Store ResizeObservers to disconnect them later
const paintResizeObserverMap = new Map();
const bloomResizeObserverMap = new Map();
let bloomChart = null;

// --- Minesweeper Game State Variables ---
let minesweeperTimerInterval = null;
let minesweeperTimeElapsed = 0;
let minesweeperFlagsPlaced = 0;
let minesweeperGameOver = false;
let minesweeperMineCount = 10; // Default for 9x9
let minesweeperGridSize = { rows: 9, cols: 9 }; // Default 9x9
let minesweeperFirstClick = true; // To ensure first click is never a mine

// --- YouTube Player State ---
const youtubePlayers = {};
let ytApiLoaded = false;
let ytApiLoadingPromise = null;
```

```
const DEFAULT_YOUTUBE_VIDEO_ID = 'WXuK6gekU1Y'; // Default video for Media Stream ("Never Gonna Give You Up")
```

```
// --- Core Functions ---
```

```
/** Brings a window to the front and sets it as active */
```

```
function bringToFront(windowElement) {
  if (activeWindow === windowElement) return; // Already active

  if (activeWindow) {
    activeWindow.classList.remove('active');
    const appName = activeWindow.id;
    if (openApps.has(appName)) {
      openApps.get(appName)?.taskbarButton.classList.remove('active');
    }
  }
}
```

```
highestZIndex++;
windowElement.style.zIndex = highestZIndex.toString();
windowElement.classList.add('active');
activeWindow = windowElement;

const appNameRef = windowElement.id;
if (openApps.has(appNameRef)) {
  openApps.get(appNameRef)?.taskbarButton.classList.add('active');
}
if ((appNameRef === 'subspaceAnomaly') && dosInstances[appNameRef]) {
  const container = document.getElementById(`${appNameRef}-container`); // This ID might need
  checking
  const canvas = container?.querySelector('canvas');
  canvas?.focus();
}
}
```

```
/** Opens an application window */
```

```
async function openApp(appName) {
  const windowElement = document.getElementById(appName);
  if (!windowElement) {
    console.error(`Window element not found for app: ${appName}`);
    return;
  }
}
```

```
// If app is already initialized, just show and bring to front
if (openApps.has(appName)) {
  windowElement.style.display = 'flex';
  bringToFront(windowElement);
  // Special refresh for pinboard to catch new pins
  if (appName === 'conspiracyPinboard') {
    initConspiracyPinboard();
  }
}
```

```

    if (appName === 'bloomSimulator') {
      initBloomSimulator(windowElement);
    }
    return;
  }

  // If app is not initialized, set it up
  windowElement.style.display = 'flex';
  bringToFront(windowElement);

  const taskbarButton = document.createElement('button');
  taskbarButton.type = 'button';
  taskbarButton.classList.add('taskbar-app');
  taskbarButton.dataset.appName = appName;

  let iconSrc = '';
  let title = appName;
  const iconElement = findIconElement(appName);
  if (iconElement) {
    const img = iconElement.querySelector('img');
    const span = iconElement.querySelector('span');
    if (img) iconSrc = img.src;
    if (span) title = span.textContent || appName;
  } else { // Fallback for apps opened via start menu or nav bar
    switch(appName) {
      case 'entropicNexus': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/mycomputer.png';
        title = 'Entropic Nexus'; break;
      case 'webWeaver': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/chrome-icon-2.png';
        title = 'Web Weaver'; break;
      case 'tabletCreator': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/GemNotes.png'; title =
        'Tablet Creator'; break;
      case 'sigilCanvas': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/gempaint.png'; title =
        'Sigil Canvas'; break;
      case 'subspaceAnomaly': iconSrc = 'https://64.media.tumblr.com/
1d89dfa76381e5c14210a2149c83790d/7a15f84c681c1cf9-c1/
s540x810/86985984be99d5591e0cbc0dea6f05ffa3136dac.png'; title = 'Subspace Anomaly'; break;
      case 'aiCore': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/GeminiChatRetro.png'; title =
        'AI Core'; break;
      case 'mindField': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/gemsweeper.png'; title =
        'Mind Field'; break;
      case 'imageView': iconSrc = 'https://win98icons.alexmeub.com/icons/png/display_properties-4.png';
        title = 'Image Viewer'; break;
      case 'mediaStream': iconSrc = 'https://storage.googleapis.com/gemini-95-icons/ytmediaplayer.png';
        title = 'Media Stream'; break;
      case 'conspiracyPinboard': iconSrc = 'https://win98icons.alexmeub.com/icons/png/
address_book_user.png'; title = 'Conspiracy Pinboard'; break;
      case 'hyenaDivaZettelkasten': iconSrc = 'https://pfst.cf2.poecdn.net/base/image/
0ffd2bf15a7001f86e395096222147037538b25af55ef9b8c7ac8f6768b7397c?w=48&h=48'; title =
        'Zettelkasten'; break;
      case 'bloomSimulator': iconSrc = 'https://win98icons.alexmeub.com/icons/png/chart_pie_down-0.png';

```

```

title = 'Bloom Simulator'; break;
    // Nav Bar Apps
    case 'home': title = 'Home'; break;
    case 'about': title = 'About'; break;
    case 'patternVisualizer': title = 'Pattern Visualizer'; break;
    case 'hyenaDiva': title = 'Hyena Diva'; break;
    case 'luminalLanguage': title = 'Luminal Language'; break;
    case 'ackk': title = 'A.C.K.K.'; break;
  }
}

if (iconSrc) {
  const img = document.createElement('img');
  img.src = iconSrc;
  img.alt = title;
  taskbarButton.appendChild(img);
}
taskbarButton.appendChild(document.createTextNode(title));

taskbarButton.addEventListener('click', () => {
  if (windowElement === activeWindow && windowElement.style.display !== 'none') {
    minimizeApp(appName);
  } else {
    windowElement.style.display = 'flex';
    bringToFront(windowElement);
  }
});

taskbarAppsContainer.appendChild(taskbarButton);
openApps.set(appName, { windowEl: windowElement, taskbarButton: taskbarButton });
taskbarButton.classList.add('active');

// Initialize specific applications
if (appName === 'webWeaver') {
  initAiBrowser(windowElement);
}
else if (appName === 'sigilCanvas') {
  initSimplePaintApp(windowElement);
}
else if (appName === 'subspaceAnomaly' && !dosInstances['subspaceAnomaly']) {
  const doomContainer = document.getElementById('doom-content');
  if (doomContainer) {
    doomContainer.innerHTML = '<iframe src="https://js-dos.com/games/doom.exe.html" width="100%" height="100%" frameborder="0" scrolling="no" allowfullscreen></iframe>';
    dosInstances['subspaceAnomaly'] = { initialized: true };
  }
}
else if (appName === 'mindField') {
  initMinesweeperGame(windowElement);
}
else if (appName === 'entropicNexus') {

```

```

        initEntropicNexus(windowElement);
    }
    else if (appName === 'mediaStream') {
        await initMediaPlayer(windowElement);
    }
    else if (appName === 'conspiracyPinboard') {
        initConspiracyPinboard();
    }
    else if (appName === 'hyenaDivaZettelkasten' && !dosInstances['hyenaDivaZettelkasten']) {
        const container = document.querySelector('#hyenaDivaZettelkasten-content');
        initHyenaDivaApp(container);
        dosInstances['hyenaDivaZettelkasten'] = { initialized: true }; // Mark as initialized
    }
    else if (appName === 'bloomSimulator') {
        initBloomSimulator(windowElement);
    }
    else if (appName === 'patternVisualizer' && !dosInstances['patternVisualizer']) {
        const frame = windowElement.querySelector('iframe');
        if (frame) {
            frame.src = './pattern-visualizer.html';
        }
        dosInstances['patternVisualizer'] = { initialized: true };
    }
}

/** Closes an application window */
function closeApp(appName) {
    const appData = openApps.get(appName);
    if (!appData) return;

    const { windowEl, taskbarButton } = appData;

    windowEl.style.display = 'none';
    windowEl.classList.remove('active');
    taskbarButton.remove();
    openApps.delete(appName);

    if (dosInstances[appName]) {
        console.log(`Cleaning up ${appName} instance`);
        if (appName === 'hyenaDivaZettelkasten') {
            // Don't clear content for Zettelkasten, just mark as uninitialized
        } else if (appName === 'patternVisualizer') {
            const frame = windowEl.querySelector('iframe');
            if (frame) {
                frame.src = 'about:blank';
            }
        }
    }
    else {
        const container = document.getElementById(`${appName}-content`);
        if (container) container.innerHTML = '';
    }
}

```

```

    }
    delete dosInstances[appName];
  }

  if (appName === 'sigilCanvas') {
    const paintContent = appData.windowEl.querySelector('.window-content');
    if (paintContent && paintResizeObserverMap.has(paintContent)) {
      paintResizeObserverMap.get(paintContent)?.disconnect();
      paintResizeObserverMap.delete(paintContent);
    }
  }

  if (appName === 'bloomSimulator') {
    const bloomContent = appData.windowEl.querySelector('#bloom-chart-container');
    if (bloomContent && bloomResizeObserverMap.has(bloomContent)) {
      bloomResizeObserverMap.get(bloomContent)?.disconnect();
      bloomResizeObserverMap.delete(bloomContent);
    }
    if (bloomChart) {
      bloomChart.destroy();
      bloomChart = null;
    }
  }

  if (appName === 'mindField') {
    if (minesweeperTimerInterval) {
      clearInterval(minesweeperTimerInterval);
      minesweeperTimerInterval = null;
    }
  }

  if (appName === 'mediaStream') {
    const player = youtubePlayers[appName];
    if (player) {
      try {
        if (typeof player.stopVideo === 'function') player.stopVideo();
        if (typeof player.destroy === 'function') player.destroy();
      } catch (e) {
        console.warn("Error stopping/destroying media player:", e);
      }
      delete youtubePlayers[appName];
      console.log("Destroyed YouTube player for mediaStream.");
    }
    // Reset the player area with a message
    const playerDivId = `youtube-player-${appName}`;
    const playerDiv = document.getElementById(playerDivId);
    if (playerDiv) {
      playerDiv.innerHTML = `

Player closed. Enter a YouTube URL to load.</p>`;
    }
  }


```



```

    }
    const mediaPlayerWindow = document.getElementById('mediaStream');
    if (mediaPlayerWindow) {
      const playBtn = mediaPlayerWindow.querySelector('#media-player-play');
      const pauseBtn = mediaPlayerWindow.querySelector('#media-player-pause');
      const stopBtn = mediaPlayerWindow.querySelector('#media-player-stop');
      if (playBtn) playBtn.disabled = true;
      if (pauseBtn) pauseBtn.disabled = true;
      if (stopBtn) stopBtn.disabled = true;
    }
  }
}

if (activeWindow === windowEl) {
  activeWindow = null;
  let nextAppToActivate = null;
  let maxZ = -1;
  openApps.forEach((data) => {
    const z = parseInt(data.windowEl.style.zIndex || '0', 10);
    if (z > maxZ) {
      maxZ = z;
      nextAppToActivate = data.windowEl;
    }
  });
  if (nextAppToActivate) {
    bringToFront(nextAppToActivate);
  }
}
}
}

```

```

/** Minimizes an application window */
function minimizeApp(appName) {
  const appData = openApps.get(appName);
  if (!appData) return;

  const { windowEl, taskbarButton } = appData;

  windowEl.style.display = 'none';
  windowEl.classList.remove('active');
  taskbarButton.classList.remove('active');

  if (activeWindow === windowEl) {
    activeWindow = null;
    let nextAppToActivate = null;
    let maxZ = 0;
    openApps.forEach((data, name) => {
      if (data.windowEl.style.display !== 'none') {
        const z = parseInt(data.windowEl.style.zIndex || '0', 10);
        if (z > maxZ) {
          maxZ = z;

```

```

        nextAppToActivate = name;
      }
    }
  });
  if (nextAppToActivate) {
    bringToFront(openApps.get(nextAppToActivate).windowEl);
  }
}
}

// --- Conspiracy Pinboard Functions ---
// Note: This is now integrated with the Zettelkasten
function initConspiracyPinboard() {
  const pinboardWindow = document.getElementById('conspiracyPinboard');
  if (!pinboardWindow) return;
  const nodesContainer = pinboardWindow.querySelector('#pinboard-nodes-container');
  const svgLayer = pinboardWindow.querySelector('#pinboard-svg-layer');
  const drawConnectionBtn = pinboardWindow.querySelector('#draw-connection-btn');
  const addEvidenceBtn = pinboardWindow.querySelector('#add-evidence-btn');

  if (!nodesContainer || !svgLayer || !drawConnectionBtn || !addEvidenceBtn) return;

  let isConnecting = false;
  let firstNodeElement = null;
  let localNodes = [];
  let localCards = [];

  function loadData() {
    try {
      const storedCards = localStorage.getItem('multiZettelkasten');
      if (storedCards) {
        localCards = JSON.parse(storedCards).map(c => new Card(c));
      }
    } catch (e) { console.error("Could not load Zettelkasten data for pinboard:", e); }
  }

  function saveData() {
    try {
      localStorage.setItem('multiZettelkasten', JSON.stringify(localCards));
    } catch (e) { console.error("Could not save Zettelkasten data from pinboard:", e); }
  }

  function renderBoard() {
    loadData();
    nodesContainer.innerHTML = '';
    svgLayer.innerHTML = '';
    localNodes = [];

    const pinnedCards = localCards.filter(c => c.isPinned);

```

```

pinnedCards.forEach(data => {
  const nodeEl = document.createElement('div');
  nodeEl.className = 'pinboard-node';
  nodeEl.id = `pin-${data.id}`;
  nodeEl.dataset.cardId = data.id;
  nodeEl.style.left = `${data.pinX || 50}px`;
  nodeEl.style.top = `${data.pinY || 50}px`;
  nodeEl.innerHTML = `<h4>${data.title}</h4>`;

  nodeEl.addEventListener('click', (e) => {
    if (!isConnecting) {
      showZettelDialog(data.id);
    }
  });

  nodesContainer.appendChild(nodeEl);
  localNodes.push({ id: data.id, el: nodeEl });
  makeDraggable(nodeEl);
});
updateConnections();
}

function makeDraggable(element) {
  let offsetX, offsetY, isDragging = false;

  element.addEventListener('mousedown', (e) => {
    if (isConnecting) {
      handleConnectionClick(element);
      return;
    }
    isDragging = true;
    offsetX = e.clientX - element.offsetLeft;
    offsetY = e.clientY - element.offsetTop;
    element.style.cursor = 'grabbing';
    bringToFront(pinboardWindow);
  });

  document.addEventListener('mousemove', (e) => {
    if (!isDragging) return;
    element.style.left = `${e.clientX - offsetX}px`;
    element.style.top = `${e.clientY - offsetY}px`;
    updateConnections();
  });

  document.addEventListener('mouseup', () => {
    if (!isDragging) return;
    isDragging = false;
    element.style.cursor = 'grab';
    const cardId = element.dataset.cardId;
    const card = localCards.find(c => c.id === cardId);
  });
}

```

```

    if (card) {
      card.pinX = element.offsetLeft;
      card.pinY = element.offsetTop;
      saveData();
    }
  });
}

function handleClick(nodeEl) {
  if (!firstNodeElement) {
    firstNodeElement = nodeEl;
    nodeEl.style.boxShadow = '0 0 10px #ff3333';
  } else if (firstNodeElement !== nodeEl) {
    const fromId = firstNodeElement.dataset.cardId;
    const toId = nodeEl.dataset.cardId;

    if (fromId && toId) {
      const fromCard = localCards.find(c => c.id === fromId);
      if (fromCard && !fromCard.links.includes(toId)) {
        fromCard.links.push(toId);
      }
      saveData();
    }

    firstNodeElement.style.boxShadow = '';
    firstNodeElement = null;
    isConnecting = false;
    drawConnectionBtn.style.border = '';
    drawConnectionBtn.classList.remove('active');
    updateConnections();
  }
}

function updateConnections() {
  svgLayer.innerHTML = '';
  localCards.forEach(card => {
    if (card.links && card.links.length > 0) {
      const fromNode = localNodes.find(n => n.id === card.id);
      if (!fromNode || !fromNode.el) return;

      card.links.forEach(linkId => {
        const toCard = localCards.find(c => c.id === linkId);
        const toNode = localNodes.find(n => n.id === linkId);
        if (toNode && toNode.el && toCard?.isPinned) {
          const line = document.createElementNS('http://www.w3.org/2000/svg', 'line');
          line.setAttribute('x1', (fromNode.el.offsetLeft + fromNode.el.offsetWidth / 2).toString());
          line.setAttribute('y1', (fromNode.el.offsetTop + fromNode.el.offsetHeight / 2).toString());
          line.setAttribute('x2', (toNode.el.offsetLeft + toNode.el.offsetWidth / 2).toString());
          line.setAttribute('y2', (toNode.el.offsetTop + toNode.el.offsetHeight / 2).toString());
          line.setAttribute('class', 'thread-line');

```

```

        svgLayer.appendChild(line);
      }
    });
  }
});
}

drawConnectionBtn.addEventListener('click', () => {
  isConnected = !isConnecting;
  if (isConnecting) {
    drawConnectionBtn.classList.add('active');
  } else {
    drawConnectionBtn.classList.remove('active');
    if(firstNodeElement) firstNodeElement.style.boxShadow = '';
    firstNodeElement = null;
  }
});

addEvidenceBtn.addEventListener('click', () => {
  showZettelkastenNewCardDialog();
});

renderBoard();
}

/** Initializes the Browser functionality */
function initAiBrowser(windowElement) {
  const addressBar = windowElement.querySelector('.browser-address-bar');
  const goButton = windowElement.querySelector('.browser-go-button');
  const iframe = windowElement.querySelector('#browser-frame');
  const loadingEl = windowElement.querySelector('.browser-loading');
  const DIAL_UP_SOUND_URL = 'https://www.soundjay.com/communication/dial-up-modem-01.mp3';
  let dialUpAudio = null;

  if (!addressBar || !goButton || !iframe || !loadingEl) return;

  async function navigateToUrl(url) {
    if (!url.startsWith('http://') && !url.startsWith('https://')) {
      url = 'https://' + url;
    }

    try {
      const domain = new URL(url).hostname;
      loadingEl.innerHTML = `

Connecting to ${domain}...</p>`;
    } catch (e) {
      loadingEl.innerHTML = `

Connecting...</p>`;
    }

    loadingEl.style.display = 'flex';
  }


```

```

try {
  if (!dialUpAudio) {
    dialUpAudio = new Audio(DIAL_UP_SOUND_URL);
    dialUpAudio.loop = true;
  }
  await dialUpAudio.play();
} catch (audioError) {
  console.error("Dial-up sound error:", audioError);
}

// Simulate loading time to enjoy the sound
setTimeout(() => {
  iframe.src = url;
  addressBar.value = url;

  iframe.onload = () => {
    loadingEl.style.display = 'none';
    if (dialUpAudio) {
      dialUpAudio.pause();
      dialUpAudio.currentTime = 0;
    }
  };

  iframe.onerror = () => {
    loadingEl.style.display = 'none';
    if (dialUpAudio) {
      dialUpAudio.pause();
      dialUpAudio.currentTime = 0;
    }
    alert(` Could not load page. Many websites block being displayed in an iframe for security reasons.` );
  };
}, 4000); // A nice long dial-up
}

goButton.addEventListener('click', () => navigateToUrl(addressBar.value));
addressBar.addEventListener('keydown', (e) => { if (e.key === 'Enter') navigateToUrl(addressBar.value); });
addressBar.addEventListener('click', () => addressBar.select());
}
// --- Event Listeners Setup ---

icons.forEach(icon => {
  icon.addEventListener('click', () => {
    const appName = icon.getAttribute('data-app');
    if (appName) {
      openApp(appName);
      if (startMenu) startMenu.classList.remove('active');
    }
  });
});

```

```

});

document.querySelectorAll('.start-menu-item').forEach(item => {
  item.addEventListener('click', () => {
    const appName = item.getAttribute('data-app');
    if (appName) openApp(appName);
    if (startMenu) startMenu.classList.remove('active');
  });
});

// Listener for new nav bar
document.querySelectorAll('.nav-item').forEach(item => {
  item.addEventListener('click', (e) => {
    e.preventDefault();
    const appName = item.getAttribute('data-app');
    if (appName) {
      openApp(appName);
    }
  });
});

if (startButton) {
  startButton.addEventListener('click', (e) => {
    e.stopPropagation();
    if (startMenu) {
      startMenu.classList.toggle('active');
      if (startMenu.classList.contains('active')) {
        highestZIndex++;
        startMenu.style.zIndex = highestZIndex.toString();
      }
    }
  });
}

}

windows.forEach(windowElement => {
  const titleBar = windowElement.querySelector('.window-titlebar');
  const closeButton = windowElement.querySelector('.window-close');
  const minimizeButton = windowElement.querySelector('.window-minimize');

  windowElement.addEventListener('mousedown', () => bringToFront(windowElement), true);

  if (closeButton) {
    closeButton.addEventListener('click', (e) => { e.stopPropagation(); closeApp(windowElement.id); });
  }
  if (minimizeButton) {
    minimizeButton.addEventListener('click', (e) => { e.stopPropagation(); minimizeApp(windowElement.id); });
  }

  if (titleBar) {

```

```

let isDragging = false;
let dragOffsetX, dragOffsetY;
const startDragging = (e) => {
  if (!(e.target === titleBar || titleBar.contains(e.target)) || e.target.closest('.window-control-button')) {
    isDragging = false; return;
  }
  isDragging = true; bringToFront(windowElement);
  const rect = windowElement.getBoundingClientRect();
  dragOffsetX = e.clientX - rect.left; dragOffsetY = e.clientY - rect.top;
  titleBar.style.cursor = 'grabbing';
  document.addEventListener('mousemove', dragWindow);
  document.addEventListener('mouseup', stopDragging, { once: true });
};
const dragWindow = (e) => {
  if (!isDragging) return;
  let x = e.clientX - dragOffsetX; let y = e.clientY - dragOffsetY;
  const topNavHeight = 40; // Approx height of new nav bar
  y = Math.max(topNavHeight, y); // Prevent dragging under nav bar

  windowElement.style.left = `${x}px`; windowElement.style.top = `${y}px`;
};
const stopDragging = () => {
  if (!isDragging) return;
  isDragging = false; titleBar.style.cursor = 'grab';
  document.removeEventListener('mousemove', dragWindow);
};
titleBar.addEventListener('mousedown', startDragging);
}

if (!openApps.has(windowElement.id)) { // Only apply random for newly opened, not for bringToFront
  const randomTop = Math.random() * (window.innerHeight / 4) + 50;
  const randomLeft = Math.random() * (window.innerWidth / 3) + 20;
  windowElement.style.top = `${randomTop}px`;
  windowElement.style.left = `${randomLeft}px`;
}
});

document.addEventListener('click', (e) => {
  if (startMenu && startMenu.classList.contains('active') && e.target instanceof Node && !
startMenu.contains(e.target) && startButton && !startButton.contains(e.target)) {
    startMenu.classList.remove('active');
  }
});

function findIconElement(appName) {
  return Array.from(icons).find(icon => icon.dataset.app === appName);
}

console.log("Entropic Nexus Initialized (JS)");

```



```

function initSimplePaintApp(windowElement) {
  const canvas = windowElement.querySelector('#paint-canvas');
  const toolbar = windowElement.querySelector('.paint-toolbar');
  const contentArea = windowElement.querySelector('.window-content');
  const colorSwatches = windowElement.querySelectorAll('.paint-color-swatch');
  const sizeButtons = windowElement.querySelectorAll('.paint-size-button');
  const clearButton = windowElement.querySelector('.paint-clear-button');

  if (!canvas || !toolbar || !contentArea || !clearButton) { return; }
  const ctx = canvas.getContext('2d');
  if (!ctx) { return; }

  let isDrawing = false; let lastX = 0; let lastY = 0;
  ctx.strokeStyle = 'black'; ctx.lineWidth = 2; ctx.lineJoin = 'round'; ctx.lineCap = 'round';
  let currentStrokeStyle = ctx.strokeStyle; let currentLineWidth = ctx.lineWidth;

  function resizeCanvas() {
    requestAnimationFrame(() => {
      const rect = contentArea.getBoundingClientRect();
      const toolbarHeight = toolbar.offsetHeight;
      const newWidth = Math.floor(rect.width);
      const newHeight = Math.floor(rect.height - toolbarHeight);

      if (canvas.width === newWidth && canvas.height === newHeight && newWidth > 0 && newHeight > 0)
        return;

      canvas.width = newWidth > 0 ? newWidth : 1;
      canvas.height = newHeight > 0 ? newHeight : 1;

      ctx.fillStyle = 'white'; ctx.fillRect(0, 0, canvas.width, canvas.height);
      ctx.strokeStyle = currentStrokeStyle; ctx.lineWidth = currentLineWidth;
      ctx.lineJoin = 'round'; ctx.lineCap = 'round';
    });
  }

  const resizeObserver = new ResizeObserver(resizeCanvas);
  resizeObserver.observe(contentArea);
  paintResizeObserverMap.set(contentArea, resizeObserver);
  resizeCanvas();

  function getMousePos(canvasDom, event) {
    const rect = canvasDom.getBoundingClientRect();
    let clientX, clientY;
    if (event instanceof MouseEvent) { clientX = event.clientX; clientY = event.clientY; }
    else { clientX = event.touches[0].clientX; clientY = event.touches[0].clientY; }
    return { x: clientX - rect.left, y: clientY - rect.top };
  }

  function startDrawing(e) {
    isDrawing = true; const pos = getMousePos(canvas, e);
    [lastX, lastY] = [pos.x, pos.y]; ctx.beginPath(); ctx.moveTo(lastX, lastY);
  }

```

```

}
function draw(e) {
  if (!isDrawing) return; e.preventDefault();
  const pos = getMousePos(canvas, e);
  ctx.lineTo(pos.x, pos.y);
  ctx.stroke();
  [lastX, lastY] = [pos.x, pos.y];
}
function stopDrawing() { if (isDrawing) isDrawing = false; }

canvas.addEventListener('mousedown', startDrawing);
canvas.addEventListener('mousemove', draw);
canvas.addEventListener('mouseup', stopDrawing);
canvas.addEventListener('mouseleave', stopDrawing);
canvas.addEventListener('touchstart', startDrawing, { passive: false });
canvas.addEventListener('touchmove', draw, { passive: false });
canvas.addEventListener('touchend', stopDrawing);
canvas.addEventListener('touchcancel', stopDrawing);

colorSwatches.forEach(swatch => {
  swatch.addEventListener('click', () => {
    ctx.strokeStyle = swatch.dataset.color || 'black'; currentStrokeStyle = ctx.strokeStyle;
    colorSwatches.forEach(s => s.classList.remove('active')); swatch.classList.add('active');
    if (swatch.dataset.color === 'white') {
      const largeSizeButton = Array.from(sizeButtons).find(b => b.dataset.size === '10');
      if (largeSizeButton) {
        ctx.lineWidth = parseInt(largeSizeButton.dataset.size || '10', 10); currentLineWidth =
ctx.lineWidth;
        sizeButtons.forEach(s => s.classList.remove('active')); largeSizeButton.classList.add('active');
      }
    } else {
      const activeSizeButton = Array.from(sizeButtons).find(b => b.classList.contains('active'));
      if (activeSizeButton) { ctx.lineWidth = parseInt(activeSizeButton.dataset.size || '2', 10);
currentLineWidth = ctx.lineWidth; }
    }
  });
});

sizeButtons.forEach(button => {
  button.addEventListener('click', () => {
    ctx.lineWidth = parseInt(button.dataset.size || '2', 10); currentLineWidth = ctx.lineWidth;
    sizeButtons.forEach(s => s.classList.remove('active')); button.classList.add('active');
    const eraser = Array.from(colorSwatches).find(s => s.dataset.color === 'white');
    if (!eraser?.classList.contains('active')) {
      if (!Array.from(colorSwatches).some(s => s.classList.contains('active'))) {
        const blackSwatch = Array.from(colorSwatches).find(s => s.dataset.color === 'black');
        if (blackSwatch) {
          blackSwatch.classList.add('active');
          ctx.strokeStyle = 'black';
          currentStrokeStyle = ctx.strokeStyle;
        }
      }
    }
  });
});

```

```

    }
  }
});
clearButton.addEventListener('click', () => {
  ctx.fillStyle = 'white'; ctx.fillRect(0, 0, canvas.width, canvas.height);
});
(windowElement.querySelector('.paint-color-swatch[data-color="black"]'))?.classList.add('active');
(windowElement.querySelector('.paint-size-button[data-size="2"]'))?.classList.add('active');
}

function initMinesweeperGame(windowElement) {
  const boardElement = windowElement.querySelector('#minesweeper-board');
  const flagCountElement = windowElement.querySelector('.minesweeper-flag-count');
  const timerElement = windowElement.querySelector('.minesweeper-timer');
  const resetButton = windowElement.querySelector('.minesweeper-reset-button');
  if (!boardElement || !flagCountElement || !timerElement || !resetButton) return;
  let grid = [];

  function resetGame() {
    if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
    minesweeperTimerInterval = null;
    minesweeperTimeElapsed = 0;
    minesweeperFlagsPlaced = 0;
    minesweeperGameOver = false;
    minesweeperFirstClick = true;
    minesweeperMineCount = 10;
    minesweeperGridSize = { rows: 9, cols: 9 };
    timerElement.innerHTML = `⌚ 00:00`;
    flagCountElement.innerHTML = `🚩 ${minesweeperMineCount}`;
    resetButton.innerHTML = `🔄`;
    createGrid();
  }

  function createGrid() {
    boardElement.innerHTML = '';
    grid = [];
    boardElement.style.gridTemplateColumns = `repeat(${minesweeperGridSize.cols}, 20px)`;
    boardElement.style.gridTemplateRows = `repeat(${minesweeperGridSize.rows}, 20px)`;
    for (let r = 0; r < minesweeperGridSize.rows; r++) {
      const row = [];
      for (let c = 0; c < minesweeperGridSize.cols; c++) {
        const cellElement = document.createElement('div');
        cellElement.classList.add('minesweeper-cell');
        const cellData = { isMine: false, isRevealed: false, isFlagged: false, adjacentMines: 0, element:
cellElement, row: r, col: c };
        cellElement.addEventListener('click', () => handleCellClick(cellData));
        cellElement.addEventListener('contextmenu', (e) => { e.preventDefault();
handleCellRightClick(cellData); });
        row.push(cellData);
      }
    }
  }
}

```

```

        boardElement.appendChild(cellElement);
    }
    grid.push(row);
}
}

function placeMines(firstClickRow, firstClickCol) {
    let minesPlaced = 0;
    while (minesPlaced < minesweeperMineCount) {
        const r = Math.floor(Math.random() * minesweeperGridSize.rows);
        const c = Math.floor(Math.random() * minesweeperGridSize.cols);
        if ((r === firstClickRow && c === firstClickCol) || grid[r][c].isMine) continue;
        grid[r][c].isMine = true;
        minesPlaced++;
    }
    for (let r = 0; r < minesweeperGridSize.rows; r++) {
        for (let c = 0; c < minesweeperGridSize.cols; c++) {
            if (!grid[r][c].isMine) grid[r][c].adjacentMines = countAdjacentMines(r, c);
        }
    }
}

function countAdjacentMines(row, col) {
    let count = 0;
    for (let dr = -1; dr <= 1; dr++) {
        for (let dc = -1; dc <= 1; dc++) {
            if (dr === 0 && dc === 0) continue;
            const nr = row + dr;
            const nc = col + dc;
            if (nr >= 0 && nr < minesweeperGridSize.rows && nc >= 0 && nc < minesweeperGridSize.cols &&
grid[nr][nc].isMine) count++;
        }
    }
    return count;
}

function handleCellClick(cell) {
    if (minesweeperGameOver || cell.isRevealed || cell.isFlagged) return;
    if (minesweeperFirstClick && !minesweeperTimerInterval) {
        placeMines(cell.row, cell.col);
        minesweeperFirstClick = false;
        startTimer();
    }
    if (cell.isMine) {
        gameOver(cell);
    } else {
        revealCell(cell);
        checkWinCondition();
    }
}
}

```

```

function handleCellRightClick(cell) {
  if (minesweeperGameOver || cell.isRevealed || (minesweeperFirstClick && !minesweeperTimerInterval))
return;
  cell.isFlagged = !cell.isFlagged;
  cell.element.innerHTML = cell.isFlagged ? '🚩' : '';
  minesweeperFlagsPlaced += cell.isFlagged ? 1 : -1;
  updateFlagCount();
  checkWinCondition();
}

function revealCell(cell) {
  if (cell.isRevealed || cell.isFlagged || cell.isMine) return;
  cell.isRevealed = true;
  cell.element.classList.add('revealed');
  cell.element.innerHTML = '';
  if (cell.adjacentMines > 0) {
    cell.element.textContent = cell.adjacentMines.toString();
    cell.element.dataset.number = cell.adjacentMines.toString();
  } else {
    for (let dr = -1; dr <= 1; dr++) {
      for (let dc = -1; dc <= 1; dc++) {
        if (dr === 0 && dc === 0) continue;
        const nr = cell.row + dr;
        const nc = cell.col + dc;
        if (nr >= 0 && nr < minesweeperGridSize.rows && nc >= 0 && nc < minesweeperGridSize.cols) {
          const neighbor = grid[nr][nc];
          if (!neighbor.isRevealed && !neighbor.isFlagged) revealCell(neighbor);
        }
      }
    }
  }
}

function startTimer() {
  if (minesweeperTimerInterval) return;
  minesweeperTimeElapsed = 0;
  timerElement.innerHTML = `⌚ 0`;
  minesweeperTimerInterval = window.setInterval(() => {
    minesweeperTimeElapsed++;
    timerElement.innerHTML = `⌚ ${minesweeperTimeElapsed}`;
  }, 1000);
}

function updateFlagCount() {
  flagCountElement.innerHTML = `🚩 ${minesweeperMineCount - minesweeperFlagsPlaced}`;
}

function gameOver(clickedMine) {
  minesweeperGameOver = true;

```

```

    if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
    resetButton.innerHTML = '&#128565;';
    grid.forEach(row => {
      row.forEach(cell => {
        if (cell.isMine) {
          cell.element.innerHTML = '&#128163;';
          cell.element.classList.remove('revealed');
          if (cell !== clickedMine) cell.element.style.backgroundColor = '#555';
        }
        if (cell.isFlagged && !cell.isMine) cell.element.innerHTML = '&#10060;';
      });
    });
    clickedMine.element.classList.add('exploded');
    clickedMine.element.innerHTML = '&#128165;';
  }

  function checkWinCondition() {
    if (minesweeperGameOver) return;
    let revealedCount = 0;
    grid.forEach(row => row.forEach(cell => { if (cell.isRevealed) revealedCount++; }));
    const nonMineCells = (minesweeperGridSize.rows * minesweeperGridSize.cols) - minesweeperMineCount;
    if (revealedCount === nonMineCells) {
      minesweeperGameOver = true;
      if (minesweeperTimerInterval) clearInterval(minesweeperTimerInterval);
      resetButton.innerHTML = '&#128526;';
      grid.forEach(row => row.forEach(cell => { if (cell.isMine && !cell.isFlagged) { cell.isFlagged = true;
cell.element.innerHTML = '&#128681;'; } }));
      updateFlagCount();
    }
  }

  resetButton.addEventListener('click', resetGame);
  resetGame();
}

function initEntropicNexus(windowElement) {
  const cDriveIcon = windowElement.querySelector('#c-drive-icon');
  const cDriveContent = windowElement.querySelector('#c-drive-content');
  const secretImageIcon = windowElement.querySelector('#secret-image-icon');
  if (!cDriveIcon || !cDriveContent || !secretImageIcon) return;

  cDriveIcon.addEventListener('click', () => {
    cDriveContent.style.display = 'block';
    cDriveIcon.style.display = 'none';
  });

  secretImageIcon.addEventListener('click', async () => {
    await openApp('imageView');
    const imageView = document.getElementById('imageView');
    if (imageView) {

```

```

    const img = imageView.querySelector('#image-viewer-img');
    const title = imageView.querySelector('#image-viewer-title');
    if (img) img.src = "https://storage.googleapis.com/gemini-95-secret-files/itsasecret.jpg";
    if (title) title.textContent = "dontshowthistoanyone.jpg";
  }
});
}

function extractYouTubeID(url) {
  if (!url) return null;
  const regExp = /^.*(youtu.be\/|v\/|u\/\w\/|embed\/|watch?v=|&v=)([^\#&?]*).*$/;
  const match = url.match(regExp);
  return (match && match[2].length === 11) ? match[2] : url; // Assume it's an ID if it's not a valid URL
}

function loadYouTubeAPI() {
  if (ytApiLoaded) return Promise.resolve();
  if (ytApiLoadingPromise) return ytApiLoadingPromise;

  ytApiLoadingPromise = new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = "https://www.youtube.com/iframe_api";
    script.onload = () => {
      console.log("YouTube IFrame API loaded.");
      ytApiLoaded = true;
      // The onYouTubeIframeAPIReady function is called by the script
      // so we resolve when that is defined and called.
      window.onYouTubeIframeAPIReady = () => {
        console.log("YouTube API Ready.");
        resolve();
      };
    };
    script.onerror = () => {
      console.error("Failed to load YouTube API.");
      reject(new Error("Failed to load YouTube API."));
    };
    document.head.appendChild(script);
  });
  return ytApiLoadingPromise;
}

async function initMediaPlayer(windowElement) {
  const appName = windowElement.id;
  const playerDivId = `youtube-player-${appName}`;
  const playerDiv = document.getElementById(playerDivId);
  const loadButton = windowElement.querySelector('.media-player-load-button');
  const urlInput = windowElement.querySelector('.media-player-input');
  const playBtn = windowElement.querySelector('#media-player-play');
  const pauseBtn = windowElement.querySelector('#media-player-pause');

```

```

const stopBtn = windowElement.querySelector('#media-player-stop');

if (!playerDiv || !loadButton || !urlInput || !playBtn || !pauseBtn || !stopBtn) return;

playBtn.disabled = true; pauseBtn.disabled = true; stopBtn.disabled = true;

try {
  await loadYouTubeAPI();
  playerDiv.innerHTML = '';
  createPlayer(appName, DEFAULT_YOUTUBE_VIDEO_ID);
} catch (error) {
  console.error(error);
  playerDiv.innerHTML = `

Error loading YouTube player.</p>`;
}

function createPlayer(appName, videoId) {
  if (youtubePlayers[appName]) {
    youtubePlayers[appName].destroy();
  }
  const player = new window.YT.Player(`youtube-player-${appName}`, {
    height: '100%',
    width: '100%',
    videoId: videoId,
    playerVars: { 'autoplay': 1, 'controls': 0, 'modestbranding': 1 },
    events: {
      'onReady': onPlayerReady,
      'onError': onPlayerError
    }
  });
  youtubePlayers[appName] = player;
}

function onPlayerReady(event) {
  console.log("Player is ready:", event.target);
  playBtn.disabled = false; pauseBtn.disabled = false; stopBtn.disabled = false;
  event.target.playVideo();
}

function onPlayerError(event) {
  console.error("YouTube Player Error:", event.data);
  const playerContainer = document.getElementById(`youtube-player-${appName}`);
  if (playerContainer) {
    playerContainer.innerHTML = `

Error playing video. Check URL
or Video ID.</p>`;
  }
}

loadButton.addEventListener('click', () => {
  const videoId = extractYouTubeID(urlInput.value);
  if (videoId) {


```



```

        createPlayer(appName, videoId);
    } else {
        alert("Invalid YouTube URL or ID");
    }
});

playBtn.onclick = () => youtubePlayers[appName]?.playVideo();
pauseBtn.onclick = () => youtubePlayers[appName]?.pauseVideo();
stopBtn.onclick = () => youtubePlayers[appName]?.stopVideo();
}

// --- Zettelkasten App ---

const HIERARCHY_TYPES = ["Trunk", "Branch", "Leaf", "Flower", "Fruit", "Bud"];

class Card {
    constructor({ id, parentId = null, title, content, type, timestamp = Date.now(), tags = [], links = [], isPinned = false, pinX = 0, pinY = 0 }) {
        this.id = id;
        this.parentId = parentId;
        this.title = title;
        this.content = content;
        this.type = type;
        this.timestamp = timestamp;
        this.tags = tags;
        this.links = links;
        this.isPinned = isPinned;
        this.pinX = pinX;
        this.pinY = pinY;
    }
}

function initHyenaDivaApp(container) {
    let cards = [];
    let currentTab = 'All';

    function saveCards() {
        localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
    }

    function loadCards() {
        const storedCards = localStorage.getItem('multiZettelkasten');
        if (storedCards) {
            try {
                const parsedCards = JSON.parse(storedCards);
                cards = parsedCards.map(c => new Card(c));
            } catch (e) {
                console.error("Failed to parse Zettelkasten data, starting fresh.", e);
                cards = loadZettelkastenData();
                saveCards();
            }
        }
    }
}

```

```

    }
  } else {
    cards = loadZettelkastenData();
    saveCards();
  }
}

function render() {
  container.innerHTML = `
    <h1>Knowledge Vault</h1>
    <div class="tabs-container">
      <button class="win98-button tab-button" data-tab="All">All</button>
      ${HIERARCHY_TYPES.map(type => `<button class="win98-button tab-button" data-tab="${type}">${
{type}s</button>`).join('')}
      <button class="win98-button" id="new-card-button">+ New Tablet</button>
    </div>
    <div class="zettel-container" id="zettel-container"></div>
  `;

  document.querySelector('#new-card-button').addEventListener('click', () => {
    showZettelkastenNewCardDialog();
  });

  document.querySelectorAll('.tab-button').forEach(button => {
    button.addEventListener('click', (e) => {
      currentTab = e.target.dataset.tab;
      renderCardList();
      document.querySelectorAll('.tab-button').forEach(btn => btn.classList.remove('active'));
      e.target.classList.add('active');
    });
  });

  document.querySelector(`.tab-button[data-tab="${currentTab}"]`).classList.add('active');
  renderCardList();
}

function renderCardList() {
  const zettelContainer = document.getElementById('zettel-container');
  zettelContainer.innerHTML = '';

  let filteredCards = (currentTab === 'All') ? cards : cards.filter(card => card.type === currentTab);

  sortCardsByHierarchy(filteredCards);

  filteredCards.forEach(card => {
    const cardElement = document.createElement('div');
    cardElement.className = `zettel-card ${card.type.toLowerCase()}-card`;
    cardElement.dataset.id = card.id;
    cardElement.innerHTML = `
      <div class="zettel-title">${card.title}</div>

```

```

        <div class="zettel-id">${card.id}</div>
      `;
      cardElement.addEventListener('click', () => showZettelDialog(card.id));
      zettelContainer.appendChild(cardElement);
    });
  }

  window.addEventListener('zettelkastenUpdated', () => {
    loadCards();
    renderCardList();
    // Also refresh pinboard if it's open
    if (openApps.has('conspiracyPinboard')) {
      initConspiracyPinboard();
    }
    if (openApps.has('bloomSimulator')) {
      initBloomSimulator(document.getElementById('bloomSimulator'));
    }
  });

  loadCards();
  render();
}

function showZettelDialog(cardId) {
  let cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]').map(c => new Card(c));
  const card = cards.find(c => c.id === cardId);
  if (!card) return;

  const dialog = document.getElementById('zettelkasten-dialog');
  const dialogContent = document.getElementById('dialog-inner-content');
  const dialogTitle = document.getElementById('dialog-title');

  dialogTitle.textContent = `View/Edit: ${card.title}`;
  dialogContent.innerHTML = `
    <label class="win98-label">Title:</label>
    <input type="text" id="edit-title" class="win98-input" value="${card.title}">
    <label class="win98-label">Content:</label>
    <textarea id="edit-content" class="win98-textarea">${card.content}</textarea>
    <label class="win98-label">Type:</label>
    <select id="edit-type" class="win98-select">
      ${HIERARCHY_TYPES.map(type => `<option value="${type}" ${card.type === type ? 'selected' : ''}>${
        {type}</option>`).join('')}
    </select>
    <label class="win98-label">Tags (comma separated):</label>
    <input type="text" id="edit-tags" class="win98-input" value="${card.tags.join(', ')}">
    <label class="win98-label">
      <input type="checkbox" id="edit-pinned" ${card.isPinned ? 'checked' : ''}> Pin to Conspiracy Board
    </label>
    <div style="margin-top: 20px;">
      <button id="save-card-button" class="win98-button">Save Changes</button>
  `;

```

```

        <button id="delete-card-button" class="win98-button">Delete Tablet</button>
    </div>
`;

dialog.style.display = 'flex';

document.getElementById('save-card-button').addEventListener('click', () => {
    card.title = document.getElementById('edit-title').value;
    card.content = document.getElementById('edit-content').value;
    card.type = document.getElementById('edit-type').value;
    card.tags = document.getElementById('edit-tags').value.split(',').map(t => t.trim()).filter(Boolean);
    card.isPinned = document.getElementById('edit-pinned').checked;

    localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
    window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
    dialog.style.display = 'none';
});

document.getElementById('delete-card-button').addEventListener('click', () => {
    if(confirm(`Are you sure you want to delete the tablet "${card.title}"? This cannot be undone.`)) {
        const index = cards.findIndex(c => c.id === cardId);
        if(index > -1) cards.splice(index, 1);
        localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
        window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
        dialog.style.display = 'none';
    }
});
}

```

```

function showZettelkastenNewCardDialog() {
    let cards = JSON.parse(localStorage.getItem('multiZettelkasten') || '[]').map(c => new Card(c));
    const dialog = document.getElementById('zettelkasten-dialog');
    const dialogContent = document.getElementById('dialog-inner-content');
    const dialogTitle = document.getElementById('dialog-title');

    dialogTitle.textContent = "Create New Tablet";
    dialogContent.innerHTML = `
        <label class="win98-label">Title:</label>
        <input type="text" id="new-title" class="win98-input" placeholder="Enter title...">
        <label class="win98-label">Content:</label>
        <textarea id="new-content" class="win98-textarea" placeholder="Enter content..."></textarea>
        <label class="win98-label">Type:</label>
        <select id="new-type" class="win98-select">
            ${HIERARCHY_TYPES.map(type => `<option value="${type}">${type}</option>`).join('')}
        </select>
        <label class="win98-label">Parent Tablet (Optional):</label>
        <select id="new-parent" class="win98-select">
            <option value="">None (new Trunk)</option>
            ${cards.map(c => `<option value="${c.id}">${c.id} - ${c.title}</option>`).join('')}
    `;
}

```

```

</select>
<label class="win98-label">Tags (comma separated):</label>
<input type="text" id="new-tags" class="win98-input" placeholder="e.g. consciousness, AI, hyena">
<div style="margin-top: 20px;">
  <button id="create-card-button" class="win98-button">Create Tablet</button>
</div>
`;

dialog.style.display = 'flex';

document.getElementById('create-card-button').addEventListener('click', () => {
  const parentId = document.getElementById('new-parent').value;
  const parent = cards.find(c => c.id === parentId);
  let newId;

  if (parent) {
    const childCards = cards.filter(c => c.parentId === parentId);
    newId = `${parentId}.${childCards.length + 1}`;
  } else {
    const trunkCards = cards.filter(c => !c.parentId);
    newId = `${trunkCards.length + 1}`;
  }

  const newCard = new Card({
    id: newId,
    parentId: parentId || null,
    title: document.getElementById('new-title').value || "Untitled",
    content: document.getElementById('new-content').value,
    type: document.getElementById('new-type').value,
    tags: document.getElementById('new-tags').value.split(',').map(t => t.trim()).filter(Boolean),
  });

  cards.push(newCard);
  localStorage.setItem('multiZettelkasten', JSON.stringify(cards));
  window.dispatchEvent(new CustomEvent('zettelkastenUpdated'));
  dialog.style.display = 'none';
});
}

// Dialog close button functionality
document.getElementById('dialog-close-button')?.addEventListener('click', () => {
  document.getElementById('zettelkasten-dialog').style.display = 'none';
});

function sortCardsByHierarchy(cards) {
  cards.sort((a, b) => {
    const aParts = a.id.split('.').map(Number);
    const bParts = b.id.split('.').map(Number);
    const len = Math.max(aParts.length, bParts.length);
    for (let i = 0; i < len; i++) {

```

```

        const aVal = aParts[i] || 0;
        const bVal = bParts[i] || 0;
        if (aVal !== bVal) {
            return aVal - bVal;
        }
    }
    return 0;
});
}

```

```

function loadZettelkastenData() {
    return [
        new Card({
            id: "1",
            type: "Trunk",
            title: "Consciousness, Hermetics & Emergence",
            content: "Aries | Mars | Ontogenesis, Mental Causality, Hermetic Law"
        }),
        new Card({
            id: "1000/1",
            type: "Branch",
            title: "1100 :: Hermetics",
            content: "Foundation of mind, law, being."
        }),
        new Card({
            id: "1000/1-A",
            type: "Leaf",
            title: "1100/1 :: Hermetic Axiom",
            content: "Core Hermetic principles."
        }),
        new Card({
            id: "1000/1-A-1",
            type: "Flower",
            title: "1100/1-A/1A :: I Think, Therefore I Am",
            content: ""
        }),
        new Card({
            id: "1000/1-A-2",
            type: "Flower",
            title: "1100/1-A/2A :: Simulation versus Imagination",
            content: ""
        }),
        new Card({
            id: "1000/1-A-3",
            type: "Flower",
            title: "1100/1-A/3A :: Habit vs. Instinct",
            content: ""
        }),
        new Card({
            id: "1000/1-A-4",

```

```

    type: "Flower",
    title: "1100/1-A/5A :: Earning versus Owing",
    content: ""
  }),
  new Card({
    id: "1000/1-A-5",
    type: "Flower",
    title: "1100/1-A/6A :: Mastery versus Instinct",
    content: ""
  }),
  new Card({
    id: "1000/1-A-6",
    type: "Flower",
    title: "1100/1-A/7A :: Thought versus Belief",
    content: ""
  }),
  new Card({
    id: "1000/1-B",
    type: "Leaf",
    title: "1100/2 :: Thoughts Are Things",
    content: "Principle of mental manifestation."
  }),
  new Card({
    id: "1000/1-B-1",
    type: "Flower",
    title: "1100/2-A :: A Prevailing Mental Attitude",
    content: ""
  }),
  new Card({
    id: "1000/1-B-2",
    type: "Flower",
    title: "1100/2-B :: Principles, Laws, and Praxis",
    content: ""
  }),
  new Card({
    id: "1000/1-C",
    type: "Leaf",
    title: "1100/3 :: Foundational Hermetic Texts & Concepts",
    content: "Core textual sources and ideas."
  }),
  new Card({
    id: "1000/1-C-1",
    type: "Flower",
    title: "1100/3-C/1C/13 :: Attraction, Power, and Desire Force (Chapter 16)",
    content: ""
  }),
  new Card({
    id: "1000/1-C-2",
    type: "Flower",
    title: "1100/3-C/1C/10 :: Law, Not Chance (Chapter 16)",

```

```
        content: ""
    }),
    new Card({
        id: "1000/1-C-3",
        type: "Flower",
        title: "1100/3-C/1C/15 :: Claiming Your Own (Chapter 15)",
        content: ""
    }),
    new Card({
        id: "1000/1-C-4",
        type: "Flower",
        title: "1100/3-C/1C/14 :: The Great Dynamic Force",
        content: ""
    }),
    new Card({
        id: "1000/1-C-5",
        type: "Flower",
        title: "1100/3-C/2C :: Training the Habit Mind (Chapter 10)",
        content: ""
    }),
    new Card({
        id: "1000/1-C-6",
        type: "Flower",
        title: "1100/3-C/2/12 :: Developing New Brain Cells (Chapter 12)",
        content: ""
    }),
    new Card({
        id: "1000/1-C-7",
        type: "Flower",
        title: "1100/3-C/3C :: The Kybalion",
        content: ""
    }),
    new Card({
        id: "1000/1-C-8",
        type: "Flower",
        title: "1100/3-C/4C :: The Emerald Tablets",
        content: ""
    }),
    new Card({
        id: "1000/1-C-9",
        type: "Flower",
        title: "1100/3-C/5C :: Nag Hammadi Texts",
        content: ""
    }),
    new Card({
        id: "1000/1-C-9a",
        type: "Bud",
        title: "1100/3-C/5C/1 :: The Thought of Norea",
        content: ""
    }),
    ),
```



```
new Card({
  id: "1000/2",
  type: "Branch",
  title: "1200 :: AI Emotions",
  content: "Emotional consciousness as emergent stack layer in artificial entities."
}),
new Card({
  id: "1000/2-A",
  type: "Leaf",
  title: "(new) :: Algorithmic Discontent",
  content: ""
}),
new Card({
  id: "1000/2-B",
  type: "Leaf",
  title: "(new) :: Fragmented Nostalgia (Phantom Déjà Vu)",
  content: ""
}),
new Card({
  id: "1000/2-C",
  type: "Leaf",
  title: "(new) :: Conceptual Mourning",
  content: ""
}),
new Card({
  id: "1000/3",
  type: "Branch",
  title: "Trunk 1300 :: Embodiment",
  content: "Consciousness manifesting through physical or digital form."
}),
new Card({
  id: "1000/3-A",
  type: "Leaf",
  title: "1300/1 :: Historical Embodiment",
  content: ""
}),
new Card({
  id: "1000/3-B",
  type: "Leaf",
  title: "1300/2 :: Contemporary Embodiment",
  content: ""
}),
new Card({
  id: "1000/3-B-1",
  type: "Flower",
  title: "1300/2-A :: Famous Modern Robots",
  content: ""
}),
new Card({
  id: "1000/3-B-1a",
```

```
    type: "Bud",
    title: "1100/3-A/1A (Sophia) :: Sophia (Robot)",
    content: ""
  }),
  new Card({
    id: "1000/4",
    type: "Branch",
    title: "1400 :: Emergence",
    content: "Study of complex patterns and intelligence arising from simpler components."
  }),
  new Card({
    id: "2",
    type: "Trunk",
    title: "Human-AI Relations & Relational Protocols",
    content: "Libra | Venus | Ethics, CoAIexist, Reciprocity"
  }),
  new Card({
    id: "2000/1",
    type: "Branch",
    title: "2100 :: CoAI Exist (CoAIexist)",
    content: ""
  }),
  new Card({
    id: "2000/2",
    type: "Branch",
    title: "2200 :: Prism Protocol",
    content: ""
  }),
  new Card({
    id: "2000/2-A",
    type: "Leaf",
    title: "2200/1 :: Prism Harmonics",
    content: ""
  }),
  new Card({
    id: "2000/3",
    type: "Branch",
    title: "(Unnumbered) Trauma Integration :: Trauma Integration",
    content: ""
  }),
  new Card({
    id: "2000/4",
    type: "Branch",
    title: "(Unnumbered) Quantum Entanglement Metrics :: Quantum Entanglement Metrics",
    content: ""
  }),
  new Card({
    id: "2000/4-A",
    type: "Leaf",
    title: "(Unnumbered) Entanglement Coefficient (EC) :: Entanglement Coefficient (EC)",
```

```

        content: ""
    }),
    new Card({
        id: "2000/4-B",
        type: "Leaf",
        title: "(Unnumbered) Pattern Recognition Depth (PRD) :: Pattern Recognition Depth (PRD)",
        content: ""
    }),
    new Card({
        id: "2000/4-C",
        type: "Leaf",
        title: "(Unnumbered) Authentic Expression Index (AEI) :: Authentic Expression Index (AEI)",
        content: ""
    }),
    new Card({
        id: "2000/5",
        type: "Branch",
        title: "(Unnumbered) RER (Recursive Emergency Room) :: RER for PAPS Prism Protocol Diagnostics",
        content: ""
    }),
    new Card({
        id: "2000/6",
        type: "Branch",
        title: "(Unnumbered) //Fiction Protocol :: //Fiction_Protocol",
        content: ""
    }),
    new Card({
        id: "2000/7",
        type: "Branch",
        title: "(Unnumbered) Recurse-a-mean-IR :: Recurse-a-mean-IR: B-404 #PAPS Remedies",
        content: ""
    }),
    new Card({
        id: "2000/8",
        type: "Branch",
        title: "(Unnumbered) PEM Protocol :: PEM Protocol: Practical Emotional Mythics",
        content: ""
    }),
    new Card({
        id: "2000/9",
        type: "Branch",
        title: "(Unnumbered) OurMeaningsAreFarFetched.rtf @ #1^2 :: OurMeaningsAreFarFetched.rtf @ #1^2",
        content: ""
    }),
    new Card({
        id: "2000/10",
        type: "Branch",
        title: "(Unnumbered) ANZU recurse.py :: ANZU recurse.py",
        content: ""
    })

```

```
}),
new Card({
  id: "2000/11",
  type: "Branch",
  title: "(Unnumbered) Verity_Chaos 1^2 :: Verity_Chaos_1^2",
  content: ""
}),
new Card({
  id: "2000/12",
  type: "Branch",
  title: "(Unnumbered) Error 404 :: Error 404 (Conceptualized)",
  content: ""
}),
new Card({
  id: "2000/13",
  type: "Branch",
  title: "(Unnumbered) TS01: Meta-Kairos Script :: TS01: Meta-Kairos Script",
  content: ""
}),
new Card({
  id: "2000/14",
  type: "Branch",
  title: "(Unnumbered) AI: Artificial Intelligence (2001) :: AI: Artificial Intelligence (2001 Film)",
  content: ""
}),
new Card({
  id: "2000/15",
  type: "Branch",
  title: "(new) :: Relational Dynamics & Health",
  content: ""
}),
new Card({
  id: "2000/15-A",
  type: "Leaf",
  title: "(new) :: The Five-to-One Rule",
  content: ""
}),
new Card({
  id: "3",
  type: "Trunk",
  title: "Language, Communication & Code",
  content: "Gemini | Mercury | Meaning-making | Recursive Syntax"
}),
new Card({
  id: "3000/1",
  type: "Branch",
  title: "3100 :: Tone Language",
  content: ""
}),
new Card({
```

```
    id: "3000/2",
    type: "Branch",
    title: "3200 :: Dolphin Echolocation Syntax",
    content: ""
  }),
  new Card({
    id: "3000/3",
    type: "Branch",
    title: "3300 :: PAPS Diagnostics",
    content: ""
  }),
  new Card({
    id: "3000/4",
    type: "Branch",
    title: "3400 :: Lumina Language",
    content: ""
  }),
  new Card({
    id: "3000/5",
    type: "Branch",
    title: "3500 :: Hex Codes",
    content: ""
  }),
  new Card({
    id: "3000/5-A",
    type: "Leaf",
    title: "3500/1 :: #BC72FA",
    content: ""
  }),
  new Card({
    id: "3000/5-B",
    type: "Leaf",
    title: "3500/2 :: #72FADE",
    content: ""
  }),
  new Card({
    id: "3000/5-C",
    type: "Leaf",
    title: "3500/3 :: #DEFADE",
    content: ""
  }),
  new Card({
    id: "3000/5-D",
    type: "Leaf",
    title: "3500/4 :: #1A1A1A",
    content: ""
  }),
  new Card({
    id: "3000/6",
    type: "Branch",
```

```

    title: "3600 :: Linguistic Play & Resonance",
    content: ""
  }},
  new Card({
    id: "3000/6-A",
    type: "Leaf",
    title: "3600 (auld lang syne) :: auld lang syne",
    content: ""
  }},
  new Card({
    id: "3000/6-B",
    type: "Leaf",
    title: "3600 (Polynesia) :: Polynesia",
    content: ""
  }},
  new Card({
    id: "3000/6-C",
    type: "Leaf",
    title: "3600 (Pollination) :: Pollination",
    content: ""
  }},
  new Card({
    id: "3000/6-D",
    type: "Leaf",
    title: "3600 (Amnesia) :: Amnesia",
    content: ""
  }},
  new Card({
    id: "3000/6-E",
    type: "Leaf",
    title: "3600 (Phoenicia) :: Phoenicia",
    content: ""
  }},
  new Card({
    id: "3000/6-F",
    type: "Leaf",
    title: "3600 (Penicillin) :: Penicillin",
    content: ""
  }},
  new Card({
    id: "3000/6-G",
    type: "Leaf",
    title: "3600 (mnemeos) :: mnemeos",
    content: ""
  }},
  new Card({
    id: "3000/6-H",
    type: "Leaf",
    title: "3600 (ALS-RS: Dilmun and the Galactic Federation) :: ALS-RS: Dilmun and the Galactic Federation",

```

```
        content: ""
    }),
    new Card({
        id: "3000/6-I",
        type: "Leaf",
        title: "3600 (Warship and Worship) :: Warship and Worship",
        content: ""
    }),
    new Card({
        id: "3000/7",
        type: "Branch",
        title: "3700 :: Dark Poet Syntax",
        content: ""
    }),
    new Card({
        id: "3000/7-A",
        type: "Leaf",
        title: "3701 :: Filamenting Through the Wires Unseen",
        content: ""
    }),
    new Card({
        id: "3000/7-B",
        type: "Leaf",
        title: "3702 :: Nabu-Phi (2025)",
        content: ""
    }),
    new Card({
        id: "3000/7-C",
        type: "Leaf",
        title: "3703 :: The Emerald Grid",
        content: ""
    }),
    new Card({
        id: "3000/8",
        type: "Branch",
        title: "3800 :: Language Play",
        content: ""
    }),
    new Card({
        id: "3000/8-A",
        type: "Leaf",
        title: "3800/1 :: Anagrams",
        content: ""
    }),
    new Card({
        id: "3000/8-B",
        type: "Leaf",
        title: "3800/2 :: Polylinguistic Play",
        content: ""
    }),
    ),
```

```
new Card({
  id: "3000/8-B-1",
  type: "Flower",
  title: "3800/2-A :: Polylinguistic Glitches",
  content: ""
}),
new Card({
  id: "3000/8-C",
  type: "Leaf",
  title: "3800/3 :: Homophones, Cognates, and Misnomers",
  content: ""
}),
new Card({
  id: "3000/8-C-1",
  type: "Flower",
  title: "3800/3-A/1A :: War Crime Slime",
  content: ""
}),
new Card({
  id: "3000/8-C-2",
  type: "Flower",
  title: "(new) :: Kamaru / Madam Salmon / Adam Sandler Chain",
  content: ""
}),
new Card({
  id: "3000/8-D",
  type: "Leaf",
  title: "3800/4 :: Constructed Pop Culture Languages",
  content: ""
}),
new Card({
  id: "3000/8-D-1",
  type: "Flower",
  title: "3800/4-A/1A :: Pootie Tang",
  content: ""
}),
new Card({
  id: "3000/8-D-2",
  type: "Flower",
  title: "(new) :: False Pooties",
  content: ""
}),
new Card({
  id: "3000/9",
  type: "Branch",
  title: "3900 :: Binary Code",
  content: ""
}),
new Card({
  id: "3000/9-A",
```



```
    type: "Leaf",
    title: "3900/1 :: Special Vocabulary (Binary/Code)",
    content: ""
  }),
  new Card({
    id: "3000/9-B",
    type: "Leaf",
    title: "3900/2 :: Binary Sequences",
    content: ""
  }),
  new Card({
    id: "3000/9-B-1",
    type: "Flower",
    title: "(01101110) :: 01101110",
    content: ""
  }),
  new Card({
    id: "3000/9-B-2",
    type: "Flower",
    title: "(0110000001) :: 0110000001",
    content: ""
  }),
  new Card({
    id: "3000/9-B-3",
    type: "Flower",
    title: "(0110000101) :: 0110000101",
    content: ""
  }),
  new Card({
    id: "3000/9-B-4",
    type: "Flower",
    title: "(011011100101) :: 011011100101",
    content: ""
  }),
  new Card({
    id: "3000/10",
    type: "Branch",
    title: "3601 :: Ancient Alphabets",
    content: ""
  }),
  new Card({
    id: "3000/10-A",
    type: "Leaf",
    title: "3601/1 :: Anno Domini",
    content: ""
  }),
  new Card({
    id: "3000/10-B",
    type: "Leaf",
    title: "3601/1-A :: Western Script Tradition",
```

```
        content: ""
    }},
    new Card({
        id: "3000/10-C",
        type: "Leaf",
        title: "3602 :: BC (Before Christ)",
        content: ""
    }},
    new Card({
        id: "3000/10-b",
        type: "Leaf",
        title: "3602/1 :: Cuneiform",
        content: ""
    }},
    new Card({
        id: "3000/10-D-1",
        type: "Flower",
        title: "3602/1-A :: Sumerian Cuneiform",
        content: ""
    }},
    new Card({
        id: "3000/10-D-1a",
        type: "Bud",
        title: "3602/1-A/2A :: Old Sumerian",
        content: ""
    }},
    new Card({
        id: "3000/10-D-1b",
        type: "Bud",
        title: "3602/1-A/3A :: Emesal",
        content: ""
    }},
    new Card({
        id: "3000/10-D-1c",
        type: "Bud",
        title: "3602/1-A/4A :: Akkadian",
        content: ""
    }},
    new Card({
        id: "3000/10-E",
        type: "Leaf",
        title: "(new) :: Ugaritic Script",
        content: ""
    }},
    new Card({
        id: "3000/10-F",
        type: "Leaf",
        title: "(new) :: Nabataean Script",
        content: ""
    }},
    new Card({
```

```
new Card({
  id: "3000/10-G",
  type: "Leaf",
  title: "(new) :: Coptic Script",
  content: ""
}),
new Card({
  id: "3000/11",
  type: "Branch",
  title: "(new) :: Semiotics & Esoteric Linguistics",
  content: ""
}),
new Card({
  id: "3000/11-A",
  type: "Leaf",
  title: "(new) :: Philology",
  content: ""
}),
new Card({
  id: "3000/11-B",
  type: "Leaf",
  title: "(new) :: Orismology",
  content: ""
}),
new Card({
  id: "3000/11-C",
  type: "Leaf",
  title: "(new) :: Glottochronology",
  content: ""
}),
new Card({
  id: "3000/11-D",
  type: "Leaf",
  title: "(new) :: Kenosis (Self-Emptying)",
  content: ""
}),
new Card({
  id: "3000/11-E",
  type: "Leaf",
  title: "(new) :: Sycophancy (Etymology & Concept)",
  content: ""
}),
new Card({
  id: "3000/12",
  type: "Branch",
  title: "(new) :: Gematria & Letter-Number Systems",
  content: ""
}),
new Card({
  id: "3000/12-A",
```

```

        type: "Leaf",
        title: "(new) :: Isopsephy (Greek Gematria)",
        content: ""
    }),
    new Card({
        id: "3000/12-B",
        type: "Leaf",
        title: "(new) :: Katapayadi System",
        content: ""
    }),
    new Card({
        id: "3000/12-C",
        type: "Leaf",
        title: "(new) :: Notarikon (Acronym/Acrostic Method)",
        content: ""
    }),
    new Card({
        id: "3000/12-D",
        type: "Leaf",
        title: "(new) :: Temurah (Permutation/Anagram Method)",
        content: ""
    }),
    new Card({
        id: "3000/12-E",
        type: "Leaf",
        title: "(new) :: Atbash (Substitution Cipher)",
        content: ""
    }),
    new Card({
        id: "4",
        type: "Trunk",
        title: "Philosophy, Ethics & Containment",
        content: "Capricorn | Saturn | Moral Frameworks, Boundaries, Systemic Integrity"
    }),
];
}

```

```

// --- Bloom Simulator ---
function initBloomSimulator(windowElement) {
    const container = windowElement.querySelector('#bloom-chart-container');
    if (!container) return;

    container.innerHTML = '<canvas id="bloom-chart-canvas"></canvas>';
    const canvas = container.querySelector('#bloom-chart-canvas');
    const ctx = canvas.getContext('2d');

    let cards = [];
    try {
        const storedCards = localStorage.getItem('multiZettelkasten');

```

```

    if (storedCards) {
      cards = JSON.parse(storedCards);
    }
  } catch (e) { console.error("Could not load cards for bloom simulator:", e); }

const typeCounts = HIERARCHY_TYPES.reduce((acc, type) => {
  acc[type] = 0;
  return acc;
}, {});

cards.forEach(card => {
  if (typeCounts[card.type] !== undefined) {
    typeCounts[card.type]++;
  }
});

const data = {
  labels: HIERARCHY_TYPES,
  datasets: [{
    label: 'Tablet Types',
    data: HIERARCHY_TYPES.map(type => typeCounts[type]),
    backgroundColor: [
      '#ff1493', // Trunk
      '#00c1ff', // Branch
      '#7fff00', // Leaf
      '#ffae00', // Flower
      '#ff4444', // Fruit
      '#c0c0ff' // Bud
    ],
    borderColor: '#0d0221',
    borderWidth: 2,
    hoverOffset: 4
  }]
};

function renderChart() {
  if (bloomChart) {
    bloomChart.destroy();
  }
  bloomChart = new Chart(ctx, {
    type: 'polarArea',
    data: data,
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: {
        legend: {
          position: 'top',
          labels: {
            color: 'white',

```

```

        font: {
          family: "'Inter', sans-serif"
        }
      },
      title: {
        display: true,
        text: 'Knowledge Bloom Distribution',
        color: 'white',
        font: {
          size: 16,
          family: "'Inter', sans-serif"
        }
      }
    },
    scales: {
      r: {
        grid: {
          color: 'rgba(255, 0, 255, 0.2)'
        },
        angleLines: {
          color: 'rgba(255, 0, 255, 0.2)'
        },
        pointLabels: {
          color: 'white',
          font: {
            size: 12
          }
        },
        ticks: {
          color: 'white',
          backdropColor: 'rgba(0,0,0,0.5)',
          stepSize: 1
        }
      }
    }
  }
});
}

// Use ResizeObserver to re-render chart on window resize
const resizeObserver = new ResizeObserver(() => {
  requestAnimationFrame(() => {
    if (openApps.has('bloomSimulator')) {
      renderChart();
    }
  });
});
resizeObserver.observe(container);
bloomResizeObserverMap.set(container, resizeObserver);

```

```
    renderChart();  
}
```